

온라인 부록 B

R 입문 및 본문 코드 실습

본문 1-5부의 R 코드를 따라가기 위한 실습 부록

통통한 데이터 분석

서울대학교 과학데이터혁신연구소 · 통계학과

B.0 R 입문

이 부록은 본문의 1부부터 5부까지에 등장하는 R 코드를, 프로그래밍이 처음인 독자도 따라갈 수 있도록 안내하기 위해 마련되었다. 같은 분석을 파이썬으로 수행한 코드와 함께 본문에 실려 있으므로, 본문은 두 언어 중 익숙한 쪽을 선택해 따라갈 수 있게 구성되어 있다. 이 부록은 그중 R을 처음 만나는 독자를 위한 출입문이다.

실행용 파일 온라인부록B_R.Rmd를 책의 루트 폴더에 두었다. 루트에서 실행하면 모든 실습 데이터는 데이터/파일명.csv 경로로 불러온다.

부록의 기본 예제는 Posit Cloud에서 실행할 수 있으며, 일부 분석 패키지는 해당 절의 안내에 따라 추가 설치가 필요하다.

B.0.1 본문 1–5 부와의 연계

각 절의 끝에는 ► 본문 미리보기 박스가 있다. 그 박스에서는 "여기서 배운 내용이 본문 어디에서 어떻게 쓰이는지"를 한두 줄로 안내한다. 본문을 읽다가 이해되지 않는 R 코드를 만나면, 해당 박스를 역으로 찾아 부록의 해당 절로 돌아오면 된다.

► 본문 미리보기 — 이 부록 ↔ 본문

1장(시각화): tidyverse(B.7)와 ggplot2(B.8) 절을 미리 읽으면 본문이 한결 매끄럽다.

2부(모델링 1): lm 과 mgcv::gam 이 핵심이다. B.7과 B.9를 권한다.

3부(모델링 2): glm, MASS::lda, randomForest 등 다양한 분류 모형이 등장한다. B.9를 권한다.

4장(추론): rnorm, sample, replicate, t.test, aov, chisq.test 가 핵심이다. B.9가 가장 중요하다.

5장(패턴 탐색): kmeans, hclust, prcomp 등이 등장한다. B.9 후반부를 보면 된다.

B.0.2 R 환경 설정

R은 1990년대 초 뉴질랜드 오클랜드 대학교의 Ross Ihaka와 Robert Gentleman이 처음 만든 통계·자료 분석에 특화된 프로그래밍 언어이다. AT&T 벨 연구소의 S 언어를 뿌리로 하며, 다음과 같은 특징이 있다.

- 통계와 자료 분석을 "가장 자연스럽게" 표현하도록 설계되었다.
- 무료 오픈소스이며, 윈도우·맥·리눅스 등 모든 운영체제에서 동일하게 동작한다.

- CRAN(Comprehensive R Archive Network)에 22,000 개 이상의 패키지가 있어, 통계학·생물정보학·계량경제학 등 거의 모든 분야의 도구를 즉시 가져다 쓸 수 있다.
- ggplot2, dplyr 등 "tidyverse" 도구는 자료 변환과 시각화에 새로운 표준을 만들었다.
- R Markdown / Quarto 로 분석·문서·보고서를 한 파일로 묶을 수 있어, 재현 가능한 분석에 강하다.

R 을 실행하는 세 가지 환경

이 책의 모든 R 코드는 다음 세 환경 중 어느 것에서도 실행할 수 있다. 처음 시작하는 독자에게는 ❶ Posit Cloud를 가장 권장한다. 설치가 필요 없고, 본인의 노트북 사양과 무관하게 동일한 결과를 얻을 수 있기 때문이다.

- ❶ Posit Cloud — 브라우저에서 바로 RStudio 를 쓰는 클라우드 서비스. 무료 플랜이 있으며, 별도 설치가 필요 없다.
- ❷ R + RStudio 데스크톱 — R 본체와 가장 널리 쓰이는 통합 환경(IDE) RStudio 를 본인 컴퓨터에 설치한다.
- ❸ R + VS Code (또는 R + Jupyter) — 이미 익숙한 다른 에디터를 그대로 쓰고 싶을 때의 선택지이다.

Posit Cloud 로 시작하기 (권장)

Posit Cloud는 별도 설치 없이 웹 브라우저에서 곧장 R과 RStudio를 사용할 수 있는 환경이다. 작성한 코드("프로젝트")는 클라우드에 자동 저장된다.

준비물

- 이메일 계정 (구글·깃허브 계정으로 바로 가입 가능)
- 크롬·엣지·사파리 등 표준 웹 브라우저

처음 사용하는 절차

- ❶ 브라우저에서 <https://posit.cloud> 에 접속한다.
- ❷ [Sign Up]을 눌러 무료 계정을 만든다 (구글·깃허브 계정 연동 가능).
- ❸ [New Project → New RStudio Project]를 누르면 RStudio 화면이 그대로 열린다.
- ❹ 오른쪽 위의 콘솔(Console) 창에 다음을 입력하고 Enter 를 누른다.

```
print("Hello, R!")
1 + 2
```

아래와 같은 출력이 콘솔에 표시되면 정상적으로 실행된 것이다.

```
[1] "Hello, R!"
[1] 3
```

Tip

[1] 은 "이 줄의 첫 번째 원소부터" 라는 표시이다. R에서 모든 값은 사실 "길이가 1 이상인 벡터" 이기 때문에 매번 인덱스를 함께 보여 준다.

프로젝트 이름은 화면 좌상단의 [Untitled Project] 를 클릭해 바꿀 수 있다.

오른쪽 아래 [Files] 탭에서 자료(예: .csv) 파일을 직접 업로드할 수 있다.

스크립트 파일과 R 노트북

콘솔에 한 줄씩 입력하기보다, 스크립트 파일(.R)이나 R Markdown / Quarto 문서(.Rmd / .qmd)에 코드를 적어 두는 편이 훨씬 편리하다.

- **.R 스크립트** — 순수한 R 코드만 모아 두는 파일. 메뉴의 [File → New File → R Script] 로 만든다. 줄에 커서를 두고 Ctrl + Enter (맥은 Cmd + Enter)를 누르면 그 줄이 콘솔로 보내진다.
- **.Rmd / .qmd** — 코드, 결과, 글, 그래프를 한 문서에 함께 둘 수 있는 "노트북". 파이썬의 Jupyter Notebook 과 비슷한 역할을 한다.

내 컴퓨터에 설치하기 (R + RStudio)

장기적으로 자료 분석을 본격적으로 다루려면, 본인의 컴퓨터에 직접 설치하는 편이 편리하다. R 본체와 RStudio 두 가지를 차례로 설치한다.

R 본체 설치

- ❶ CRAN 미러 사이트(<https://cran.r-project.org>)에 접속한다.
- ❷ 본인의 운영체제(Windows / macOS / Linux)에 맞는 [Download R for ...]를 클릭한다.
- ❸ [base]를 클릭하고, 가장 위에 있는 최신 버전 설치 파일을 받아 실행한다 (수 분 소요).

RStudio Desktop 설치

- ❶ Posit 사이트(<https://posit.co/download/rstudio-desktop/>)에 접속한다.
- ❷ 본인의 운영체제에 맞는 무료 버전(RStudio Desktop Open Source) 설치 파일을 받아 실행한다.
- ❸ R 본체가 먼저 설치되어 있어야 RStudio 가 정상 동작한다. 두 단계의 순서를 지키자.

RStudio 의 4 분할 화면

RStudio 를 처음 열면 화면이 네 영역으로 나뉘어 있다.

- **좌상** — 스크립트/노트북 편집기
- **좌하** — 콘솔(Console). R 이 실제로 실행되는 곳
- **우상** — 환경(Environment) 탭. 만들어진 변수의 목록을 보여 준다.
- **우하** — 파일(Files) / 그래프(Plots) / 패키지(Packages) / 도움말(Help) 탭

자주 쓰는 단축키 (RStudio)

Ctrl + Enter (Cmd + Enter) — 현재 줄/선택 영역을 콘솔로 보내 실행

Ctrl + Shift + Enter — 스크립트 전체 실행

Alt + - (Option + -) — 할당 연산자 <- 자동 입력

Ctrl + Shift + M — 파이프 %>% (또는 |>) 자동 입력

Ctrl + L — 콘솔 비우기

패키지 설치하기

이 책에서 다루는 R 코드는 다음 패키지들을 사용한다. 이름과 역할을 가볍게 훑어 두자.

- tidyverse — dplyr, ggplot2, tidyr, readr 등을 한꺼번에 설치하는 "메타 패키지"
- data.table — 큰 자료를 빠르게 다루는 자료구조 (fread)
- GGally, ggcorrplot — ggpairs, 상관 히트맵 등 시각화 보조
- mgcv, splines, KernSmooth — 일반화가법모형, 스플라인, 커널 회귀
- MASS, class, e1071, randomForest, xgboost — 분류 모형들

패키지를 설치하려면 콘솔에 다음과 같이 입력한다. 한 번 설치하면 같은 컴퓨터에서는 다시 설치할 필요가 없다.

```
install.packages("tidyverse")
install.packages(c("data.table", "GGally", "mgcv"))
```

설치된 패키지는 사용하기 전에 library() 로 한 번씩 "불러와야" 한다. R 세션을 새로 시작할 때마다 library() 로 다시 불러주어야 한다.

```
library(tidyverse) # dplyr, ggplot2, readr 등이 한꺼번에 로딩된다
library(mgcv)
```

주의

Posit Cloud에는 tidyverse가 기본 설치되어 있지 않을 수도 있다. 처음 만든 프로젝트에서 `install.packages("tidyverse")` 를 한 번 실행하면 충분하다.

설치는 `install.packages("...")`, 사용은 `library(...)` 로 "이름의 따옴표"가 다르다는 점을 주의하자.

함수 이름이 어느 패키지의 것인지 분명히 하고 싶을 때는 패키지명::함수명 표기도 자주 쓰인다.

예: `dplyr::filter(...)`

패키지 불러와 쓰기

이 부록과 본문의 R 코드는 자료 분석에서 사실상 표준이 된 다음의 import 패턴을 따른다.

```
library(tidyverse)      # dplyr, ggplot2, tidyr, readr, ...
# 필요한 분석에 따라 추가로 로딩
library(mgcv)           # GAM
library(MASS)           # lda/qda
library(randomForest)
library(GGally)         # ggpairs
library(ggcorrplot)
```

앞으로 이 부록과 본문에 나오는 `ggplot`, `filter`, `mutate`, `%>%` 등은 모두 위 import 문이 먼저 실행됐다고 가정한다.

▶ 본문 미리보기 — 본문 1-5부 모두

위의 `library` 문은 사실상 본문 모든 챕터의 R 코드 첫 부분에 등장한다.

본문 코드를 따라 실행할 때는 그 챕터의 첫 셀(또는 스크립트 첫 부분)에 있는 `library` 문을 가장 먼저 실행해야 한다.

B.0.3 첫 코드: 출력, 주석, 변수

출력 — `print`, `cat`

R에서는 단순히 변수 이름만 적어도 그 값이 화면에 출력된다. 명시적으로 출력하려면 `print()` 또는 `cat()` 함수를 쓴다. `cat()` 은 따옴표나 `[1]` 같은 메타정보 없이 "보기 좋게" 출력해 준다.

```
print("Hello, R!")
1 + 2      # 자동 출력
print(3.14)
cat("π ≈", 3.14, "\n")  # \n 은 줄바꿈

[1] "Hello, R!"
[1] 3
[1] 3.14
π ≈ 3.14
```

주석 —

코드 일부를 설명하려면 # 기호 뒤에 글을 적는다. # 뒤의 내용은 실행되지 않는다. R은 다중 줄 주석 기호가 따로 없으므로, 여러 줄을 주석 처리하려면 각 줄 앞에 # 을 붙인다 (RStudio에서 Ctrl + Shift + C 로 한꺼번에 토글할 수 있다).

```
# 이 줄은 실행되지 않는다.
print("Hello")    # 줄 가운데에 써도 된다.
```

변수 — 할당 연산자 <-

R에서 가장 특징적인 표기는 할당 연산자 <- 이다. 화살표 모양처럼 "오른쪽 값을 왼쪽 이름에 넣는다"는 의미이다. = 도 사용할 수는 있지만, R 커뮤니티의 관습은 <- 를 쓰는 것이다.

```
x <- 3.14
y <- "Hello"

print(x)
print(y)

# RStudio에서는 Alt + - 단축키로 <- 가 자동으로 입력된다.
```

```
[1] 3.14
[1] "Hello"
```

변수 이름을 정할 때는 다음 규칙을 따른다.

- 영문자·숫자·점(.)·밑줄(_)을 조합해 만든다. 점(.) 도 변수 이름에 쓸 수 있다는 점이 R의 독특함이다.
- 숫자나 점 뒤의 숫자로 시작할 수 없다. (예: 1st 는 안 됨, x1 은 됨)
- 대소문자를 구분한다. (x 와 X 는 다른 변수)
- R 예약어(if, else, for, while, function, return, TRUE, FALSE, NA, NULL 등)는 변수명으로 쓸 수 없다.

이름 짓기 관습

R의 전통적 관습은 점으로 잇는 dot.case (예: total.count)이지만, 최근 tidyverse 진영은 snake_case (예: total_count) 를 권장한다.

대문자 카멜케이스(totalCount)는 R에서는 드물다. 본문에서는 snake_case 를 사용한다.

B.0.4 기본 자료형

R의 단일 값 자료형은 다음과 같다. 다만 R에서는 "단일 값"이라는 개념조차도 "길이가 1인 벡터"로 취급된다. 그래서 `print(3.14)`의 출력에 `[1] 3.14` 처럼 인덱스가 함께 나타난다.

- **숫자형(numeric, double)** — 실수. R의 기본 숫자 자료형이다.
- **정수형(integer)** — 정수. 명시적으로 만들려면 숫자 뒤에 `L`을 붙인다 (예: `3L`).
- **논리형(logical)** — `TRUE`, `FALSE` (단축형 `T`, `F`도 가능하지만 권장되지 않는다)
- **문자형(character)** — `"..."` 또는 `'...'`로 둘러싼 글자열
- **결측값(NA), 빈 값(NULL)** — R에서 가장 자주 만나는 "특별한 값"
- **날짜(Date), 시간(POSIXct)** — 날짜·시간 자료형

어떤 변수가 어떤 자료형인지 확인하려면 `class()` 또는 `typeof()` 함수를 쓴다.

```
class(3.14)
class(3L)
class(TRUE)
class("Hello")
class(NA)
class(NULL)

[1] "numeric"
[1] "integer"
[1] "logical"
[1] "character"
[1] "logical"
[1] "NULL"
```

숫자형과 산술 연산

R의 산술 연산자는 파이썬과 거의 같다. 단, 거듭제곱은 `^` (또는 `**`)으로 쓴다.

- `+` 더하기, `-` 빼기, `*` 곱하기, `/` 나누기
- `%%` 나머지, `/%` 몫(정수 나눗셈), `^` 또는 `**` 거듭제곱
- `==` 같다, `!=` 다르다, `<` `<=` `>` `>=` 대소 비교

```
a <- 7; b <- 3
a + b
a - b
a * b
a / b
a %% b      # 나머지
a %/% b     # 몫
a ^ b      # 거듭제곱

[1] 10
[1] 4
[1] 21
[1] 2.333333
[1] 1
```

```
[1] 2
[1] 343
```

실수의 자릿수 정리는 `round()` 함수로 한다.

```
round(3.1415, 2)
signif(0.0001234, 2)
```

```
[1] 3.14
[1] 0.00012
```

논리형

TRUE 와 FALSE 두 값으로 이뤄진 자료형이다. 비교 연산이나 조건 판단의 결과로 자주 등장하며, 산술 연산에서는 1 과 0 으로 동작한다.

```
3 > 2
3 == 4
```

```
# 논리 연산자: &(AND), |(OR), !(NOT). 벡터 연산용이며, 단일 값에서는 &&, || 도 사용된다.
TRUE & FALSE
TRUE | FALSE
!TRUE
```

```
# 논리값을 산술처럼 쓰기
sum(c(TRUE, FALSE, TRUE, TRUE)) # 조건이 참인 개수
mean(c(TRUE, FALSE, TRUE, TRUE)) # 비율
```

```
[1] TRUE
[1] FALSE
[1] FALSE
[1] TRUE
[1] FALSE
[1] 3
[1] 0.75
```

문자형

문자열은 따옴표로 둘러싼다. R은 인덱싱·슬라이싱이 직관적이지 않은 대신, 문자열 처리 함수가 풍부하다.

```
s <- "abcdef"
nchar(s) # 길이
substr(s, 2, 4) # 2~4번째 글자
toupper(s); tolower("HELLO")
paste("Hello,", "world") # 공백을 끼워 결합
paste0("Hello,", "world") # 공백 없이 결합
sprintf("x = %.2f, log = %.3f", 3.14, log(3.14))
```

```
[1] 6
[1] "bcd"
[1] "ABCDEF"
```

```
[1] "hello"
[1] "Hello, world"
[1] "Hello,world"
[1] "x = 3.14, log = 1.144"
```

sprintf — 형식 있는 출력

본문 4장의 검정 결과 출력에서 `sprintf("p = %.4f", p_value)` 같은 표현이 자주 등장한다. `%.4f` 는 "소수점 아래 4자리", `%d` 는 "정수", `%s` 는 "문자열" 자리표시자이다.

결측값(NA) 과 NULL

NA(Not Available) 는 R에서 "빠진 자료"를 표현하는 특별한 값이다. NULL 은 "아예 비어 있는 것"을 뜻한다. NA 가 섞인 벡터에 대해 산술 함수를 그대로 쓰면 결과도 NA 가 된다는 점을 기억하자.

```
x <- c(1, 2, NA, 4)
mean(x)
mean(x, na.rm = TRUE)      # NA를 무시하고 계산
is.na(x)                   # 어느 자리가 NA인지

length(NULL)               # NULL 의 길이는 0

[1] NA
[1] 2.333333
[1] FALSE FALSE TRUE FALSE
[1] 0
```

요인(factor) — 범주형 자료

factor 는 "고정된 종류의 라벨을 갖는 자료"를 표현하는 자료형이다. 통계 모형에 범주형 변수를 넣으면 R이 알아서 더미 변수를 만들어주는데, 이 때 factor 가 그 안내자 역할을 한다.

```
grade <- factor(c("B", "A", "C", "A", "B"),
               levels = c("C", "B", "A"))
grade
table(grade)
levels(grade)

[1] B A C A B
Levels: C B A

grade
C B A
1 2 2

[1] "C" "B" "A"
```

▶ 본문 미리보기 — 2부 모델링 1 / 3부 모델링 2

본문에서 `lm(log_salary ~ G + ERA + closer, ...)` 처럼 회귀식에 범주형 변수를 넣을 때, `closer` 가

factor 로 되어 있어야 R이 자동으로 더미 변수를 만든다.
as.factor() 로 문자형 열을 요인으로 바꿔 두는 코드가 자주 등장한다.

날짜와 시간

날짜는 Date, 시간이 함께 있는 자료는 POSIXct 클래스로 다룬다.

```
today <- as.Date("2026-04-26")
class(today)
today + 5                # 5일 뒤
format(today, "%Y년 %m월 %d일")
as.numeric(today)        # 1970-01-01 부터의 일수

[1] "Date"
[1] "2026-05-01"
[1] "2026년 04월 26일"
[1] 20569
```

B.0.5 자료 구조

앞 절의 자료형들이 "값 한 개"를 담는 그릇이라면, 자료 구조는 "여러 값을 묶는" 그릇이다. R에서 가장 자주 쓰는 네 가지를 살펴본다.

벡터 — c()

R의 가장 기본적인 자료 구조는 "벡터(vector)"이다. 같은 자료형의 값을 일렬로 묶어 놓은 것으로, c() (combine) 함수로 만든다.

```
scores <- c(80, 90, 75, 100, 65)
scores
length(scores)
scores[1]                # R의 인덱스는 1부터 시작 !
scores[5]                # 마지막
scores[2:4]              # 2번부터 4번까지
scores[-1]               # 첫 원소 "제외"

[1] 80 90 75 100 65
[1] 5
[1] 80
[1] 65
[1] 90 75 100
[1] 90 75 100 65
```

R 인덱싱은 1부터

파이썬과 가장 헷갈리는 차이이다. R은 첫 원소가 [1] 이고, 음수 인덱스는 "제외"의 뜻이다.
x[-1] 은 파이썬처럼 "마지막 원소"가 아니라 "첫 원소를 뺀 나머지" 이다.

벡터 연산 — 자동 원소별 적용

R의 매력 중 하나는, 벡터끼리의 연산이 별다른 표시 없이 "원소별로" 자동 적용된다는 점이다. 같은 일을 파이썬 리스트에서는 NumPy 배열로 바꿔야 하지만, R에서는 처음부터 그렇게 동작한다.

```
x <- c(1, 2, 3, 4)
y <- c(10, 20, 30, 40)

x + y
x * y
x ^ 2
sqrt(x)

x + 100          # 스칼라는 모든 원소에 적용
(x - mean(x)) / sd(x)  # 표준화
```

```
[1] 11 22 33 44
[1] 10 40 90 160
[1] 1 4 9 16
[1] 1.000000 1.414214 1.732051 2.000000
[1] 101 102 103 104
[1] -1.1618950 -0.3872983 0.3872983 1.1618950
```

자주 쓰는 벡터 생성 함수

```
1:10          # 1부터 10까지의 정수
seq(0, 1, by = 0.25)  # 등간격 수열
seq(0, 1, length.out = 5)
rep("a", 4)      # 같은 값 반복
rep(1:3, times = 2)
rep(1:3, each = 2)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
[1] 0.00 0.25 0.50 0.75 1.00
[1] 0.00 0.25 0.50 0.75 1.00
[1] "a" "a" "a" "a"
[1] 1 2 3 1 2 3
[1] 1 1 2 2 3 3
```

불리언 마스크와 통계 함수

```
a <- c(3, 1, 4, 1, 5, 9, 2, 6, 5)

sum(a)
mean(a); median(a)
sd(a); var(a)
min(a); max(a); range(a)
quantile(a, c(0.25, 0.5, 0.75))

a > 3
a[a > 3]          # 조건을 만족하는 원소만
```

```
[1] 36
```

```
[1] 4
[1] 4
[1] 2.645751
[1] 7
[1] 1
[1] 9
[1] 1 9
25% 50% 75%
  2   4   5
[1] FALSE FALSE TRUE FALSE TRUE TRUE FALSE TRUE TRUE
[1] 4 5 9 6 5
```

리스트(list)

벡터는 같은 자료형의 값만 담을 수 있는 반면, 리스트는 종류가 다른 값들을 함께 담을 수 있다. 함수가 여러 종류의 결과를 함께 돌려줄 때(예: `lm()`의 결과)도 리스트로 반환된다.

```
person <- list(name = "Hong", age = 30, scores = c(80, 90, 75))
person$name           # $로 이름 접근
person[["age"]]        # [[ ]]도 가능
person$scores[1]
names(person)
```

```
[1] "Hong"
[1] 30
[1] 80
[1] "name" "age" "scores"
```

데이터프레임(data.frame)

데이터프레임은 R 자료 분석의 주인공이다. "각 열이 같은 자료형의 벡터이고, 모든 열의 길이가 같은 표"이다.

```
df <- data.frame(
  이름 = c("민수", "수민", "지영", "지수"),
  키 = c(175, 168, 162, 180),
  몸무게 = c(68, 55, 50, 78)
)
df
nrow(df); ncol(df); dim(df)
str(df)
summary(df)
```

```
이름   키   몸무게
1 민수 175    68
2 수민 168    55
3 지영 162    50
4 지수 180    78
[1] 4
[1] 3
[1] 4 3
'data.frame': 4 obs. of 3 variables:
 $ 이름 : chr "민수" "수민" "지영" "지수"
 $ 키 : num 175 168 162 180
```

```
$ 몸무게: num 68 55 50 78
```

열·행·셀 접근하기

```
df$키          # 한 열 (벡터로 반환)
df[, "몸무게"]  # 같은 결과
df[2, ]        # 두 번째 행 전체
df[df$키 > 170, ] # 키가 170 초과인 행
df[, c("이름", "키")] # 두 열만
```

tibble 이라는 "현대적 데이터프레임"

tidyverse 가 사용하는 tibble 은 data.frame 의 개선판이다. read_csv() 의 결과는 tibble 이다. 기본 동작은 같지만, 출력이 더 깔끔하고, 부분 매칭("키"의 일부만 적어도 자동 인식하는 동작)을 하지 않아 더 안전하다.

행렬(matrix) — 같은 자료형의 2 차원

행렬은 모든 원소가 같은 자료형(보통 숫자)인 2차원 자료 구조이다. 회귀의 설계행렬, scikit-learn 품의 X 행렬을 다룰 때 자주 등장한다.

```
M <- matrix(1:12, nrow = 3, byrow = TRUE)
M
M[2, 3]
M[, 2]          # 두 번째 열 (벡터)
t(M)            # 전치
M %% t(M)       # 행렬 곱
```

B.0.6 제어 흐름과 함수

R도 다른 언어처럼 조건문·반복문·함수를 갖고 있지만, 일상적인 자료 분석에서는 "벡터 연산"과 "apply 가족"으로 같은 일을 더 짧게 표현하는 경우가 많다.

조건문 — if / else if / else

```
x <- 75
if (x >= 90) {
  grade <- "A"
} else if (x >= 80) {
  grade <- "B"
} else if (x >= 70) {
  grade <- "C"
} else {
  grade <- "D"
}
grade
```

벡터 전체에 조건을 한 번에 적용하려면 ifelse() 또는 dplyr::case_when() 을 쓴다.

```
scores <- c(95, 82, 75, 60)
ifelse(scores >= 80, "Pass", "Fail")
```

```
# 조건이 셋 이상이면 case_when
library(dplyr)
case_when(
  scores >= 90 ~ "A",
  scores >= 80 ~ "B",
  scores >= 70 ~ "C",
  TRUE      ~ "D"
)
```

```
[1] "Pass" "Pass" "Fail" "Fail"
[1] "A" "B" "C" "D"
```

반복문 — for, while

```
for (s in c(80, 90, 75, 100, 65)) {
  print(s)
}

for (i in 1:3) print(paste("반복", i))

n <- 1
while (n < 100) {
  n <- n * 2
}
n
```

apply 가족 — 반복문을 함수형으로

R 사용자는 명시적인 for 문보다 sapply, lapply, vapply 를 즐겨 쓴다. "각 원소에 함수를 적용"하는 패턴은 한 줄로 표현된다.

```
# 1부터 5까지에 함수를 적용해 벡터로 반환
sapply(1:5, function(x) x^2)

# 행렬 행/열 단위로 적용
M <- matrix(1:6, nrow = 2)
apply(M, 1, sum)      # 행 합
apply(M, 2, sum)      # 열 합

# 리스트로 반환하려면 lapply
lapply(c(80, 90, 75), function(x) x + 10)
```

▶ 본문 미리보기 — 4장 추론 / 부트스트랩

4장에서는 부트스트랩 표본 1,000개의 평균을 만들 때 replicate(1000, mean(sample(x, replace = TRUE))) 가 자주 등장한다. replicate() 도 사실은 sapply 의 친척이다.

함수 정의 — function

자주 쓰는 작업은 함수로 묶어 두면 같은 코드를 다시 쓸 필요가 없다.

```
average <- function(xs) {
  sum(xs) / length(xs)
}

grade <- function(x, cutoffs = c(90, 80, 70)) {
  if (x >= cutoffs[1]) "A"
  else if (x >= cutoffs[2]) "B"
  else if (x >= cutoffs[3]) "C"
  else "D"
}

average(c(80, 90, 75, 100, 65))
grade(85)
grade(72, cutoffs = c(95, 85, 75))
```

```
[1] 82
[1] "B"
[1] "D"
```

함수의 마지막 표현식이 자동으로 반환값이 된다. 명시적으로 `return()` 을 써도 무방하다. 인자에는 기본값을 줄 수 있고, 호출할 때 이름을 지정해 전달할 수도 있다.

익명 함수와 $w(x)$ 단축 문법

R 4.1 이상에서는 `function(x) ...` 의 단축형 `w(x) ...` 을 쓸 수 있다. `dplyr`의 `mutate` / `summarise`, `sapply` 등에 자주 쓰인다.

```
sapply(1:5, \(x) x^2)

# tidyverse 와 함께 쓰면 자연스럽다
library(dplyr)
tibble(x = 1:5) |>
  mutate(square = sapply(x, \(v) v^2))
```

▶ 본문 미리보기 — 1장 시각화 / 4장 추론

본문에서는 `mutate(분 = (시간 %% 60), 초 = (시간 %% 60))` 처럼 `mutate()` 안에서 새 열을 만드는 코드가 자주 등장한다.

이 부록의 B.7 `dplyr` 절에서 더 자세히 다룬다.

B.0.7 base R 로 자료 다루기

R에는 데이터프레임을 다루기 위한 두 갈래의 도구가 있다. 하나는 R에 처음부터 들어 있는 "base R" 함수이고, 다른 하나는 "tidyverse"라는 현대적인 도구 모음이다. 본문은 주로 tidyverse를 사용하지만,

두 도구가 함께 등장할 때도 있으므로 base R도 가볍게 익혀 두자.

CSV 파일 읽기

실제 자료는 보통 CSV(.csv) 또는 Excel(.xlsx) 파일로 주어진다. base R의 read.csv() 또는 readr 패키지의 read_csv() 한 줄로 데이터프레임으로 불러올 수 있다.

```
# base R
df <- read.csv("데이터/iris.csv")
head(df)           # 처음 6행
str(df)            # 구조 요약 (열 자료형)
summary(df)        # 수치 열 요약 통계
dim(df)            # 행 수, 열 수
names(df)          # 열 이름들

# tidyverse 방식 (권장)
library(tidyverse)
df <- read_csv("데이터/iris.csv") # tibble 로 반환된다
glimpse(df)
```

한글이 포함된 CSV

한글 헤더가 깨지면 인코딩을 지정한다.

```
base R: read.csv("file.csv", fileEncoding = "UTF-8")
readr: read_csv("file.csv", locale = locale(encoding = "UTF-8"))
```

base R 로 행과 열 선택하기

```
# 한 열만 (벡터로 반환)
df$Sepal.Length

# 여러 열 (데이터프레임)
df[, c("Sepal.Length", "Species")]

# 조건으로 행 선택
setosa <- df[df$Species == "setosa", ]
nrow(setosa)

# 조건 결합: & (AND), | (OR)
big <- df[(df$Sepal.Length > 6.0) & (df$Species == "virginica"), ]
```

base R에서 자주 쓰는 또 하나의 함수가 subset() 이다. "열 이름을 따옴표 없이" 쓸 수 있어 가독성이 좋다.

```
subset(df, Species == "setosa", select = c(Sepal.Length, Sepal.Width))
```

base R 의 그룹별 집계 — aggregate

```
aggregate(Sepal.Length ~ Species, data = df, FUN = mean)
aggregate(cbind(Sepal.Length, Sepal.Width) ~ Species, data = df, FUN = mean)
```

이 정도까지가 base R로 자주 쓰는 자료 처리이다. 자료가 조금만 복잡해져도 코드가 길어지므로, 본문은 다음 절의 dplyr 를 주로 쓴다.

B.0.8 tidyverse — dplyr 와 파이프로 데이터 다루기

tidyverse 는 자료 분석에 자주 쓰는 패키지들을 모은 "메타 패키지"이다. 그중에서도 dplyr 의 다섯 가지 "동사"와 파이프 연산자가 데이터 처리의 거의 모든 일을 담당한다.

```
library(tidyverse)
```

파이프 — %>% 와 |>

파이프는 "왼쪽의 결과를 오른쪽 함수의 첫 번째 인자로 넘긴다" 라는 단순한 규칙이다. 코드 흐름을 "왼쪽에서 오른쪽으로, 위에서 아래로" 읽을 수 있게 해 준다.

```
# 같은 일을 두 가지 방식으로
mean(sqrt(c(1,4,9,16)))

c(1,4,9,16) %>% sqrt() %>% mean()
c(1,4,9,16) |> sqrt() |> mean()
```

```
[1] 2.5
[1] 2.5
[1] 2.5
```

%>% vs |>

%>% 는 magrittr / dplyr 가 제공하는 "전통적" 파이프이다. tidyverse 를 로딩하면 자동으로 사용 가능.

|> 는 R 4.1부터 기본 제공되는 "새 파이프"이다. 본문은 두 가지 모두 사용한다.

RStudio 단축키: Ctrl+Shift+M (옵션에서 |> 와 %>% 중 택할 수 있다).

dplyr 의 다섯 가지 동사

- **select()** — 열 고르기 (열 추출)
- **filter()** — 행 고르기 (조건으로 추출)
- **mutate()** — 새 열 만들기 또는 기존 열 변경
- **arrange()** — 행 정렬
- **summarise()** — 여러 행을 하나로 요약 (group_by 와 짝꿍)

이 다섯 동사는 "데이터프레임을 받아 데이터프레임을 돌려준다" 라는 같은 모양을 갖기 때문에, 파이프로 자연스럽게 이어진다.

```
iris_tb <- as_tibble(iris)

iris_tb %>%
  dplyr::select(Species, Sepal.Length, Sepal.Width) %>%
  filter(Sepal.Length > 6) %>%
  mutate(Sepal.Area = Sepal.Length * Sepal.Width) %>%
  arrange(desc(Sepal.Area)) %>%
  slice_head(n = 5)
```

group_by 와 summarise — 그룹별 요약

"성별로", "국가별로", "품종별로" 평균을 구하는 식의 "분리-적용-결합" 작업은 `group_by` → `summarise` 한 줄로 표현된다.

```
iris_tb %>%
  group_by(Species) %>%
  summarise(
    mean_sl = mean(Sepal.Length),
    sd_sl   = sd(Sepal.Length),
    n       = n()
  )
```

```
# A tibble: 3 × 4
  Species    mean_sl sd_sl    n
  <fct>      <dbl> <dbl> <int>
1 setosa      5.01 0.352   50
2 versicolor  5.94 0.516   50
3 virginica   6.59 0.636   50
```

▶ 본문 미리보기 — 1장 시각화 / 4장 추론

1장에서는 배급사·연도별 평균 관객 수를 계산하는 코드에 `group_by` + `summarise` 가 등장한다. 4장에서는 보스턴 마라톤 자료를 연령 그룹·성별로 묶어 평균 완주 시간을 비교할 때 같은 패턴이 쓰인다.

두 표를 합치기 — *_join, bind_rows

열 이름 중 하나를 기준으로 두 데이터프레임을 "가로로" 합치려면 `*_join`, 같은 모양을 "세로로" 쌓으려면 `bind_rows` 를 쓴다.

```
left  <- tibble(id = 1:3, score = c(80, 90, 70))
right <- tibble(id = c(1, 2, 4), name = c("민수", "수민", "지영"))

inner_join(left, right, by = "id")  # 둘 다 가진 id만
left_join(left, right, by = "id")  # 왼쪽 기준
full_join(left, right, by = "id")  # 합집합

bind_rows(left, left)              # 세로 결합
```

long ↔ wide — pivot_longer / pivot_wider

"세로로 길게" 와 "가로로 넓게" 사이를 오가는 변형은 시각화나 모델링 입력을 만들 때 자주 필요하다.

```
wide <- tibble(
  반      = c("1반", "2반"),
  국어    = c(85, 78),
  영어    = c(90, 82),
  수학    = c(76, 88)
)

long <- wide %>%
  pivot_longer(cols = c(국어, 영어, 수학),
               names_to = "과목", values_to = "점수")
long

# 다시 wide 로
long %>% pivot_wider(names_from = 과목, values_from = 점수)
```

결측값 처리

```
x <- c(1, 2, NA, 4)
mean(x, na.rm = TRUE)

# 데이터프레임에서 결측값이 있는 행 제거
df %>% drop_na()
df %>% drop_na(Sepal.Length)      # 특정 열 기준

# 결측값을 다른 값으로 대체
df %>% mutate(Sepal.Length = replace_na(Sepal.Length, 0))
```

B.0.9 ggplot2 — 레이어 문법으로 그래프 그리기

ggplot2 는 "그래픽 문법(grammar of graphics)" 이라는 일관된 사고로 그래프를 만든다. 거의 모든 그림은 다음 세 가지를 정해 주면 만들어진다.

- **data** — 어느 데이터프레임을 그릴까?
- **aes()** — 어떤 변수가 x, y, 색, 모양 등에 "매핑"될까?
- **geom_*()** — 점, 선, 막대, 히스토그램 등 어떤 "기하 객체"로 표현할까?

첫 번째 그림 — 산점도

```
library(ggplot2)

ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(
```

```

title = "Iris - 꽃받침 길이 vs 너비",
x = "Sepal Length (cm)",
y = "Sepal Width (cm)",
color = "품종"
) +
theme_minimal()

```

ggplot2의 핵심 — +로 레이어를 더한다

ggplot(...)으로 "빈 캔버스"를 만들고, geom_*() / labs() / theme_*()를 +로 차례차례 더한다.

한 그림에 점·선·구간 추정 등을 함께 그리려면 geom_point() + geom_smooth()처럼 단순히 더하면 된다.

한글 폰트가 깨질 때: theme(text = element_text(family = "AppleGothic")) // 맥, 또는 "Malgun Gothic" // 윈도우

자주 쓰는 geom 들

히스토그램과 밀도

```

ggplot(iris, aes(x = Sepal.Length)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +
  labs(x = "Sepal Length", y = "count")

# 품종별로 색을 달리하여 밀도
ggplot(iris, aes(x = Sepal.Length, fill = Species)) +
  geom_density(alpha = 0.4) +
  theme_minimal()

```

상자그림

```

ggplot(iris, aes(x = Species, y = Sepal.Length, fill = Species)) +
  geom_boxplot() +
  theme_minimal() +
  theme(legend.position = "none")

```

산점도 + 회귀곡선

```

ggplot(iris, aes(x = Sepal.Length, y = Petal.Length, color = Species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal()

```

막대그래프 — 그룹 평균

```

iris %>%
  group_by(Species) %>%
  summarise(mean_sl = mean(Sepal.Length)) %>%
  ggplot(aes(x = Species, y = mean_sl, fill = Species)) +
  geom_col() +
  geom_text(aes(label = round(mean_sl, 2)), vjust = -0.4) +
  ylim(0, 8) +

```

```
theme_minimal()
```

패싯(facet) — 한 그림에 여러 패널

"같은 종류의 그림을 그룹별로 따로따로" 그리고 싶을 때 `facet_wrap()` / `facet_grid()` 가 강력하다.

```
ggplot(iris, aes(x = Sepal.Length, y = Petal.Length)) +
  geom_point(alpha = 0.6) +
  facet_wrap(~ Species, ncol = 3) +
  theme_minimal()
```

색·테마·라벨 정돈

```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_brewer(palette = "Set2") +      # 색 팔레트
  labs(title = "Iris", subtitle = "꽃받침 길이 vs 너비",
        x = "길이 (cm)", y = "너비 (cm)") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```

▶ 본문 미리보기 — 1장 시각화 — 거의 전체

1장은 ggplot2 를 가장 적극적으로 쓰는 장이다. `geom_histogram`, `geom_density`, `geom_boxplot`, `geom_point`, `geom_smooth`, `geom_col`, `geom_tile`, `geom_mosaic` 등이 모두 등장한다.

`ggExtra`, `ggmosaic` 같은 보조 패키지가 보조적으로 쓰이지만, 핵심은 위에서 본 "data + aes() + geom_*" 의 조합이다.

▶ 본문 미리보기 — 4장 추론

4장에서는 부트스트랩 분포를 `geom_histogram` + `geom_vline` 으로 그리고, 신뢰구간 경계를 `vline` 으로 표시한다.

표본 평균의 분포에 정규곡선을 겹쳐 그릴 때는 `stat_function(fun = dnorm, ...)` 가 자주 쓰인다.

B.0.10 통계와 모델링

R은 통계학자가 만들고, 통계학자가 가장 먼저 쓰는 언어다. 이 절에서는 본문 4장의 추론 코드와 2부·3부·5장의 모델링 코드를 따라가는 데 필요한 핵심 함수들을 "맛보기" 수준으로 정리한다.

확률 분포 다루기 — d/p/q/r 패밀리

R에서는 모든 확률 분포가 네 가지 형태의 함수로 일관되게 제공된다.

- **d*** — 확률밀도(또는 확률질량)함수 (예: `dnorm`)
- **p*** — 누적확률함수 (`pnorm`)

- **q*** — 분위수함수 (qnorm)
- **r*** — 난수 생성 (rnorm)

```
# 표준정규
dnorm(0)           # f(0)
pnorm(1.96)        # P(Z <= 1.96)
qnorm(0.975)       # 97.5% 분위수 ≈ 1.96

# 평균 170, 표준편차 10 인 정규분포에서 표본 100개
set.seed(0)
sample <- rnorm(100, mean = 170, sd = 10)
head(round(sample, 2))

[1] 0.3989423
[1] 0.9750021
[1] 1.959964
[1] 182.62 166.74 183.30 182.95 174.13 182.69
```

이항분포·t 분포·카이제곱분포도 같은 패턴으로 쓸 수 있다 (rbinom, qt, pchisq 등). "분포 이름"을 외우면 어떤 통계 계산도 한 줄로 끝난다.

가설검정

R의 검정 함수는 결과를 보기 좋게 출력해 주는 동시에 결과 객체로도 반환한다. 결과의 P-값·신뢰구간 등은 \$ 로 꺼내 쓸 수 있다.

일표본 t-검정

```
t.test(sample, mu = 170)

One Sample t-test
data: sample
t = 1.18, df = 99, p-value = 0.24
alternative hypothesis: true mean is not equal to 170
95 percent confidence interval:
 168.05 172.36
sample estimates:
mean of x
 170.20
```

이표본 t-검정

```
set.seed(1); x <- rnorm(30, 10, 2)
set.seed(2); y <- rnorm(30, 12, 2)

t.test(x, y, var.equal = FALSE)
```

카이제곱 독립성 검정

```
tab <- matrix(c(30, 20, 10,
                25, 35, 20),
              nrow = 2, byrow = TRUE)
tab
chisq.test(tab)
```

일원분산분석(ANOVA)

```
m <- aov(Sepal.Length ~ Species, data = iris)
summary(m)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Species	2	63.21	31.61	119.3	<2e-16 ***
Residuals	147	38.96	0.27		

▶ 본문 미리보기 — 4장 추론 — 거의 전 절

위의 t.test / chisq.test / aov 가 4장 추론 코드 대부분의 핵심을 이룬다.

신뢰구간을 부트스트랩으로 만들 때는 quantile(boot_means, c(0.025, 0.975)) 가 자주 쓰인다.

부트스트랩과 시뮬레이션

R에서 부트스트랩이나 몬테카를로 시뮬레이션은 sample(..., replace = TRUE) 와 replicate() 두 함수의 조합으로 깔끔하게 표현된다.

```
set.seed(42)
x <- rnorm(50, mean = 100, sd = 15)

# 평균의 부트스트랩 분포 (1000회)
boot_means <- replicate(1000, mean(sample(x, replace = TRUE)))

# 표본 평균과 부트스트랩 95% 신뢰구간
mean(x)
quantile(boot_means, c(0.025, 0.975))

[1] 99.79
    2.5% 97.5%
95.99 103.97
```

선형회귀와 로지스틱 회귀 — lm, glm

R의 "식(formula)" 표기 $y \sim x_1 + x_2$ 는 R의 가장 큰 매력 중 하나다. 회귀모형, ANOVA, 일반화선형 모형 등이 모두 같은 표기를 공유한다.

선형회귀

```
set.seed(0)
n <- 100
x <- rnorm(n)
y <- 2 + 3 * x + rnorm(n)
```

```
df <- data.frame(x = x, y = y)

m <- lm(y ~ x, data = df)
summary(m)
coef(m)
confint(m)
predict(m, newdata = data.frame(x = c(-1, 0, 1)),
       interval = "confidence")
```

범주형 변수와 다중회귀

```
df$g <- factor(ifelse(df$x > 0, "A", "B"))

m2 <- lm(y ~ x + g, data = df)
summary(m2)$coefficients
```

R은 factor 형 열을 자동으로 "기준 수준 대비 더미" 형태로 코딩해 준다. 식의 우변에 별다른 표시 없이 그대로 쓰면 된다.

로지스틱 회귀

```
df$y_bin <- ifelse(df$y > median(df$y), 1, 0)

log_m <- glm(y_bin ~ x, data = df, family = binomial)
summary(log_m)$coefficients

predict(log_m, newdata = data.frame(x = c(-1, 0, 1)), type = "response")
```

▶ 본문 미리보기 — 2부 모델링 1 / 3부 모델링 2

2부에서는 야구 투수의 연봉 자료에 `lm(log_salary ~ G + ERA + closer, data = df)` 형태로 회귀를 적합한다.

3부에서는 `glm(diabetes ~ ., data = df, family = binomial)` 형태의 로지스틱 회귀로 분류를 다룬다.

비모수·고급 모형들

본문 2부에서는 직선이 아닌 매끄러운 곡선을 적합하기 위해 다음 도구들을 사용한다.

```
library(splines)
m_bs <- lm(y ~ bs(x, df = 4), data = df)

ss <- smooth.spline(df$x, df$y, df = 6)

library(mgcv)
m_gam <- gam(y ~ s(x, k = 8), data = df)
summary(m_gam)
```

스플라인의 직관

B-스플라인이나 GAM의 $s(x)$ 는 "x 를 매끄럽게 휘어진 곡선으로 적합한다" 라고 이해하면 된다. 일반 $\text{lm}(y \sim x)$ 가 $y = a + b \cdot x$ 라는 직선을 가정한다면, $\text{lm}(y \sim \text{bs}(x, \text{df}=4))$ 는 "4개의 자유도로 휘어진 곡선"을 가정한다.

분류 모형들 — 같은 식 표기 한 묶음

R의 분류 함수들은 식 표기와 `predict()` 인터페이스를 거의 공유한다. 모형만 바꿔 끼우면 된다는 점이 큰 장점이다.

```
library(MASS)
library(class)
library(randomForest)
library(e1071)

# 품종별 80%를 무작위로 훈련 자료에 배정한다.
set.seed(123)
train_idx <- unlist(tapply(
  seq_len(nrow(iris)),
  iris$Species,
  function(i) sample(i, floor(0.8 * length(i)))
))
iris_tr <- iris[train_idx, ]
iris_te <- iris[-train_idx, ]

m_lda <- lda(Species ~ ., data = iris_tr)
m_qda <- qda(Species ~ ., data = iris_tr)
m_rf <- randomForest(Species ~ ., data = iris_tr, ntree = 200)
m_nb <- naiveBayes(Species ~ ., data = iris_tr)

pred_knn <- knn(
  train = iris_tr[, 1:4],
  test = iris_te[, 1:4],
  cl = iris_tr$Species,
  k = 5
)

mean(predict(m_lda, iris_te)$class == iris_te$Species)
mean(predict(m_qda, iris_te)$class == iris_te$Species)
mean(predict(m_rf, iris_te) == iris_te$Species)
mean(predict(m_nb, iris_te) == iris_te$Species)
mean(pred_knn == iris_te$Species)
```

▶ 본문 미리보기 — 3부 모델링 2

3부에서는 위의 `lda`, `qda`, `naiveBayes`, `knn`, `randomForest`, 그리고 `xgboost` / `lightgbm` / `catboost` 까지 같은 데이터셋(hn19)에 적용해 비교한다.

어느 모형이든 (식, 데이터) → `fit` → `predict` 라는 같은 패턴을 따른다.

군집화와 차원 축소 — kmeans, hclust, prcomp

5장 패턴 탐색에서는 다음 함수들이 등장한다.

```
# 데이터 표준화
X <- scale(iris[, 1:4])

# k-평균 군집 (3개)
set.seed(1)
km <- kmeans(X, centers = 3, nstart = 25)
table(iris$Species, km$cluster)

# 계층적 군집 (Ward 연결)
d <- dist(X)
hc <- hclust(d, method = "ward.D2")
plot(hc, labels = FALSE)
clusters <- cutree(hc, k = 3)

# 주성분분석
pc <- prcomp(X)
summary(pc)
plot(pc$x[, 1], pc$x[, 2], col = iris$Species, pch = 19)
```

▶ 본문 미리보기 — 5장 패턴 탐색

5장은 prcomp / kmeans / hclust 가 핵심이다.

데이터를 scale() 로 표준화한 뒤 prcomp 또는 kmeans 에 넣는 패턴이 반복적으로 등장한다.

Ridge 회귀의 계수 경로 같은 그림은 glmnet 패키지를 추가로 쓴다.

B.0.11 자주 만나는 오류, 그리고 AI 에게 물어보기

프로그래밍을 처음 시작하는 사람의 가장 큰 어려움은 "오류 메시지" 그 자체다. R의 오류 메시지는 처음엔 다소 무뚝뚝해 보이지만, 자주 만나는 종류는 정해져 있다. 이 절에서는 본문의 R 코드를 따라 하다가 마주치기 쉬운 오류를 정리한다.

자주 만나는 오류 5 가지

"there is no package called ..." — 패키지가 없다

```
Error in library(mgcv) :
  there is no package called 'mgcv'
```

- 원인 — 해당 패키지가 설치되어 있지 않다.
- 해결 — 콘솔에서 한 번 install.packages("mgcv") 를 실행한 뒤, 다시 library(mgcv) 를 실행한다.

"object 'df' not found" — 정의되지 않은 이름

```
Error: object 'df' not found
```

- 원인 — 그 변수를 만든 줄을 아직 실행하지 않았거나, RStudio 를 새로 열어 이전에 만들었던 객체가 사라진 경우가 가장 많다.
- 해결 — 위쪽 줄부터 순서대로 다시 실행한다. RStudio 메뉴의 [Code → Run Region → Run All] 이 편리하다.

"could not find function" — 함수가 없다

```
Error in filter(df, x > 1) : could not find function "filter"
```

- 원인 — 그 함수가 들어 있는 패키지를 아직 library() 로 불러오지 않았다.
- 해결 — 해당 패키지를 로딩한다 (예: library(dplyr)). 또는 dplyr::filter(...) 처럼 패키지명을 명시한다.

"\$ operator is invalid for atomic vectors" — 자료 구조가 안 맞는다

```
Error in v$x : $ operator is invalid for atomic vectors
```

- 원인 — 데이터프레임이 아니라 "벡터"에 \$ 로 접근하려 했다.
- 해결 — class(v) 또는 str(v) 로 자료 구조를 확인하고, 인덱싱 방식을 다시 고른다.

"contrasts can be applied only to factors with 2 or more levels"

```
Error in 'contrasts<-'(...):  
contrasts can be applied only to factors with 2 or more levels
```

- 원인 — 회귀식의 우변에 들어간 범주형 변수가 사실 한 가지 값만 갖고 있어 더미 변수를 만들 수 없다.
- 해결 — table(df\$g) 로 어느 변수에서 발생했는지 확인하고, 그 변수에 다른 값이 있는지 확인한다.

오류 메시지 읽는 법

R의 오류 메시지는 보통 두 줄로 이루어진다. 첫 줄은 "어디서" 났는지, 둘째 줄은 "무슨" 오류인지를 알려 준다.

```
Error in mean(x) :  
argument "x" is missing, with no default
```

- **첫 줄** — 오류가 난 함수 호출 위치 (mean(x) 호출 안에서)
- **둘째 줄** — 직접적 원인 ("x 라는 인자가 빠졌다")

이 둘만 천천히 읽어도 대부분의 문제는 빠르게 잡힌다. 메시지 안에 변수 이름이나 함수 이름이 포함되어 있다면, 그것을 먼저 의심하면 된다.

traceback() — 오류의 호출 흐름 보기

오류가 함수 안의 함수에서 났을 때는 traceback() 을 부르면 "어떤 호출 사슬에서" 났는지 보여 준다.

```
f <- function(x) g(x)
g <- function(x) mean(x[10])    # x가 짧으면 NA 문제
f(c(1,2,3))
traceback()
```

?와 ?? — R의 도움말

R의 가장 큰 장점 중 하나는 모든 함수에 잘 정리된 도움말이 따라온다는 점이다. 함수 이름 앞에 ? 를 붙이면 그 함수의 도움말이, ?? 를 붙이면 그 키워드를 포함한 모든 도움말이 검색된다.

```
?lm
?dplyr::filter
??regression
```

AI 에게 물어보기

ChatGPT, Claude, Gemini 같은 대화형 AI는 R 코드 작성에서도 "옆자리 동료"처럼 활용할 수 있다. 효과적으로 쓰려면 다음을 기억하면 좋다.

좋은 질문 = 충분한 맥락

초보자가 가장 흔히 저지르는 실수는 "왜 안 되죠?" 만 묻는 것이다. 다음 세 가지를 함께 주면 답의 정확도가 크게 올라간다.

- **내가 무엇을 하려고 했는지** (목표)
- **내가 실행한 코드** (가능하면 통째로)
- **받은 오류 메시지 또는 출력 결과** (전체 그대로)

다음은 잘 쓴 질문의 예시이다.

질문 예시

[목표] iris 데이터를 species 별로 묶어 평균 sepal_length를 구해 막대그래프로 그리려고 합니

다.

[코드] 다음을 실행했습니다.

```
library(tidyverse)
```

```
iris %>% group_by(species) %>% summarize(m = mean(Sepal.Length))
```

[오류] Error: Must group by variables found in `.data`. * Column `species` is not found.

왜 이 오류가 나며, 어떻게 고치면 되는지 알려 주세요.

(참고로 위 예시의 정답은 "R의 iris 데이터에서 열 이름은 첫 글자가 대문자인 Species이다" 이다. 변수명·열이름의 대소문자를 자주 의심하자.)

AI 를 활용하는 다섯 가지 패턴

- **코드 설명 부탁** — "이 R 코드가 무엇을 하는지 한 줄씩 설명해 줘."
- **오류 진단** — "다음 R 오류 메시지의 원인과 해결법을 알려 줘. (오류 첨부)"
- **코드 작성** — "데이터프레임 df 에서 income 이 상위 10%인 행만 골라 평균 BMI 를 구하는 dplyr 코드를 써 줘."
- **파이썬 ↔ R 번역** — "이 pandas 코드를 dplyr 로 옮겨 줘." (반대로도 가능)
- **개념 설명** — "R 의 factor 와 character 의 차이를 모형 적합 관점에서 설명해 줘."

AI 의 답을 검증하는 습관

AI는 종종 그럴듯하게 들리지만 실제로는 틀린 답을 줄 수 있다. 다음 습관을 들이면 좋다.

- 받은 코드는 반드시 직접 실행해 본다 — 출력이 기대와 같은지 확인한다.
- 함수 이름이 의심되면 ?함수명 으로 도움말을 한 번 더 본다.
- 결과의 단위·부호·자릿수가 상식적인지 의심한다. 회귀계수가 너무 크거나, P-값이 음수라면 어딘가 잘못된 것이다.

AI는 빠른 길잡이일 뿐, 최종 판단의 책임은 결국 사용자에게 있다는 점을 기억하면 충분하다.

더 공부하고 싶다면

- **공식 문서** — 가장 정확한 정보의 출처. 본문에서 사용된 패키지는 모두 잘 정리된 공식 문서를 가지고 있다.
 - R 공식: <https://www.r-project.org/>

- tidyverse: <https://www.tidyverse.org/>
- dplyr cheatsheet: <https://posit.co/resources/cheatsheets/>
- ggplot2: <https://ggplot2.tidyverse.org/>
- R for Data Science (책): <https://r4ds.hadley.nz/>
- **커뮤니티** — Stack Overflow 의 [r] 태그, RStudio Community, R-Bloggers.
- **동영상 강의** — "통통한 데이터 분석 공개강좌" 영상이 본 책의 출발점이며, 본문의 코드와 직접 짝을 이룬다.

이 부록의 분량으로 다루지 못한 주제는 매우 많지만, 본문 1-5부의 R 코드를 따라가는 데 필요한 디딤돌은 모두 엮어 두었다. 새 명령어를 만나면 두려워하지 말고,

"AI에게 묻고, ? 로 도움말을 확인하고, 직접 실행해 보는" 세 단계를 반복하면 된다. 그것만 익숙해지면, 이 책 이후의 어떤 데이터 분석도 두렵지 않을 것이다.

B.1 1 부 실습

이 절에서는 본문 1부에서 다룬 자료 정리와 시각화의 개념을 영화 데이터, 서울 아파트 거래 데이터, 서울시 문화행사 데이터를 이용해 R로 진행해 본다. 파이썬 실습과 동일한 데이터와 분석 흐름을 사용하므로, 두 언어의 표현 방식을 비교해 가며 읽으면 도움이 된다.

B.1.1 환경 설정과 데이터 로딩

R에서는 `library()` 함수로 분석에 필요한 패키지를 불러온다. tidyverse는 데이터 정리(dplyr), 형태 변환(tidyr), 시각화(ggplot2) 등을 한 번에 제공하는 메타 패키지이다.

[R 코드 1-1]

```
# 패키지 로딩
library(tidyverse)
library(readr)
library(ggplot2)
```

세 가지 데이터를 불러온다. R의 `read_csv()`는 데이터 구조를 자동으로 파악해 빠르게 읽어들이는다.

[R 코드 1-2]

```
# 데이터 로딩
영화데이터 <- read_csv("데이터/movie_data.csv")
부동산데이터 <- read_csv("데이터/seoul_apartment_transactions.csv")
서울시문화행사정보 <- read_csv("데이터/seoul_cultural_events.csv")

# 데이터 앞부분 확인
head(영화데이터)
```

부동산 데이터에서는 분석에 적합하도록 '아파트' 거래만 추리고 일부 변수의 이름을 줄여 사용한다.

[R 코드 1-3]

```
아파트데이터 <- 부동산데이터 %>%
  filter(건물용도 == "아파트", !is.na(건축년도)) %>%
  rename(가격 = `물건금액(만원)`, 건물면적 = `건물면적(m²)`)
```

B.1.2 자료 변환: Wide ↔ Long

치료제 A, B 자료를 wide와 long 형식 사이에서 변환해 본다. R에서는 tidyr의 pivot_longer()와 pivot_wider()를 사용한다.

[R 코드 1-4]

```
# 치료제 데이터 (wide 형식)
trt_wide <- tibble::tribble(
  ~id, ~a, ~b,
  1, 23, 16,
  2, 17, NA,
  3, 9, 11
)
trt_wide
```

pivot_longer()는 a, b 두 열을 하나의 'treatment' 열로 모아 long 형식으로 바꾼다.

[R 코드 1-5]

```
# wide → long
trt_long <- trt_wide %>%
  pivot_longer(
    cols = c(a, b),
    names_to = "treatment",
    values_to = "response"
  )
trt_long
```

반대로 long → wide 변환은 pivot_wider()로 수행한다.

[R 코드 1-6]

```
# long → wide
trt_wide_recovered <- trt_long %>%
  pivot_wider(
    names_from = treatment,
    values_from = response
  )
trt_wide_recovered
```

같은 방식으로 결핵 데이터에도 두 방향의 변환을 적용해 본다.

[R 코드 1-7]

```
tb_wide <- tibble::tibble(
  country   = c("Afghanistan", "Afghanistan", "Brazil", "Brazil",
               "China", "China"),
  year      = c(1999, 2000, 1999, 2000, 1999, 2000),
  cases     = c(745, 2666, 37737, 80488, 212258, 213766),
  population = c(19987071, 20595360, 172006362, 174504898,
               1272915272, 1280428583)
)

# wide → long
tb_long <- tb_wide %>%
  pivot_longer(cols = c(cases, population),
               names_to = "type", values_to = "count")

# long → wide
tb_wide_recovered <- tb_long %>%
  pivot_wider(names_from = type, values_from = count)
```

B.1.3 데이터 변환의 5 가지 문법: 배급사 RTP 분석

본문에서 다룬 RTP(Ratio to Peak) 분석을 R로 구현한다. `mutate`, `select`, `filter`, `group_by` + `summarise`, `arrange`가 차례로 연결되는 흐름이다.

1) 새로운 변수 만들기(Mutate): 대표배급사 만들기

배급사 정보에서 첫 번째로 표기된 배급사를 대표배급사로 정의한다. R의 `str_split_i()`는 문자열을 구분자로 분리한 뒤 특정 위치의 값을 가져온다.

[R 코드 1-8]

```
movies <- 영화데이터 %>%
  mutate(대표배급사 = str_trim(str_split_i(배급사, ";", 1)))
```

2~4) 행·열 정리와 그룹별 요약

분석에 필요한 변수만 남기고 결측값을 제거한 뒤, 대표배급사와 개봉년도를 기준으로 평균 관객수를 요약한다.

[R 코드 1-9]

```
stats <- movies %>%
  filter(!is.na(대표배급사), !is.na(개봉년도), !is.na(관객수)) %>%
  group_by(대표배급사, 개봉년도) %>%
  summarise(평균관객수 = mean(관객수),
            영화수 = n(),
            .groups = "drop")
```

5) 새로운 변수 만들기(Mutate): RTP 계산하기

배급사별 전성기 평균 관객수를 구한 뒤, 연도별 평균 관객수를 그 전성기로 나눈 비율을 RTP로 정의한다.

[R 코드 1-10]

```
peak <- stats %>%
  group_by(대표배급사) %>%
  summarise(peak_평균관객수 = max(평균관객수), .groups = "drop")
```

6) 행 추출하기(Filter) + 정렬하기(Arrange): 최근 연도 기준 RTP 가 낮은 배급사

가장 최근의 개봉년도에 해당하는 자료만 남기고, RTP가 낮은 순으로 정렬한다.

[R 코드 1-11]

```
recent_year <- max(movies$개봉년도, na.rm = TRUE)

recent <- stats %>%
  filter(개봉년도 == recent_year) %>%
  left_join(peak, by = "대표배급사") %>%
  mutate(RTP = 평균관객수 / peak_평균관객수) %>%
  arrange(RTP) %>%
  slice_head(n = 10)

recent
```

이제 RTP가 낮은 상위 7개 배급사를 막대그래프로 시각화한다.

[R 코드 1-12]

```
recent_top7 <- recent %>% slice_head(n = 7)

ggplot(recent_top7, aes(x = reorder(대표배급사, RTP), y = RTP)) +
  geom_col(fill = "skyblue") +
  labs(title = "RTP가 가장 낮은 7개 배급사",
       x = "대표배급사", y = "RTP") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

B.1.4 자료 결합

자치구 단위로 요약된 문화행사 자료와 아파트 거래 자료를 결합한다. R에서는 dplyr의 inner_join(), left_join(), full_join()을 사용한다.

[R 코드 1-13]

```
# 자치구별 문화행사 수 요약
문화_요약 <- 서울시문화행사정보 %>%
  count(자치구, name = "행사수")

# 자치구별 아파트 거래 요약
부동산_요약 <- 아파트데이터 %>%
  filter(건축년도 != 0) %>%
  group_by(자치구명) %>%
  summarise(
    거래건수 = n(),
    물건금액_평균 = mean(가격, na.rm = TRUE),
    물건금액_표준편차 = sd(가격, na.rm = TRUE),
    건물면적_평균 = mean(건물면적, na.rm = TRUE),
    건물면적_표준편차 = sd(건물면적, na.rm = TRUE),
    .groups = "drop"
  )
```

세 가지 결합 방식을 비교한다.

[R 코드 1-14]

```
# inner join
문화_부동산_inner <- inner_join(
  문화_요약, 부동산_요약, by = c("자치구" = "자치구명")
)
nrow(문화_부동산_inner)

# left join
문화_부동산_left <- left_join(
  문화_요약, 부동산_요약, by = c("자치구" = "자치구명")
)
nrow(문화_부동산_left)

# full join
문화_부동산_full <- full_join(
  문화_요약, 부동산_요약, by = c("자치구" = "자치구명")
)
nrow(문화_부동산_full)
```

같은 두 자료를 합치더라도 어떤 join을 선택하느냐에 따라 행 수와 결측값의 분포가 달라진다.

B.1.5 범주형 자료의 분포

영화 데이터의 대표국적 변수를 이용해 범주형 자료의 분포를 시각화한다. `count()`로 도수분포표를 만든다.

[R 코드 1-15]

```
대표국적_집계 <- count(영화데이터, 대표국적, sort = TRUE)
head(대표국적_집계, 10)
```

가로 막대그래프

`reorder()`로 영화 수가 많은 순서로 정렬하고, `coord_flip()`으로 가로 방향으로 뒤집는다.

[R 코드 1-16]

```
ggplot(대표국적_집계, aes(x = reorder(대표국적, n), y = n)) +
  geom_col(fill = "mediumturquoise") +
  coord_flip() +
```

```
labs(x = NULL, y = "영화 수",
     title = "대표국적별 영화 수",
     caption = "데이터: 영화데이터")
```

파이차트

ggplot2에서는 막대그래프를 만든 뒤 `coord_polar()`로 좌표계를 극좌표로 바꾸어 파이차트를 그린다.

[R 코드 1-17]

```
top8 <- 영화데이터 %>%
  count(대표국적, sort = TRUE) %>%
  slice_head(n = 8) %>%
  mutate(비율 = n / sum(n),
         퍼센트 = paste0(round(비율 * 100, 1), "%"),
         대표국적 = fct_inorder(대표국적))

ggplot(top8, aes(x = "", y = 비율, fill = 대표국적)) +
  geom_col(width = 1, color = "white") +
  coord_polar("y", direction = 1) +
  geom_text(aes(label = 퍼센트),
            position = position_stack(vjust = 0.5),
            color = "black", size = 3) +
  scale_fill_brewer(palette = "Dark2") +
  labs(fill = "대표국적", title = "상위 8개 대표국적 내 구성 비율") +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

B.1.6 연속형 자료의 분포

아파트 거래 데이터의 가격(로그 변환)을 이용해 연속형 자료의 분포를 시각화한다.

히스토그램

[R 코드 1-18]

```
아파트데이터 %>%
  filter(가격 > 0) %>%
```

```
mutate(log_가격 = log(가격)) %>%
  ggplot(aes(x = log_가격)) +
  geom_histogram(bins = 60, fill = "lightblue", color = "white") +
  labs(title = "아파트 가격 (log) 분포",
        x = "log(가격)", y = "건수") +
  theme_minimal()
```

KDE 와 대역폭 비교

대역폭을 다르게 설정한 세 KDE 곡선을 한 그래프에 그린다. R의 `density()` 함수에서는 `bw` 인자로 대역폭을 지정한다.

[R 코드 1-19]

```
x <- log(아파트데이터$가격[아파트데이터$가격 > 0])

density_df <- bind_rows(
  data.frame(density(x, bw = 0.05)[c("x", "y")], bandwidth = "bandwidth = 0.05"),
  data.frame(density(x, bw = 0.10)[c("x", "y")], bandwidth = "bandwidth = 0.10"),
  data.frame(density(x, bw = 0.20)[c("x", "y")], bandwidth = "bandwidth = 0.20")
)

ggplot() +
  geom_histogram(
    data = data.frame(x = x),
    aes(x = x, y = after_stat(density)),
    bins = 60, fill = "gray80", color = "white"
  ) +
  geom_line(
    data = density_df,
    aes(x = x, y = y, color = bandwidth),
    linewidth = 1.2
  ) +
  scale_color_manual(
    name = "Bandwidth",
    values = c("bandwidth = 0.05" = "red",
              "bandwidth = 0.10" = "blue",
```

```

    "bandwidth = 0.20" = "green")
  ) +
  labs(title = "아파트 가격 (log) 분포: 히스토그램과 커널 밀도 추정",
        x = "log(가격)", y = "밀도") +
  theme_minimal() +
  theme(legend.position = "top")

```

대역폭이 작을수록 곡선이 데이터의 미세한 변동에 민감하게 반응하고, 클수록 부드럽게 다듬어진다.

B.1.7 두 범주형 변수의 관계

분할표

R에서는 `table()`로 두 변수의 빈도 분할표를 만들고, `addmargins()`로 행·열 합계를 추가할 수 있다.

[R 코드 1-20]

```

분할표 <- table(영화데이터$대표국적, 영화데이터$5년내개봉작)
분할표_한계포함 <- addmargins(분할표)
분할표_한계포함

```

모자이크 플롯

`ggmosaic` 패키지의 `geom_mosaic()`을 이용해 직사각형 면적으로 빈도를 표현한다. 범주가 많으면 보기 어려우므로 상위 4개 국적만 골라 그린다.

[R 코드 1-21]

```

영화데이터$개봉5년이내 <- 영화데이터$5년내개봉작

df_mosaic <- na.omit(영화데이터[, c("대표국적", "개봉5년이내")])
df_mosaic$대표국적 <- factor(df_mosaic$대표국적)
df_mosaic$개봉5년이내 <- factor(df_mosaic$개봉5년이내)

top4 <- names(sort(table(df_mosaic$대표국적), decreasing = TRUE))[1:4]
df_mosaic <- subset(df_mosaic, 대표국적 %in% top4)
df_mosaic$대표국적 <- droplevels(df_mosaic$대표국적)

```

```
ggplot(df_mosaic) +
  geom_bar(
    aes(x = 대표국적, fill = 개봉5년이내),
    position = "fill", color = "white"
  ) +
  labs(title = "상위 4개 대표국적과 5년내개봉작 여부",
    x = "대표국적", y = "비율", fill = "5년내개봉작") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

히트맵

geom_tile()로 격자를 그리고 geom_text()로 빈도 숫자를 함께 표시한다.

[R 코드 1-22]

```
heatmap_data <- 영화데이터 %>%
  count(대표국적, 개봉5년이내)

ggplot(heatmap_data, aes(x = 개봉5년이내, y = 대표국적, fill = n)) +
  geom_tile(color = "black") +
  geom_text(aes(label = n), color = "white") +
  scale_fill_gradient(low = "lightgray", high = "black", name = "빈도수") +
  labs(title = "대표국적과 5년내개봉작 여부 간 교차 빈도 히트맵",
    x = "5년내개봉작", y = "대표국적") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(hjust = 0.5))
```

B.1.8 두 연속형 변수의 관계

기본 산점도

[R 코드 1-23]

```
ggplot(영화데이터, aes(x = 스크린수, y = 매출액)) +
  geom_point(alpha = 0.6) +
  labs(title = "스크린수와 매출액의 관계",
```

```
x = "스크린수", y = "매출액") +
theme_minimal()
```

로그 변환

값의 범위가 너무 넓을 때는 로그 변환을 적용한다. 변수 자체를 로그 변환하는 방법과, 축 스케일만 로그로 바꾸는 방법 두 가지를 모두 보여준다.

[R 코드 1-24]

```
영화데이터2 <- 영화데이터 %>%
  filter(스크린수 > 0, 매출액 > 0) %>%
  mutate(log_매출액 = log(매출액))

# (1) 변수를 직접 로그 변환
ggplot(영화데이터2, aes(x = 스크린수, y = log_매출액)) +
  geom_point(alpha = 0.6) +
  labs(title = "로그 변환된 매출액 vs 스크린수",
       x = "스크린수", y = "log(매출액)") +
  theme_minimal()

# (2) 축 스케일만 로그로 변환
ggplot(영화데이터2, aes(x = 스크린수, y = 매출액)) +
  geom_point(alpha = 0.6) +
  scale_y_log10() +
  labs(title = "로그 축 적용: 매출액 vs 스크린수",
       x = "스크린수", y = "매출액 (log scale)") +
  theme_minimal()
```

산점도 + 주변 분포 + 추세선

ggExtra 패키지의 ggMarginal()을 사용하면 산점도 위·오른쪽에 주변 분포를 추가할 수 있다. 직선 추세선은 geom_smooth(method = "lm")으로 그린다.

[R 코드 1-25]

```
library(ggExtra)

data_sc <- 영화데이터 %>%
```

```

select(스크린수, 매출액) %>%
drop_na()

p_sc <- ggplot(data_sc, aes(x = 스크린수, y = 매출액)) +
  geom_point(alpha = 0.4, size = 1.5) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(title = "스크린수와 매출액: 산점도와 주변 분포, 직선 추세선",
        x = "스크린수", y = "매출액") +
  theme_minimal()

ggMarginal(p_sc, type = "histogram",
           bins = 30, fill = "skyblue", alpha = 0.6)

```

B.1.9 연속형 자료와 범주형 자료의 관계

대표국적별 스크린수 분포를 상자그림으로 비교한다. 영화 수가 많은 상위 6개 국적만 골라 그린다.

[R 코드 1-26]

```

top6 <- 영화데이터 %>%
  count(대표국적, sort = TRUE) %>%
  slice_head(n = 6) %>%
  pull(대표국적)

영화데이터 %>%
  filter(대표국적 %in% top6) %>%
  ggplot(aes(x = 대표국적, y = 스크린수)) +
  geom_boxplot(fill = "lightblue", outlier.alpha = 0.3) +
  labs(title = "대표국적별 스크린수 분포",
        x = "대표국적", y = "스크린 수") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

B.1.10 조건부 비교: 패싯

ggplot2에서는 `facet_grid()`로 행과 열 방향의 조건을 동시에 지정해 여러 개의 패널을 만들 수 있다. 5년내개봉작 여부(행)와 대표국적 상위 4개(열)를 조건으로 삼는다.

[R 코드 1-27]

```

top4 <- 영화데이터 %>%
  count(대표국적, sort = TRUE) %>%
  slice_head(n = 4) %>%
  pull(대표국적)

df_facet <- 영화데이터 %>%
  filter(
    스크린수 > 0, 관객수 > 0,
    !is.na(대표국적), !is.na(`5년내개봉작`),
    대표국적 %in% top4
  )

ggplot(df_facet, aes(x = 스크린수, y = 관객수)) +
  geom_point(alpha = 0.6) +
  scale_y_log10() +
  facet_grid(`5년내개봉작` ~ 대표국적) +
  labs(title = "5년내개봉작 여부와 대표국적에 따른 스크린수 vs 관객수",
        x = "스크린수", y = "관객수 (로그 스케일)") +
  theme_minimal()

```

행 방향으로는 개봉 시기 조건이, 열 방향으로는 국적 조건이 동일한 좌표축 위에서 비교된다. 패싯을 사용하면 각 패널 안의 비교(스크린수와 관객수의 관계)와 패널 사이의 비교(조건 간 차이)가 공간적으로 분리되어, 색이나 기호만으로 구분할 때보다 인지 부담이 크게 줄어든다.

B.2 2 부 실습

이 절에서는 우리나라 프로야구 불펜 투수 데이터를 이용하여 앞에서 다룬 선형회귀분석과 비모수 회귀분석을 R로 진행해 본다. R은 통계 계산과 시각화에 최적화된 언어로, 복잡한 회귀 모형의 결과를 직관적으로 확인하는 데 강점이 있다.

B.2.1 탐색적 자료분석

파이썬 실습에서와 마찬가지로 본격적인 분석 전 데이터의 특징을 알아보는 탐색적 자료분석을 진행한다. 먼저 필요한 패키지를 불러온다. R에서는 `library()` 함수를 사용하여 설치된 패키지를 로드한다.

[R 코드 2-1]

```
# 패키지 로딩
library(tidyverse) # 데이터 전처리(dplyr), 시각화(ggplot2) 등을 포함한 메타 패키지
library(data.table) # 대용량 데이터를 빠르게 읽고 쓰기 위한 패키지
```

`data.table` 패키지의 `fread()`를 이용하여 데이터 파일 `reliever_data.csv`를 불러오자. `fread()`는 파일 구조를 자동으로 파악하며 매우 빠른 속도를 제공한다. 불러온 데이터의 변수 구성은 파이썬 실습의 같은 부분을 참고하기 바란다.

[R 코드 2-2]

```
# 데이터 로딩
reliever_data_file <- '데이터/reliever_data.csv' # 필요시 데이터 경로 설정
baseball <- fread(reliever_data_file, encoding = 'UTF-8')
# 데이터 앞부분 확인 (5행까지)
head(baseball, n = 5)
```

여기서 우리는 지난 시즌 성적인 출전 경기 수(G), 평균자책점(ERA), 마무리 투수 여부(closer)를 설명변수로, 로그 변환된 다음 시즌 연봉(log_salary)을 반응변수로 사용한다.

먼저 변수 간의 선형적 연관성을 파악하기 위해 상관계수 행렬을 계산하고 시각화해 보자. R에서는 `select()` 함수로 필요한 변수만 뽑아내고, 이를 상관계수 계산 함수인 `cor()`에 전달하는 **파이프 연산자(%>%)**를 활용하면 코드가 훨씬 간결해진다.

[R 코드 2-3]

```
# 상관계수 계산과 시각화 패키지
library(ggcorrplot)
# 상관계수 계산을 위한 변수 선택
selected_cols <- baseball %>% select(log_salary, G, ERA)
# 상관계수 행렬 계산
cor_mat <- cor(selected_cols, use = "complete.obs")
# 상관계수 행렬 시각화 (Heatmap)
ggcorrplot(cor_mat, lab = TRUE)
```

상관계수는 두 변수의 관계를 하나의 숫자로 요약해 주지만, 데이터 속에 숨겨진 곡선 패턴이나 이상치를 확인하려면 산점도를 그려보는 것이 필수적이다. R의 GGally 패키지에 포함된 ggpairs는 파이썬의 pairplot보다 더 풍부한 정보(상관계수 수치, 밀도 함수 등)를 한 화면에 담아준다.

[R 코드 2-4]

```
# 산점도를 위한 패키지
library(GGally)
# 주요 변수들 간의 산점도, 분포, 상관계수를 동시에 시각화
ggpairs(baseball, columns = c("log_salary", "G", "ERA"))
```

B.2.2 선형회귀분석 실습

이제 선형회귀분석을 R을 이용해 진행해 보자. R은 lm 함수 하나로 회귀 모델링의 거의 모든 과정을 수행할 수 있을 만큼 직관적이다.

설명변수가 출전 경기 수와 평균자책점인 경우 다중선형회귀분석 및 진단

출전 경기 수 G와 평균자책점 ERA를 설명변수로, 로그 변환한 다음 시즌 연봉 log_salary를 반응변수로 사용한 선형회귀모형을 적합한다.

[R 코드 2-5]

```
# 다중선형회귀모형 적합 (로그 연봉 ~ 경기 수 + 평균 자책점 )
fit_G_ERA <- lm(log_salary ~ G + ERA, data = baseball)

# 다중선형회귀모형 적합 결과 확인
summary(fit_G_ERA)
```

Call:

```
lm(formula = log_salary ~ G + ERA, data = baseball)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-1.13768 -0.33322 -0.06387  0.26421  2.27406
```

Coefficients:

```
      Estimate Std. Error t value Pr(>|t|)
(Intercept)  8.540129   0.118566  72.029 < 2e-16 ***
G             0.022783   0.001498  15.210 < 2e-16 ***
ERA          -0.079338   0.015040  -5.275 2.02e-07 ***
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4718 on 478 degrees of freedom

Multiple R-squared: 0.4932, Adjusted R-squared: 0.4911

F-statistic: 232.6 on 2 and 478 DF, p-value: < 2.2e-16

summary() 함수의 결과는 파이썬의 결과와 수치적으로 동일하지만, 구성 방식이 조금 다르다. 주요 항목의 의미를 정리하면 다음과 같다.

구분	항목	의미 및 해석	
모형 성능	Multiple R-squared	결정계수. 모델의 설명력 (0.4932)을 나타낸다.	
Adjusted R-squared	수정된 결정계수. 변수 개수를 고려해 보정된 설명력이다.		
F-statistic/p-value	모형 전체의 유의성 검정을 위한 F-검정통계량과 P-값.		
회귀 계수	Estimate	회귀계수 추정치.	
Std. Error/ t value	회귀계수의 표준오차와 t-통계량이다.		

Pr(>W	tW	)	개별 변수의 P-값. 별표 (***)가 많을수록 연봉에 미치는 영향이 확실하다는 증거다.
잔차 진단	Residuals	잔차의 분포 요약. Median(중앙값)이 0에 가깝고 사분위수(1Q, 3Q)가 대칭적일수록 이상적이다.	

추가로 Call 부분에서는 모델 적합에 사용된 수식(formula)과 데이터(data)를 확인할 수 있다. 하단부의 Residual standard error는 잔차의 표준오차로, 자유도와 함께 계산되어 실제 관측값이 회귀선으로부터 평균적으로 얼마나 떨어져 있는지를 나타낸다.

이번에는 회귀모형의 통계적 타당성을 검토하기 위해 잔차 분석을 실시한다. 먼저 위 모형의 스튜던트화 잔차를 계산하여 잔차도를 그려보자.

[R 코드 2-6]

```
# 선형모형을 이용한 예측값 계산
fitted_vals <- predict(fit_G_ERA)
# 스튜던트화 잔차 계산
residuals_stud <- rstudent(fit_G_ERA)
```

[R 코드 2-7]

```
# 잔차도 그리기
ggplot(data = NULL, aes(x = fitted_vals, y = residuals_stud)) +
  geom_point() + geom_hline(yintercept = 0)
```

이어 잔차의 정규분포 Q-Q 플롯을 확인한다.

[R 코드 2-8]

```
# 정규분포 Q-Q 플롯
qqnorm(residuals_stud)
# 참조선을 추가
qqline(residuals_stud)
```

설명변수가 출전 경기 수와 마무리 투수 여부인 경우 다중선회귀 분석

이번에는 범주형 설명변수가 포함된 선형회귀분석을 해보자. R에서 범주형 변수를 설명변수로 사용하기 위해서는 변수의 데이터형이 팩터 factor로 설정되어 있어야 한다. 다음은 closer 변수의 데이터형을 확인하는 코드이다.

[R 코드 2-9]

```
# closer 변수의 데이터형 확인
class(baseball$closer)
```

만약 closer 변수가 factor 형이 아니라면(예: 정수형이나 문자형), 다음과 같이 데이터형을 factor형으로 변환하여 사용한다. closer 변수가 숫자(0 또는 1)로 저장되어 있어도, 이를 단순한 수치가 아닌 '그룹 이름표'로 인식시켜야 하기 때문에 factor형으로 변환하는 과정이 필요하다.

[R 코드 2-10]

```
# closer 변수를 범주형(factor)으로 변환
baseball$closer <- as.factor(baseball$closer)
# 변환 후 다시 확인
class(baseball$closer)
```

이제 출전 경기 수 G와 마무리 투수 여부 closer를 설명변수로, 로그 변환한 다음 시즌 연봉 log_salary를 반응변수로 사용한 선형회귀모형을 적합해 본다.

[R 코드 2-11]

```
# 교호작용을 고려하지 않은 모형
factor_fit1 <- lm(log_salary ~ G + closer, data = baseball)
summary(factor_fit1)
```

결과표에서 closer1의 계수를 확인해 보자. 반응변수가 로그 연봉이므로, 이 값은 경기 수가 같더라도 마무리 보직을 맡는 것만으로 연봉이 몇 퍼센트(%)나 '점프'하는지를 나타내는 마무리 보직 프리미엄이다.

이번에는 보직에 따라 경기 수당 연봉 상승 폭이 다를 수 있음을 가정하는 교호작용 모형을 적합해 본다. R에서는 두 변수 사이에 * 기호를 써서 간단히 표현할 수 있다.

[R 코드 2-12]

```
# 교호작용을 고려한 모형
factor_fit2 <- lm(log_salary ~ G * closer, data = baseball)
summary(factor_fit2)
```

결과표에서 교호작용 항(G:closer1)의 계수가 음수이므로 볼펜 투수보다 연봉 상승의 '효율'이 상대적으로 낮음을 의미한다.

이번에는 범주형 설명변수가 포함된 다중선형회귀모형을 시각화해보자. 교호작용을 고려하지 않은 경우를 시각화해 본다. 교호작용을 고려한 경우는 아래 코드에서 factor_fit1을 factor_fit2로 변경하면 된다.

[R 코드 2-13]

```
# 시각화를 위한 설명변수 G의 범위에서 예측용 값 100개 생성
var_range <- seq(min(baseball$G), max(baseball$G), length.out = 100)
pred_df1 <- expand.grid(G = var_range, closer = factor(c(0, 1)))
pred_df1$log_salary <- predict(factor_fit1, newdata = pred_df1)

ggplot() +
  geom_point(data = baseball, aes(x = G, y = log_salary, color = closer)) +
  geom_line(data = pred_df1, aes(x = G, y = log_salary, color = closer)) +
  scale_color_manual(values = c("0" = "red", "1" = "blue"),
    labels = c("0" = "relief pitcher", "1" = "closer"))
```

설명변수가 출전 경기 수인 경우 선형회귀분석을 이용한 예측

회귀분석의 핵심적인 목적 중 하나는 새로운 데이터가 주어졌을 때의 결과를 예측하는 것이다. 여기서는 출전 경기 수(G)만을 설명변수로 하는 선형회귀모형을 적합하고, 특정 경기 수(G = 40)를 소환한 선수의 예상 연봉을 산출해 본다.

[R 코드 2-14]

```
# 선형회귀모형 적합 (로그 연봉 ~ 경기 수)
fit_G <- lm(log_salary ~ G, data = baseball)
# 새로운 선수 (G = 40)
new_G <- data.frame(G = 40)
# 예측값 및 95% 예측구간 계산
predict(fit_G, newdata = new_G, interval = "prediction", level = 0.95)
```

위 코드의 결과에서 fit에 해당하는 값이 반응변수(로그 연봉)의 평균 예측값이며, lwr과 upr은 실제 연봉이 존재할 것으로 기대되는 95% 예측구간이다. 출력된 로그 단위의 예측치와 이를 지수 변환(exp())하여 도출한 실제 연봉(만 원) 단위의 환산 값은 앞선 **파이썬 실습(2.4.2)**의 결과와 수치적으로 동일하므로 해당 절의 상세 설명을 참고하기 바란다.

B.2.3 비모수회귀분석 실습

이제 비모수 회귀분석을 R로 실습해 보자. 비모수 회귀는 선형이라는 틀에 얽매이지 않고 데이터의 흐름을 유연하게 따라가는 방식이다. 여기에서도 반응변수는 로그 변환된 연봉(log_salary)을 사용하며, 먼저 설명변수 하나를 이용한 커널 회귀부터 시작해 본다.

커널 회귀분석

출전 경기 수(G)에 따른 로그 연봉의 변화를 포착하기 위해 KernSmooth 패키지를 이용해 보자. 파이썬의 KernelReg와 마찬가지로, 여기서도 곡선의 '부드러움'을 결정하는 대역폭(bandwidth) 설정이 가장 중요하다. 아래 코드에서는 대역폭을 3으로 설정하고, 국소 상수 회귀(degree = 0) 방식으로 적합을 진행한다.

[R 코드 2-15]

```
# 커널 회귀를 위한 패키지 로드
library(KernSmooth)

fit_ks <- locpoly(
  x = baseball$G,
  y = baseball$log_salary,
  bandwidth = 3,
  degree = 0
)
```

이제 추정된 곡선을 시각화해 보자. R의 locpoly 함수로 만든 fit_ks 객체는 내부적으로 이미 경기 수(G)의 전체 범위에 대해 촘촘한 격자점(x)을 생성하고, 각 점에서의 예측값(y)을 계산하여 저장하고 있다. 따라서 파이썬처럼 시각화를 위한 별도의 예측 데이터를 만들 필요 없이, 적합된 객체의 결과 값을 그대로 산점도 위에 그릴 수 있어 매우 편리하다.

커널 회귀의 확장

Locpoly 함수는 이름에서 알 수 있듯이 **국소 다항 회귀(Local Polynomial Regression)**를 수행하는 범용적인 함수이다. 여기서 degree = 0으로 하는 경우 국소적으로 상수 취급을 하는 우리가 소개한 커널 회귀가 된다. 반면 degree = 1로 설정하면 국소 선형 회귀(Local Linear Regression)가 된다. 이는 국소 구간마다 직선을 적합 하는 방식으로, 커널 회귀의 확장형이라 할 수 있다. 이 방법은 데이터의 양 끝단인 경계선 근처에서 추정치가 왜곡되는 편향(Bias) 문제를 개선해 준다는 장점이 있다.

[R 코드 2-16]

```
ggplot() +
  geom_point(data = baseball, aes(x = G, y = log_salary)) +
  geom_line(aes(x = fit_ks$x, y = fit_ks$y), color = "red")
```

스플라인 회귀

이번에는 데이터를 구간별로 나누어 다항식을 연결하는 3차 B-스플라인(B-Spline) 회귀모형을 적합해 보자. R에서는 `splines` 패키지의 `bs` 함수를 사용하면 별도의 복잡한 변환 없이 회귀식 내에서 즉시 스플라인 기저(Basis)를 생성할 수 있다. 파이썬 실습과 동일하게 경기 수(G)의 사분위수를 매듭점으로 설정하여 모형을 적합한다.

[R 코드 2-17]

```
# 스플라인 회귀를 위한 패키지 로드
library(splines)
# 매듭점 설정: G의 1사분위수, 중앙값, 3사분위수
G_knots <- quantile(baseball$G, probs = c(0.25, 0.50, 0.75))
# B-스플라인 회귀모형 적합
# bs() 함수 내에서 degree=3(3차 다항식)과 매듭점을 지정
fit_spline <- lm(log_salary ~ bs(G, knots = G_knots, degree = 3), data = baseball)
```

이 결과를 시각화하기 위해서 적합한 스플라인 곡선을 그리기 위해 예측용 G 값을 만들고, 이에 대한 예측값을 계산한 후 산점도와 함께 추정된 스플라인 곡선을 그려보자. 이 결과를 시각화하기 위해, 모델 적합 결과인 `fit_spline` 객체에 저장된 예측값(`fitted.values`)을 활용하여 산점도 위에 추정된 스플라인 곡선을 그려보자.

[R 코드 2-18]

```
pred_spline <- data.frame(
  G = seq(min(baseball$G), max(baseball$G), length.out = 200)
)
pred_spline$log_salary <- predict(fit_spline, newdata = pred_spline)

ggplot(baseball, aes(x = G, y = log_salary)) +
  geom_point() +
  geom_line(
    data = pred_spline,
```

```
aes(x = G, y = log_salary),
color = "blue"
)
```

스무딩 스플라인

이번에는 모든 데이터 포인트를 매듭점으로 활용하되, 곡선의 급격한 변화에 별점을 부여하여 흐름을 매끄럽게 다듬는 스무딩 스플라인을 적합해 보자. R의 기본 함수인 `smooth.spline`을 사용한다. 파이썬 실습(2.4.5)에서는 중복된 설명변수 값을 직접 처리하고 정렬해야 하는 번거로움이 있었지만, R의 `smooth.spline`은 이러한 전처리를 내부적으로 자동 수행하므로 원본 데이터를 그대로 사용할 수 있다는 장점이 있다.

[R 코드 2-19]

```
# 스무딩 스플라인 적합
# x: 설명변수, y: 반응변수
# spar: 매끄러움의 정도를 조절하는 조정 모수 (0~1 사이 값)
fit_smooth <- smooth.spline(x = baseball$G,
                             y = baseball$log_salary, spar = 0.7)
```

본문에서 언급하였듯이 R에서 사용하는 조정 모수 `spar`는 사용자가 조절하기 편하도록 별점 매개변수 λ 를 **재정의(Reparameterization)**한 값이다. 보통 0과 1 사이의 값을 가지며, 1에 가까울수록 곡선은 더 매끄러워져 직선에 가까운 형태를 띈다. 이제 적합된 스무딩 스플라인 곡선을 시각화해 보자. `smooth.spline` 객체는 커널 회귀와 마찬가지로 시각화에 최적화된 격자점(x)과 예측값(y)을 내부에 저장하고 있어 별도의 예측값 계산 절차 없이 즉시 그래프를 그릴 수 있다.

[R 코드 2-20]

```
# 결과 시각화
ggplot() +
  geom_point(data = baseball, aes(x = G, y = log_salary)) +
  geom_line(aes(x = fit_smooth$x, y = fit_smooth$y), color = "blue")
```

일반화 가법모형

마지막으로 여러 설명변수의 비선형적 효과를 동시에 고려하는 **일반화 가법 모형(GAM)**을 적합해 보자. 여기서는 출전 경기 수(G)와 평균자책점(ERA)을 설명변수로 사용한다. R에서는 이 분야의 표준 패키지인 `mgcv`의 `gam` 함수를 활용한다. 각 변수 앞에 붙는 `s` 함수는 해당 변수를 매끄러운 스플라인 항으로 처리하겠다는 의미이다.

[R 코드 2-21]

```
# GAM 분석을 위한 패키지 로드
library(mgcv)
# GAM 모형 적합
# k: 기저 함수의 개수 (매듭점의 수와 유사), 파이썬의 n_splines와 대응됨
gam_fit <- gam(log_salary ~ s(G, k = 8) + s(ERA, k = 8), data = baseball)
```

파이썬 실습에서는 특정 변수의 순수한 기여도를 보기 위해 다른 변수를 평균으로 고정하고 부분 잔차를 직접 계산하는 복잡한 과정이 필요했다. 하지만 R의 mgcv 패키지는 plot 함수 하나만으로 이 모든 과정을 자동으로 처리하여 시각화해 준다.

[R 코드 2-22]

```
# 각 설명변수의 부분 효과 그래프 시각화
# residuals = TRUE: 부분 잔차(실제 데이터의 흠어짐)를 점으로 표시
plot(gam_fit, residuals = TRUE, col = "blue")
```

B.3 3 부 실습

본 부록은 3부 본문 1-6장의 분류 방법을 R로 실행해 보는 자료이다. 주요 분류 실습은 hn19.csv를 사용하며, 일부 신경망 구조 예시는 별도의 일반 예제로 제시한다.

B.3.1 R 환경 준비

데이터 파일 위치

본 부록의 코드를 실행하려면 책 루트의 데이터/ 폴더에 hn19.csv를 둔다. 데이터 출처와 변수 설명은 온라인 데이터 가이드를 참조한다.

필수 R 패키지

본 부록 전반에서 사용하는 R 패키지의 목록은 다음과 같다. 처음 한 번만 install.packages()로 설치한 뒤 library()로 불러온다.

[R 코드 3-1]

```
# 기본 데이터 처리·시각화·분류 패키지
# MASS를 먼저 불러온 뒤 tidyverse를 불러 dplyr::select()가 우선되게 한다.
library(MASS); library(e1071); library(data.table); library(tidyverse)
library(ggcorrplot); library(gridExtra)
library(class); library(kknn); library(tree); library(randomForest)

# 선택 심화 패키지는 해당 예제 코드에서 별도로 불러온다.
```

본 부록의 그래프는 한글 레이블을 사용하므로, macOS/Windows에서 한글이 깨질 경우 showtext 패키지 등을 추가로 로드한다.

B.3.2 분류 도입과 데이터 탐색

실습 1: 국민건강영양조사 데이터를 이용한 당뇨 위험군 분류

분석을 위해 필요한 라이브러리 및 데이터 불러오기

rm(list = ls())를 이용해 R의 environment를 초기화 할 수 있다.

[R 코드 3-2]

```
rm(list = ls())
```

분석을 위해 필요한 R 라이브러리를 불러온다.

[R 코드 3-3]

```
# 기본 실습 패키지
# install.packages(c("data.table", "tidyverse", "ggcorrplot", "gridExtra",
#                    "kknn", "tree", "randomForest", "e1071"))

# 선택 심화 실습 패키지
# install.packages(c("xgboost", "lightgbm", "keras"))
# keras::install_keras() # Keras 백엔드 설치(최초 1회)
# CatBoost는 공식 R 설치 안내를 따른다.
# https://catboost.ai/docs/en/concepts/r-installation

library(data.table)
library(tidyverse)
```

`data.table` 라이브러리의 `fread()` 함수를 이용하여 분석에 사용할 데이터인 `hn19.csv`를 불러온다. 이 데이터는 2019년 국민건강영양조사 자료에서 분석에 필요한 변수만을 추출한 것이다. 각 변수의 의미는 다음과 같다.

- `age`: 만 나이 (단위: 세)
- `HE_BMI`: 체질량지수 BMI (단위: kg/m^2)
- `HE_HP`: 혈압 상태 (1 = 정상, 2 = 고혈압 전단계, 3 = 고혈압)
- `HE_DM_HbA1c`: 당뇨 상태 (1 = 정상, 2 = 당뇨 전단계, 3 = 당뇨)

[R 코드 3-4]

```
data <- fread("데이터/hn19.csv", encoding = "UTF-8")
head(data, n = 5)
```

결측값(NA)가 포함된 행은 분석에서 제외한다.

[R 코드 3-5]

```
data <- data %>%
  dplyr::select(age, HE_BMI, HE_HP, HE_DM_HbA1c) %>%
  drop_na()

nrow(data)
```

분류 모형의 반응변수와 설명변수를 정의하기 위해 변수 변환을 진행한다.

[R 코드 3-6]

```
data <- data %>%
  mutate(
    diabetes = ifelse(HE_DM_HbA1c >= 2, 1, 0),
    hp = ifelse(HE_HP >= 2, 1, 0),
    bmi = HE_BMI
  ) %>%
  mutate(diabetes = as.factor(diabetes), hp = as.factor(hp)) %>%
  dplyr::select(age, bmi, hp, diabetes)
```

```
head(data, n = 3)
```

반응변수의 분포를 확인한다.

[R 코드 3-7]

```
table(data$diabetes)
prop.table(table(data$diabetes))
```

변수 간의 상관관계 및 분포 파악하기

[R 코드 3-8]

```
plt1 <- ggplot(data, aes(x = diabetes, y = age, fill = diabetes)) +
  geom_boxplot(alpha = 0.6) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ggtitle("당뇨 위험군별 나이 분포") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5), legend.position = "none")

plt2 <- ggplot(data, aes(x = diabetes, y = bmi, fill = diabetes)) +
  geom_boxplot(alpha = 0.6) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ggtitle("당뇨 위험군별 BMI 분포") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5), legend.position = "none")

gridExtra::grid.arrange(plt1, plt2, ncol = 2)
```

[R 코드 3-9]

```
table(hp = data$hp, diabetes = data$diabetes)
```

[R 코드 3-10]

```
ggplot(data, aes(x = hp, fill = diabetes)) +
  geom_bar(position = "fill", alpha = 0.8) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ylab("비율") +
  ggtitle("혈압 상태별 당뇨 위험군 비율") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```

B.3.3 로지스틱 회귀와 다범주 분류

단순 로지스틱 회귀모형 적합

[R 코드 3-11]

```
simple_fit <- glm(diabetes ~ bmi, data = data, family = binomial)
summary(simple_fit)
```

[R 코드 3-12]

```
exp(coef(simple_fit))
```

[R 코드 3-13]

```
exp(coef(simple_fit)["bmi"] * 5)
```

다중 로지스틱 회귀모형 적합**[R 코드 3-14]**

```
multi_fit <- glm(diabetes ~ age + bmi + hp, data = data, family = binomial)
summary(multi_fit)
```

[R 코드 3-15]

```
coef_table <- data.frame(
  Estimate = coef(multi_fit),
  StdError = summary(multi_fit)$coefficients[, "Std. Error"],
  zvalue = summary(multi_fit)$coefficients[, "z value"],
  pvalue = summary(multi_fit)$coefficients[, "Pr(>|z|)"],
  OddsRatio = exp(coef(multi_fit))
); coef_table
```

[R 코드 3-16]

```
new_obs <- data.frame(
  age = c(30, 30, 60, 60),
  bmi = c(22, 27, 22, 27),
  hp = factor(c(0, 1, 0, 1), levels = c(0, 1))
)

pred_probs <- predict(multi_fit, newdata = new_obs, type = "response")

cbind(new_obs, predicted_prob = round(pred_probs, 3))
```

분류 성능의 평가**[R 코드 3-17]**

```
pred_prob <- predict(multi_fit, newdata = data, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)
pred_class <- factor(pred_class, levels = c(0, 1))
```

[R 코드 3-18]

```
conf_mat <- table(predicted = pred_class, actual = data$diabetes); conf_mat
```

[R 코드 3-19]

```
TN <- conf_mat["0", "0"]
FN <- conf_mat["0", "1"]
FP <- conf_mat["1", "0"]
TP <- conf_mat["1", "1"]

accuracy <- (TP + TN) / sum(conf_mat)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)

cat("정확도: ", round(accuracy, 4), "\n")
cat("민감도: ", round(sensitivity, 4), "\n")
cat("특이도: ", round(specificity, 4), "\n")
```

[R 코드 3-20]

```
calc_metrics <- function(prob, actual, threshold) {
  pred <- factor(ifelse(prob > threshold, 1, 0), levels = c(0, 1))
  cm <- table(predicted = pred, actual = actual)
  TN <- cm["0", "0"]; FN <- cm["0", "1"]
  FP <- cm["1", "0"]; TP <- cm["1", "1"]
  data.frame(
    threshold = threshold,
    accuracy = (TP + TN) / sum(cm),
    sensitivity = TP / (TP + FN),
    specificity = TN / (TN + FP)
  )
}

thresholds <- c(0.3, 0.4, 0.5, 0.6, 0.7)
metrics_df <- do.call(rbind, lapply(
  thresholds, function(t) calc_metrics(pred_prob, data$diabetes, t)
))
metrics_df
```

[R 코드 3-21]

```
thr_seq <- seq(0.05, 0.95, by = 0.01)
trade_df <- do.call(rbind, lapply(thr_seq, function(t) calc_metrics(pred_prob,
  data$diabetes, t)))

trade_long <- trade_df %>%
  pivot_longer(cols = c(sensitivity, specificity), names_to = "metric", values_to =
    "value")

ggplot(trade_long, aes(x = threshold, y = value, color = metric)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("sensitivity" = "#E07B91", "specificity" = "#6888BE")) +
  ggtitle("임계값에 따른 민감도와 특이도") +
  theme_minimal() +
```

```
theme(plot.title = element_text(hjust = 0.5))
```

B.3.4 이웃에게 물어보는 분류 — 비모수적 접근(KNN)

변수 표준화

[R 코드 3-22]

```
set.seed(2025)
train_idx <- sample(seq_len(nrow(data)), size = floor(0.7 * nrow(data)))
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]

X_train_raw <- train_data %>%
  transmute(age, bmi, hp = as.numeric(as.character(hp)))
X_test_raw <- test_data %>%
  transmute(age, bmi, hp = as.numeric(as.character(hp)))
y_train <- train_data$diabetes
y_test <- test_data$diabetes

X_train <- scale(X_train_raw)
X_test <- scale(
  X_test_raw,
  center = attr(X_train, "scaled:center"),
  scale = attr(X_train, "scaled:scale")
)
```

KNN 실습

[R 코드 3-23]

```
library(class)
knn.pred <- knn(train = X_train, test = X_test, cl = y_train, k = 5)
mean(knn.pred == y_test)
```

거리 가중 KNN

[R 코드 3-24]

```
library(kknn)
kknn_fit <- kknn(
  diabetes ~ age + bmi + hp,
  train = train_data,
  test = test_data,
  k = 10,
  distance = 2,
  kernel = "optimal"
)
mean(fitted(kknn_fit) == test_data$diabetes)
```

K-최근접 이웃(KNN) 분류기

[R 코드 3-25]

```
X <- data %>%
  transmute(age, bmi, hp = as.numeric(as.character(hp)))
y <- data$diabetes
```

[R 코드 3-26]

```
set.seed(2025)
train_idx <- sample(seq_len(nrow(data)), size = floor(0.7 * nrow(data)))
X_train_raw <- X[train_idx, ]
X_test_raw <- X[-train_idx, ]
y_train <- y[train_idx]
y_test <- y[-train_idx]

X_train <- scale(X_train_raw)
X_test <- scale(
  X_test_raw,
  center = attr(X_train, "scaled:center"),
  scale = attr(X_train, "scaled:scale")
)
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]
```

[R 코드 3-27]

```
library(class)

knn_pred <- knn(train = X_train, test = X_test, cl = y_train, k = 5)
table(predicted = knn_pred, actual = y_test)
mean(knn_pred == y_test)
```

[R 코드 3-28]

```
library(class)

set.seed(2025)
k_vals <- c(1, 3, 5, 10, 20, 50, 100)
n_folds <- 5
folds <- sample(rep(seq_len(n_folds), length.out = nrow(X_train_raw)))

err_rates <- sapply(k_vals, function(k) {
  fold_err <- sapply(seq_len(n_folds), function(i) {
    fold_train <- X_train_raw[folds != i, , drop = FALSE]
    fold_valid <- X_train_raw[folds == i, , drop = FALSE]
    fold_train_scaled <- scale(fold_train)
    fold_valid_scaled <- scale(
      fold_valid,
      center = attr(fold_train_scaled, "scaled:center"),
      scale = attr(fold_train_scaled, "scaled:scale")
    )
```

```

    )
    pred <- knn(
      train = fold_train_scaled,
      test = fold_valid_scaled,
      cl = y_train[folds != i],
      k = k
    )
    mean(pred != y_train[folds == i])
  })
  mean(fold_err)
})

knn_results <- data.frame(K = k_vals, error_rate = err_rates)
knn_results

```

[R 코드 3-29]

```

ggplot(knn_results, aes(x = K, y = error_rate)) +
  geom_line(color = "#6888BE", linewidth = 1) +
  geom_point(color = "#6888BE", size = 2) +
  scale_x_log10() +
  ggtitle("K에 따른 훈련 자료 내부 교차검증 오류율") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

```

[R 코드 3-30]

```

set.seed(2025)
k_vals_cv <- c(1, 3, 5, 10, 15, 20, 30, 50, 100)
n_folds <- 5
folds <- sample(rep(seq_len(n_folds), length.out = nrow(X_train_raw)))

cv_err <- sapply(k_vals_cv, function(k) {
  fold_err <- sapply(seq_len(n_folds), function(i) {
    fold_train <- X_train_raw[folds != i, , drop = FALSE]
    fold_valid <- X_train_raw[folds == i, , drop = FALSE]
    fold_train_scaled <- scale(fold_train)
    fold_valid_scaled <- scale(
      fold_valid,
      center = attr(fold_train_scaled, "scaled:center"),
      scale = attr(fold_train_scaled, "scaled:scale")
    )
  })
  pred <- knn(
    train = fold_train_scaled,
    test = fold_valid_scaled,
    cl = y_train[folds != i],
    k = k
  )
  mean(pred != y_train[folds == i])
})
mean(fold_err)
})

cv_results <- data.frame(K = k_vals_cv, cv_error = cv_err)
cv_results

```

[R 코드 3-31]

```
best_k <- cv_results$K[which.min(cv_results$cv_error)]
cat("최적의 K:", best_k, "\n")
```

B.3.5 나무로 분류하기 — 단일 나무에서 앙상블까지

분류 나무 실습

[R 코드 3-32]

```
library(tree)
tree.fit <- tree(diabetes ~ age + bmi + hp, data = train_data)
plot(tree.fit)
text(tree.fit, pretty = 0)
```

XGBoost

[R 코드 3-33]

```
library(xgboost)

X_train_model <- model.matrix(~ age + bmi + hp - 1, data = train_data)
y_train_num <- as.numeric(as.character(train_data$diabetes))
dtrain <- xgb.DMatrix(data = X_train_model, label = y_train_num)
xgb.fit <- xgb.train(
  params = list(objective = "binary:logistic", eta = 0.05, max_depth = 3),
  data = dtrain,
  nrounds = 500
)
```

LightGBM

[R 코드 3-34]

```
library(lightgbm)

dtrain_lgb <- lgb.Dataset(data = X_train_model, label = y_train_num)
lgbm.fit <- lgb.train(
  params = list(objective = "binary", learning_rate = 0.05, num_leaves = 31),
  data = dtrain_lgb,
  nrounds = 500
)
```

CatBoost

[R 코드 3-35]

```
library(catboost)
```

```
train_pool <- catboost.load_pool(data = X_train_model, label = y_train_num)
cat_fit <- catboost.train(
  train_pool,
  params = list(
    loss_function = "Logloss",
    iterations = 500,
    learning_rate = 0.05,
    depth = 6
  )
)
```

분류 나무

[R 코드 3-36]

```
library(tree)
```

[R 코드 3-37]

```
tree_fit <- tree(diabetes ~ age + bmi + hp, data = data)
summary(tree_fit)
```

[R 코드 3-38]

```
plot(tree_fit)
text(tree_fit, pretty = 0, cex = 0.8)
```

[R 코드 3-39]

```
tree_pred <- predict(tree_fit, newdata = data, type = "class")
table(predicted = tree_pred, actual = data$diabetes)
mean(tree_pred == data$diabetes) # 훈련 정확도
```

[R 코드 3-40]

```
set.seed(2025)
# cv.tree()이 실행 환경과 무관하게 원자료를 찾도록 호출식에 자료를 보관한다.
tree_fit$call <- quote(tree(diabetes ~ age + bmi + hp, data = data))
tree_fit$call$data <- data
cv_tree <- cv.tree(tree_fit, FUN = prune.misclass)
cv_tree
```

[R 코드 3-41]

```
plot(cv_tree$size, cv_tree$dev, type = "b",
     xlab = "트리 크기", ylab = "교차검증 오분류율",
     col = "#6888BE", pch = 19)
```

[R 코드 3-42]

```
best_size <- cv_tree$size[which.min(cv_tree$dev)]
```

```
cat("최적 트리 크기:", best_size, "\n")

pruned_tree <- prune.misclass(tree_fit, best = best_size)
plot(pruned_tree); text(pruned_tree, pretty = 0, cex = 0.8)
```

랜덤 포레스트

[R 코드 3-43]

```
library(randomForest)
set.seed(2025)
rf_fit <- randomForest(diabetes ~ age + bmi + hp, data = data,
                       ntree = 500, mtry = 2, importance = TRUE)
rf_fit
```

[R 코드 3-44]

```
varImpPlot(rf_fit, main = "변수 중요도")
```

[R 코드 3-45]

```
importance(rf_fit)
```

[R 코드 3-46]

```
rf_pred <- predict(rf_fit, newdata = data)
table(predicted = rf_pred, actual = data$diabetes)
mean(rf_pred == data$diabetes) # 훈련 정확도
```

B.3.6 신경망 기반 분류

MLP (keras)

[R 코드 3-47]

```
library(keras)

X_train_nn <- model.matrix(~ age + bmi + hp - 1, data = train_data)
y_train_nn <- as.numeric(as.character(train_data$diabetes))

model <- keras_model_sequential() %>%
  layer_dense(units = 64, activation = "relu", input_shape = ncol(X_train_nn)) %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dense(units = 1, activation = "sigmoid")

model %>% compile(
  loss = "binary_crossentropy",
```

```

    optimizer = "adam",
    metrics = "accuracy"
  )
model %>% fit(
  X_train_nn,
  y_train_nn,
  epochs = 30,
  batch_size = 32,
  validation_split = 0.2
)

```

CNN (keras)

[R 코드 3-48]

```

library(keras)

# 이미지 분류용 별도 구조 예시이며 hn19 표형 자료에는 직접 적용하지 않는다.
num_classes <- 10
cnn <- keras_model_sequential() %>%
  layer_conv_2d(
    filters = 16, kernel_size = c(3, 3), padding = "same",
    activation = "relu", input_shape = c(28, 28, 1)
  ) %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_conv_2d(
    filters = 32, kernel_size = c(3, 3), padding = "same",
    activation = "relu"
  ) %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_flatten() %>%
  layer_dense(units = 64, activation = "relu") %>%
  layer_dense(units = num_classes, activation = "softmax")

cnn %>% compile(
  loss = "sparse_categorical_crossentropy",
  optimizer = "adam",
  metrics = "accuracy"
)

```

B.3.7 생성 모형 기반 분류

LDA 와 QDA

[R 코드 3-49]

```

library(MASS)

lda.fit <- lda(diabetes ~ age + bmi + hp, data = train_data)
qda.fit <- qda(diabetes ~ age + bmi + hp, data = train_data)
pred.lda <- predict(lda.fit, newdata = test_data)$class

```

```
pred.qda <- predict(qda.fit, newdata = test_data)$class
```

[R 코드 3-50]

```
library(MASS)
```

[R 코드 3-51]

```
lda_fit <- lda(diabetes ~ age + bmi + hp, data = data); lda_fit
```

[R 코드 3-52]

```
lda_pred <- predict(lda_fit, newdata = data)
table(predicted = lda_pred$class, actual = data$diabetes)
mean(lda_pred$class == data$diabetes) # 훈련 정확도
```

[R 코드 3-53]

```
qda_fit <- qda(diabetes ~ age + bmi + hp, data = data); qda_fit
```

[R 코드 3-54]

```
qda_pred <- predict(qda_fit, newdata = data)
table(predicted = qda_pred$class, actual = data$diabetes)
mean(qda_pred$class == data$diabetes) # 훈련 정확도
```

나이프 베이즈

[R 코드 3-55]

```
library(e1071)
nb_fit <- naiveBayes(diabetes ~ age + bmi + hp, data = data)
nb_fit
```

[R 코드 3-56]

```
nb_pred <- predict(nb_fit, newdata = data)
table(predicted = nb_pred, actual = data$diabetes)
mean(nb_pred == data$diabetes) # 훈련 정확도
```

여러 분류 방법의 성능 비교

[R 코드 3-57]

```
set.seed(2025)
train_idx <- sample(1:nrow(data), size = floor(0.7 * nrow(data)))
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]
```

[R 코드 3-58]

```
metrics <- function(pred, actual) {
  cm <- table(
    factor(pred, levels = c(0, 1)),
    factor(actual, levels = c(0, 1))
  )
  TN <- cm["0", "0"]; FN <- cm["0", "1"]
  FP <- cm["1", "0"]; TP <- cm["1", "1"]
  c(
    accuracy = (TP + TN) / sum(cm),
    sensitivity = TP / (TP + FN),
    specificity = TN / (TN + FP)
  )
}
```

[R 코드 3-59]

```
lr_fit <- glm(diabetes ~ age + bmi + hp, data = train_data, family = binomial)
lr_pred <- ifelse(predict(lr_fit, newdata = test_data, type = "response") > 0.5, 1, 0)

X_tr_raw <- train_data %>%
  transmute(age, bmi, hp = as.numeric(as.character(hp)))
X_te_raw <- test_data %>%
  transmute(age, bmi, hp = as.numeric(as.character(hp)))
X_tr <- scale(X_tr_raw)
X_te <- scale(
  X_te_raw,
  center = attr(X_tr, "scaled:center"),
  scale = attr(X_tr, "scaled:scale")
)
knn_pred_test <- knn(
  train = X_tr, test = X_te, cl = train_data$diabetes, k = best_k
)

tree_fit2 <- tree(diabetes ~ age + bmi + hp, data = train_data)
tree_pred_test <- predict(tree_fit2, newdata = test_data, type = "class")

set.seed(2025)
rf_fit2 <- randomForest(
  diabetes ~ age + bmi + hp, data = train_data, ntree = 500, mtry = 2
)
rf_pred_test <- predict(rf_fit2, newdata = test_data)

lda_pred_test <- predict(
  lda(diabetes ~ age + bmi + hp, data = train_data), newdata = test_data
)$class
qda_pred_test <- predict(
  qda(diabetes ~ age + bmi + hp, data = train_data), newdata = test_data
)$class
nb_pred_test <- predict(
  naiveBayes(diabetes ~ age + bmi + hp, data = train_data), newdata = test_data
)
```

[R 코드 3-60]

```
result_table <- rbind(
  Logistic = metrics(lr_pred, test_data$diabetes),
  KNN      = metrics(as.numeric(as.character(knn_pred_test)), test_data$diabetes),
  Tree     = metrics(as.numeric(as.character(tree_pred_test)), test_data$diabetes),
  RandomForest = metrics(as.numeric(as.character(rf_pred_test)), test_data$diabetes),
  LDA      = metrics(as.numeric(as.character(lda_pred_test)), test_data$diabetes),
  QDA      = metrics(as.numeric(as.character(qda_pred_test)), test_data$diabetes),
  NaiveBayes = metrics(as.numeric(as.character(nb_pred_test)), test_data$diabetes)
)
round(result_table, 4)
```

[R 코드 3-61]

```
result_df <- data.frame(model = rownames(result_table), accuracy = result_table[,
"accuracy"])
result_df$model <- factor(result_df$model, levels = result_df$model)

ggplot(result_df, aes(x = model, y = accuracy, fill = model)) +
  geom_bar(stat = "identity", alpha = 0.8) +
  geom_text(aes(label = round(accuracy, 3)), vjust = -0.3, size = 3.5) +
  ylim(0, 1) +
  ggtitle("분류기별 점정 정확도") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5),
        legend.position = "none",
        axis.text.x = element_text(angle = 30, hjust = 1))
```

B.3.8 실습 2: 고혈압 위험군 분류**실습 2: 국민건강영양조사 데이터를 이용한 고혈압 위험군 분류****분석을 위해 필요한 라이브러리 및 데이터 불러오기****[R 코드 3-62]**

```
rm(list = ls())

library(data.table)
library(tidyverse)

data <- fread("데이터/hn19.csv", encoding = "UTF-8")
head(data, n = 3)
```

[R 코드 3-63]

```
data <- data %>%
  dplyr::select(age, HE_BMI, HE_DM_HbA1c, HE_HP) %>%
  drop_na()

nrow(data)
```

[R 코드 3-64]

```
data <- data %>%
  mutate(
    hp = ifelse(HE_HP >= 2, 1, 0),
    dm = ifelse(HE_DM_HbA1c >= 2, 1, 0),
    bmi = HE_BMI
  ) %>%
  mutate(hp = as.factor(hp), dm = as.factor(dm)) %>%
  dplyr::select(age, bmi, dm, hp)

head(data, n = 3)
```

[R 코드 3-65]

```
table(data$hp)
prop.table(table(data$hp))
```

변수 간의 상관관계 및 분포 파악하기**[R 코드 3-66]**

```
plt1 <- ggplot(data, aes(x = hp, y = age, fill = hp)) +
  geom_boxplot(alpha = 0.6) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ggtitle("혈압 상태별 나이 분포") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5), legend.position = "none")

plt2 <- ggplot(data, aes(x = hp, y = bmi, fill = hp)) +
  geom_boxplot(alpha = 0.6) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ggtitle("혈압 상태별 BMI 분포") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5), legend.position = "none")

gridExtra::grid.arrange(plt1, plt2, ncol = 2)
```

[R 코드 3-67]

```
table(dm = data$dm, hp = data$hp)
```

[R 코드 3-68]

```
ggplot(data, aes(x = dm, fill = hp)) +
  geom_bar(position = "fill", alpha = 0.8) +
  scale_fill_manual(values = c("0" = "#6888BE", "1" = "#E07B91")) +
  ylab("비율") +
  ggtitle("당뇨 상태별 고혈압 비율") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```

단순 로지스틱 회귀모형 적합

[R 코드 3-69]

```
simple_fit <- glm(hp ~ bmi, data = data, family = binomial)
summary(simple_fit)
```

[R 코드 3-70]

```
exp(coef(simple_fit))
```

다중 로지스틱 회귀모형 적합

[R 코드 3-71]

```
multi_fit <- glm(hp ~ age + bmi + dm, data = data, family = binomial)
summary(multi_fit)
```

[R 코드 3-72]

```
coef_table <- data.frame(
  Estimate = coef(multi_fit),
  StdError = summary(multi_fit)$coefficients[, "Std. Error"],
  zvalue = summary(multi_fit)$coefficients[, "z value"],
  pvalue = summary(multi_fit)$coefficients[, "Pr(>|z|)"],
  OddsRatio = exp(coef(multi_fit))
); coef_table
```

[R 코드 3-73]

```
new_obs <- data.frame(
  age = c(30, 30, 60, 60),
  bmi = c(22, 27, 22, 27),
  dm = factor(c(0, 1, 0, 1), levels = c(0, 1))
)

pred_probs <- predict(multi_fit, newdata = new_obs, type = "response")
cbind(new_obs, predicted_prob = round(pred_probs, 3))
```

분류 성능의 평가

[R 코드 3-74]

```
pred_prob <- predict(multi_fit, newdata = data, type = "response")
pred_class <- factor(ifelse(pred_prob > 0.5, 1, 0), levels = c(0, 1))
conf_mat <- table(predicted = pred_class, actual = data$hp); conf_mat
```

[R 코드 3-75]

```
TN <- conf_mat["0", "0"]; FN <- conf_mat["0", "1"]
FP <- conf_mat["1", "0"]; TP <- conf_mat["1", "1"]
```

```
cat("정확도:   ", round((TP + TN) / sum(conf_mat), 4), "\n")
cat("민감도:   ", round(TP / (TP + FN), 4), "\n")
cat("특이도:   ", round(TN / (TN + FP), 4), "\n")
```

[R 코드 3-76]

```
calc_metrics <- function(prob, actual, threshold) {
  pred <- factor(ifelse(prob > threshold, 1, 0), levels = c(0, 1))
  cm <- table(predicted = pred, actual = actual)
  TN <- cm["0", "0"]; FN <- cm["0", "1"]
  FP <- cm["1", "0"]; TP <- cm["1", "1"]
  data.frame(
    threshold = threshold,
    accuracy = (TP + TN) / sum(cm),
    sensitivity = TP / (TP + FN),
    specificity = TN / (TN + FP)
  )
}

thr_seq <- seq(0.05, 0.95, by = 0.01)
trade_df <- do.call(rbind, lapply(
  thr_seq, function(t) calc_metrics(pred_prob, data$hp, t)
))
trade_long <- trade_df %>%
  pivot_longer(
    cols = c(sensitivity, specificity),
    names_to = "metric",
    values_to = "value"
  )

ggplot(trade_long, aes(x = threshold, y = value, color = metric)) +
  geom_line(linewidth = 1) +
  scale_color_manual(
    values = c("sensitivity" = "#E07B91", "specificity" = "#6888BE")
  ) +
  ggtitle("임계값에 따른 민감도와 특이도") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```

K-최근접 이웃(KNN) 분류기**[R 코드 3-77]**

```
X <- data %>%
  transmute(age, bmi, dm = as.numeric(as.character(dm)))
y <- data$hp
```

[R 코드 3-78]

```
set.seed(2025)
train_idx <- sample(seq_len(nrow(data)), size = floor(0.7 * nrow(data)))
```

```

X_train_raw <- X[train_idx, ]
X_test_raw <- X[-train_idx, ]
y_train <- y[train_idx]
y_test <- y[-train_idx]

X_train <- scale(X_train_raw)
X_test <- scale(
  X_test_raw,
  center = attr(X_train, "scaled:center"),
  scale = attr(X_train, "scaled:scale")
)

```

[R 코드 3-79]

```

library(class)

set.seed(2025)
k_vals_cv <- c(1, 3, 5, 10, 15, 20, 30, 50, 100)
n_folds <- 5
folds <- sample(rep(seq_len(n_folds), length.out = nrow(X_train_raw)))

cv_err <- sapply(k_vals_cv, function(k) {
  fold_err <- sapply(seq_len(n_folds), function(i) {
    fold_train <- X_train_raw[folds != i, , drop = FALSE]
    fold_valid <- X_train_raw[folds == i, , drop = FALSE]
    fold_train_scaled <- scale(fold_train)
    fold_valid_scaled <- scale(
      fold_valid,
      center = attr(fold_train_scaled, "scaled:center"),
      scale = attr(fold_train_scaled, "scaled:scale")
    )
    pred <- knn(
      train = fold_train_scaled,
      test = fold_valid_scaled,
      cl = y_train[folds != i],
      k = k
    )
    mean(pred != y_train[folds == i])
  })
  mean(fold_err)
})

cv_results <- data.frame(K = k_vals_cv, cv_error = cv_err)
cv_results

```

[R 코드 3-80]

```

best_k <- cv_results$K[which.min(cv_results$cv_error)]
cat("최적의 K:", best_k, "\n")

```

분류 나무**[R 코드 3-81]**

```
library(tree)
tree_fit <- tree(hp ~ age + bmi + dm, data = data)
summary(tree_fit)
```

[R 코드 3-82]

```
plot(tree_fit)
text(tree_fit, pretty = 0, cex = 0.8)
```

[R 코드 3-83]

```
set.seed(2025)
# cv.tree()이 실행 환경과 무관하게 원자료를 찾도록 호출식에 자료를 보관한다.
tree_fit$call <- quote(tree(hp ~ age + bmi + dm, data = data))
tree_fit$call$data <- data
cv_tree <- cv.tree(tree_fit, FUN = prune.misclass)
cv_tree
```

[R 코드 3-84]

```
best_size <- cv_tree$size[which.min(cv_tree$dev)]
cat("최적 트리 크기:", best_size, "\n")

pruned_tree <- prune.misclass(tree_fit, best = best_size)
plot(pruned_tree); text(pruned_tree, pretty = 0, cex = 0.8)
```

랜덤 포레스트

[R 코드 3-85]

```
library(randomForest)
set.seed(2025)
rf_fit <- randomForest(hp ~ age + bmi + dm, data = data,
                       ntree = 500, mtry = 2, importance = TRUE)
rf_fit
```

[R 코드 3-86]

```
varImpPlot(rf_fit, main = "변수 중요도")
```

[R 코드 3-87]

```
rf_pred <- predict(rf_fit, newdata = data)
table(predicted = rf_pred, actual = data$hp)
mean(rf_pred == data$hp) # 훈련 정확도
```

LDA 와 QDA

[R 코드 3-88]

```
library(MASS)
```

```
lda_fit <- lda(hp ~ age + bmi + dm, data = data)
lda_pred <- predict(lda_fit, newdata = data)
table(predicted = lda_pred$class, actual = data$hp)
mean(lda_pred$class == data$hp) # 훈련 정확도
```

[R 코드 3-89]

```
qda_fit <- qda(hp ~ age + bmi + dm, data = data)
qda_pred <- predict(qda_fit, newdata = data)
table(predicted = qda_pred$class, actual = data$hp)
mean(qda_pred$class == data$hp) # 훈련 정확도
```

나이프 베이즈**[R 코드 3-90]**

```
library(e1071)
nb_fit <- naiveBayes(hp ~ age + bmi + dm, data = data)
nb_fit
```

[R 코드 3-91]

```
nb_pred <- predict(nb_fit, newdata = data)
table(predicted = nb_pred, actual = data$hp)
mean(nb_pred == data$hp) # 훈련 정확도
```

여러 분류 방법의 성능 비교**[R 코드 3-92]**

```
set.seed(2025)
train_idx <- sample(seq_len(nrow(data)), size = floor(0.7 * nrow(data)))
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]

metrics <- function(pred, actual) {
  cm <- table(
    factor(pred, levels = c(0, 1)),
    factor(actual, levels = c(0, 1))
  )
  TN <- cm["0", "0"]; FN <- cm["0", "1"]
  FP <- cm["1", "0"]; TP <- cm["1", "1"]
  c(
    accuracy = (TP + TN) / sum(cm),
    sensitivity = TP / (TP + FN),
    specificity = TN / (TN + FP)
  )
}

# Logistic Regression
lr_fit <- glm(hp ~ age + bmi + dm, data = train_data, family = binomial)
```

```

lr_pred <- ifelse(
  predict(lr_fit, newdata = test_data, type = "response") > 0.5, 1, 0
)

# KNN: 다른 모형과 같은 age, bmi, dm 사용
X_tr_raw <- train_data %>%
  transmute(age, bmi, dm = as.numeric(as.character(dm)))
X_te_raw <- test_data %>%
  transmute(age, bmi, dm = as.numeric(as.character(dm)))
X_tr <- scale(X_tr_raw)
X_te <- scale(
  X_te_raw,
  center = attr(X_tr, "scaled:center"),
  scale = attr(X_tr, "scaled:scale")
)
knn_pred_test <- knn(
  train = X_tr, test = X_te, cl = train_data$hp, k = best_k
)

# Tree
tree_fit2 <- tree(hp ~ age + bmi + dm, data = train_data)
tree_pred_test <- predict(tree_fit2, newdata = test_data, type = "class")

# Random Forest
set.seed(2025)
rf_fit2 <- randomForest(
  hp ~ age + bmi + dm, data = train_data, ntree = 500, mtry = 2
)
rf_pred_test <- predict(rf_fit2, newdata = test_data)

# LDA, QDA, Naive Bayes
lda_pred_test <- predict(
  lda(hp ~ age + bmi + dm, data = train_data), newdata = test_data
)$class
qda_pred_test <- predict(
  qda(hp ~ age + bmi + dm, data = train_data), newdata = test_data
)$class
nb_pred_test <- predict(
  naiveBayes(hp ~ age + bmi + dm, data = train_data), newdata = test_data
)

```

[R 코드 3-93]

```

result_table <- rbind(
  Logistic = metrics(lr_pred, test_data$hp),
  KNN      = metrics(as.numeric(as.character(knn_pred_test)), test_data$hp),
  Tree     = metrics(as.numeric(as.character(tree_pred_test)), test_data$hp),
  RandomForest = metrics(as.numeric(as.character(rf_pred_test)), test_data$hp),
  LDA      = metrics(as.numeric(as.character(lda_pred_test)), test_data$hp),
  QDA      = metrics(as.numeric(as.character(qda_pred_test)), test_data$hp),
  NaiveBayes = metrics(as.numeric(as.character(nb_pred_test)), test_data$hp)
)
round(result_table, 4)

```

[R 코드 3-94]

```
result_df <- data.frame(model = rownames(result_table), accuracy = result_table[,  
"accuracy"])  
result_df$model <- factor(result_df$model, levels = result_df$model)  
  
ggplot(result_df, aes(x = model, y = accuracy, fill = model)) +  
  geom_bar(stat = "identity", alpha = 0.8) +  
  geom_text(aes(label = round(accuracy, 3)), vjust = -0.3, size = 3.5) +  
  ylim(0, 1) +  
  ggtitle("분류기별 검정 정확도") +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5),  
        legend.position = "none",  
        axis.text.x = element_text(angle = 30, hjust = 1))
```

B.4 4 부 실습

B.4.1 확률과 확률분포

앞서 우리는 실제 세계의 데이터를 설명하기 위해 이론 세계의 도구인 '확률분포'를 도입했다. 이제 R을 이용해 이 추상적인 개념들을 구체적인 숫자로 확인해 보자. 이 실습에서는 가장 대표적인 이산형 분포인 **이항분포**와 연속형 분포인 **정규분포**를 다룬다.

이항분포: 동전 던지기의 확률 계산

성공 확률이 50%($p = 0.5$)인 공정한 동전을 10번 던지는 실험($n = 10$)을 생각해 보자. 이때 앞면이 나오는 횟수를 확률변수 X 라고 할 때, 우리는 `dbinom`(확률질량함수)과 `pbinom`(누적분포함수) 함수를 이용해 이론적인 확률을 계산할 수 있다.

[R 코드 4-1]

```
# 설정: 시행 횟수 n=10, 성공 확률 p=0.5
n <- 10
p <- 0.5

# 1) P(X = 2): 앞면이 정확히 2회 나올 확률
# dbinom의 'd'는 Density(확률 밀도/질량)를 의미합니다.
prob_exact <- dbinom(2, size = n, prob = p)
cat(sprintf("P(X = 2) = %.4f\n", prob_exact))

# 2) P(X <= 3): 앞면이 3회 이하로 나올 확률
# pbinom의 'p'는 Probability(누적 확률)를 의미합니다.
prob_less_equal <- pbinom(3, size = n, prob = p)
cat(sprintf("P(X <= 3) = %.4f\n", prob_less_equal))

# 3) P(X > 7): 앞면이 7회 초과로 나올 확률
# lower.tail = FALSE 옵션을 사용하면 'X > k'의 확률을 바로 구합니다.
prob_over <- pbinom(7, size = n, prob = p, lower.tail = FALSE)
cat(sprintf("P(X > 7) = %.4f\n", prob_over))
```

결과는 다음과 같다.

```
P(X = 2) = 0.0439
```

```
P(X <= 3) = 0.1719
```

```
P(X > 7) = 0.0547
```

분석 결과:

- $P(X = 2)$: 동전을 10 번 던져 앞면이 고작 2 번 나올 확률은 약 0.0439(4.4%)로 꽤 드문 일이다.
- $P(X \leq 3)$: 앞면이 3 번 이하로 나올 확률은 약 0.1719(17.2%)다.
- $P(X > 7)$: 반대로 앞면이 8 번, 9 번, 10 번 나올 확률, 즉 7 번을 초과할 확률은 0.0547(5.5%)로 계산된다.

정규분포: 이론과 현실의 대조

다음으로 통계학의 꽃인 정규분포를 살펴보자. 평균이 0이고 표준편차가 1인 **표준정규분포**($Z \sim N(0,1)$)에서 데이터를 무작위로 추출(Simulation)해 보자. 이론적으로 완벽한 곡선(PDF)과 무작위로 뽑힌 데이터의 분포(히스토그램)가 어떻게 다른지 비교하는 것이 핵심이다.

[R 코드 4-2]

```
library(ggplot2)

# 1. 설정 및 데이터 생성
set.seed(123) # 결과 재현을 위해 시드 설정
mu <- 0
sigma <- 1
size <- 1000

# 현실의 데이터 시뮬레이션 (경험적 데이터)
simulated_data <- data.frame(value = rnorm(size, mean = mu, sd = sigma))

# 2. 시각화: 경험적 분포 vs 이론적 분포
ggplot(simulated_data, aes(x = value)) +
  # 히스토그램: 실제 수집된 데이터의 모습
  geom_histogram(aes(y = ..density..), bins = 30,
                 fill = "skyblue", color = "black", alpha = 0.7) +
  # 이론적 밀도 곡선 (정규분포 곡선)
  stat_function(fun = dnorm, args = list(mean = mu, sd = sigma),
               color = "red", linewidth = 1) +
```

```
labs(title = paste0("정규분포 시뮬레이션 비교 (mu=", mu, ", sigma=", sigma, ")"),
     x = "X 값", y = "밀도") +
theme_minimal()
```

[그림 B-4-1] 표준정규분포의 경험적 분포(히스토그램)와 이론적 분포(선 그래프)

[그림 B-4-1]에서 붉은색 실선은 수학적으로 정의된 이론적 분포다. 반면 파란색 막대는 우리가 컴퓨터를 통해 무작위로 뽑은 1,000개의 데이터(경험적 분포)다. 데이터의 수가 많아질수록 막대 그래프의 모양은 붉은 실선에 점점 더 가까워진다. 이것이 통계적 추론이 가능한 이유다. 우리는 불확실한 데이터(파란 막대)를 보고, 그 배후에 있는 이론적 진실(붉은 선)을 추측해 낸다.

분포의 모양을 결정하는 두 가지 열쇠: μ 와 σ

정규분포의 모양은 평균(μ)과 표준편차(σ)에 의해 결정된다. 이 파라미터들이 변할 때 분포가 어떻게 바뀌는지 확인해 보자. 다음 코드는 $\mu = 0, 10$, $\sigma = 1, 4$ 각각 경우의 정규분포로부터 표집한 1000개의 표본을 보여준다.

[R 코드 4-3]

```
library(ggplot2)
library(dplyr)
library(tidyr)

set.seed(123)
mu_vals <- c(0, 10)
sigma_vals <- c(1, 4)
sim_size <- 1000

sim_data <- expand.grid(mu = mu_vals, sigma = sigma_vals) %>%
  group_by(mu, sigma) %>%
  reframe(value = rnorm(sim_size, mean = mu, sd = sigma))

theoretical_curves <- sim_data %>%
  group_by(mu, sigma) %>%
  reframe(
    x = seq(min(value), max(value), length.out = 200),
```

```

  y = dnorm(x, mean = first(mu), sd = first(sigma))
)

ggplot() +
  geom_histogram(
    data = sim_data,
    aes(x = value, y = after_stat(density)),
    bins = 30, fill = "skyblue", color = "black", alpha = 0.6
  ) +
  geom_line(
    data = theoretical_curves,
    aes(x = x, y = y),
    color = "red", linewidth = 1
  ) +
  facet_grid(mu ~ sigma, labeller = label_both) +
  labs(
    title = "정규분포 파라미터 변화 시뮬레이션 (맞춤형 밀도 곡선)",
    subtitle = "빨간 선은 각 파라미터 조합에 따른 이론적 정규분포 곡선",
    x = "X 값", y = "밀도"
  ) +
  theme_minimal()

```

[그림 B-4-2] 파라미터에 따른 정규분포의 변화

[그림 B-4-2]를 보면, μ 가 변하면 분포의 중심이 이동하고, σ 가 커지면 분포가 옆으로 퍼지면서 높이가 낮아지는 것을 확인할 수 있다. (변화가 없는 것처럼 보인다면, X축의 위치를 확인해 보자.)

정규분포의 확률 계산: 중요한 숫자들

우리가 추론에서 자주 사용하게 될 중요한 확률값들을 계산해 보자. 특히 **1.96**이라는 숫자에 주목하라.

[R 코드 4-4]

```

# 설정: 표준정규분포 (mu=0, sigma=1)
mu <- 0
sigma <- 1

```

```
# 1) P(X <= 1.64): 상위 5% 지점 (단측 검정 기준)
prob1 <- pnorm(1.64, mean = mu, sd = sigma)
cat(sprintf("P(X <= 1.64) = %.4f\n", prob1))

# 2) P(-1 < X < 1): 평균에서 1표준편차 이내에 있을 확률
# (1표준편차 구간의 법칙: 약 68%)
prob2 <- pnorm(1, mean = mu, sd = sigma) - pnorm(-1, mean = mu, sd = sigma)
cat(sprintf("P(-1 < X < 1) = %.4f\n", prob2))

# 3) P(|X| >= 1.96): 양쪽 꼬리 확률의 합 (95% 신뢰구간 외곽)
# lower.tail = FALSE는 P(X > 1.96)을 의미합니다.
prob3 <- pnorm(-1.96, mean = mu, sd = sigma) + pnorm(1.96, mean = mu, sd = sigma, lower.tail = FALSE)
cat(sprintf("P(|X| >= 1.96) = %.4f\n", prob3))
```

```
P(X <= 1.64) = 0.9495
P(-1 < X < 1) = 0.6827
P(|X| >= 1.96) = 0.0500
```

핵심 결과:

- $P(|X| \geq 1.96) \approx 0.05$: 표준정규분포에서 1.96 보다 크거나 -1.96 보다 작은 값은 전체의 5%에 불과하다. 즉, 데이터의 95%는 -1.96 과 1.96 사이에 존재한다. 이것이 우리가 95% 신뢰구간을 구할 때 '표준오차의 약 2 배(정확히는 1.96 배)'를 사용하는 수학적 근거다.

분위수(Quantile) 구하기

마지막으로, 확률이 주어졌을 때 그에 해당하는 x 값을 찾아보자. 이를 분위수라고 한다.

[R 코드 4-5]

```
# 설정: 표준정규분포 (mu=0, sigma=1)
mu <- 0
sigma <- 1

# 1) 구하고자 하는 분위수 설정
quantiles <- c(0.25, 0.5, 0.75, 0.975)
```

```
# 2) 이론적 분위수 계산 (Percent Point Function -> qnorm)
q_values <- qnorm(quantiles, mean = mu, sd = sigma)

# 3) 결과 출력
for (i in seq_along(quantiles)) {
  cat(sprintf("%g%% 이론적 분위수: X = %.4f\n", quantiles[i] * 100, q_values[i]))
}
```

```
25% 이론적 분위수: X = -0.6745
50% 이론적 분위수: X = 0.0000
75% 이론적 분위수: X = 0.6745
97.5% 이론적 분위수: X = 1.9600
```

결과를 보면 50% 분위수는 평균인 0이 나오며, 25%와 75% 분위수는 0을 중심으로 대칭인 값을 가진다는 것을 확인할 수 있다.

B.4.2 모집단과 표집

통계적 추론의 핵심은 '부분(표본)'을 통해 '전체(모집단)'를 짐작하는 것이다. 이 개념을 명확히 이해하기 위해, 우리는 2023년 보스턴 마라톤 대회 데이터를 활용한 시뮬레이션을 진행해 보자.

여기서 우리는 **전체 참가자 데이터를 이미 가지고 있다고** 가정한다. 즉, 우리는 신(God)의 관점에서 모집단의 모든 정보를 알고 있다. 이 상태에서 일부를 무작위로 뽑았을 때, 과연 그 표본이 모집단을 얼마나 잘 닮아 있는지 확인해 보는 것이 이번 실습의 목표다.

데이터 준비: 모집단 정의하기

먼저 데이터를 불러온다. 분석의 대상을 명확히 하기 위해, 전체 참가자 중 **'40-44세 여성'** 그룹만 추출하여 이를 우리의 **관심 모집단(Population)**으로 정의하자.

[R 코드 4-6]

```
library(readr)
library(dplyr)
library(tidyr)

boston <- read_csv("데이터/boston_marathon_2023.csv")
```

```
boston40 <- boston %>%
  drop_na(age_group, gender, finish_net_minutes) %>%
  filter(age_group == "40-44", gender == "W")

cat(sprintf("모집단 크기: %d명\n", nrow(boston40)))
boston40 %>%
  dplyr::select(age_group, gender, finish_net_minutes) %>%
  head(5)
```

모집단 크기: 1878명

	age_group	gender	finish_net_minutes
20059	40-44	W	145.800000
20060	40-44	W	154.666667
20061	40-44	W	159.266667
20062	40-44	W	165.366667
20063	40-44	W	165.683333

이 코드를 실행하면 1,878명의 데이터가 추출된다. 이 1,878명이 우리가 알고자 하는 전체 모집단이다.

모집단의 분포 확인

이제 모집단 전체의 완주 시간 기록이 어떻게 분포되어 있는지 히스토그램으로 확인해 보자. 이것은 우리가 현실에서는 결코 알 수 없는 '진실(Truth)'의 모습이다.

[R 코드 4-7]

```
library(ggplot2)

# 2. 모집단의 히스토그램 그리기 (40-44세 여성 전체)
# aes(x = ...)를 통해 어떤 변수를 X축에 놓을지 결정합니다.
ggplot(boston40, aes(x = finish_net_minutes)) +
```

```
# binwidth = 5는 5분 단위로 막대를 묶겠다는 의미입니다.
geom_histogram(binwidth = 5, fill = "lightcoral",
               color = "black", alpha = 0.8) +
# 제목과 축 레이블 설정
labs(title = "2023 보스턴 마라톤: 40-44세 여성 완주 시간 분포 (모집단)",
     x = "완주 시간 (분)",
     y = "빈도") +
# 깔끔한 배경 설정 (격자선 포함)
theme_minimal() +
# 격자선의 스타일을 파이썬의 linestyle='--'처럼 점선으로 조정
theme(panel.grid.major = element_line(linetype = "dashed", color = "gray80"))
```

[그림 B-4-3] 2023 보스턴 마라톤 40-44세 여성 완주 시간 분포

전체 분포는 평균(약 230분)을 중심으로 종 모양과 유사하게 퍼져 있음을 알 수 있다.

표집(Sampling)

이제 현실의 상황을 가정해 보자. 우리는 시간과 비용의 제약으로 전체 1,878명을 모두 조사할 수 없다. 대신 무작위로 100명($n = 100$)을 뽑아 그 기록을 확인한다고 하자. 이를 표집(Sampling)이라고 한다.

결과의 재현성을 위해 seed를 설정하여 추출한다.

[R 코드 4-8]

```
library(dplyr)

# 3. 표본 추출: 재현성을 위해 seed 설정 후 100개 표본 추출
# set.seed는 무작위 번호 생성기를 고정하여 나중에 똑같은 표본을 뽑을 수 있게 합니다.
set.seed(123)

# boston40 데이터에서 무작위로 100명을 추출하여 sample_data에 저장합니다.
sample_data <- boston40 %>% sample_n(100)

# 추출된 100명의 평균 완주 시간을 계산합니다.
sample_mean <- mean(sample_data$finish_net_minutes)
```

```
# 결과 출력 (cat과 sprintf를 사용하여 파이썬의 f-string처럼 깔끔하게 출력)
cat(sprintf("표본 평균: %.2f분\n", sample_mean))
```

```
표본 평균: 223.27분
```

객체 `sample_data`에 우리의 표본인 100개의 관측값이 저장되어 있다.

표본의 분포: 부분은 전체를 닮았는가?

우리가 손에 쥐게 된 100명의 데이터, 즉 **표본(Sample)**의 분포를 그려보자. 과연 이 작은 조각이 거대한 모집단의 모습을 얼마나 잘 반영하고 있을까?

```
library(ggplot2)

# 4. 표본의 상대빈도 분포 및 밀도 곡선 그리기
# ggplot의 aes(x = ...) 내에 분석할 변수를 지정합니다.
ggplot(sample_data, aes(x = finish_net_minutes)) +
  # 히스토그램: y축을 밀도(density)로 설정하여 전체 면적의 합이 1이 되게 합니다.
  geom_histogram(aes(y = after_stat(density)), binwidth = 5,
                 fill = "steelblue", color = "black", alpha = 0.8) +
  # 밀도 곡선(KDE): 데이터의 전체적인 흐름을 부드러운 선으로 연결합니다.
  geom_density(color = "red", size = 1) +
  # 제목과 축 레이블 설정
  labs(title = "표본의 상대빈도 분포 (100개 표본)",
       x = "완주 시간 (분)",
       y = "밀도 (상대빈도)") +
  # 격자선 스타일을 점선으로 조정하고 미니멀 테마 적용
  theme_minimal() +
  theme(panel.grid.major = element_line(linetype = "dashed", color = "gray80"))
```

[그림 B-4-4] 표본의 상대빈도 분포와 밀도 곡선

[그림 B-4-4]에서 표본의 분포는 모집단과 비슷해 보이지만 완벽히 같지는 않다. 울퉁불퉁한 정도나 봉우리의 높이가 미세하게 다르다. 이것이 바로 표본의 변동성(Sampling Variability)이다. `set.seed()`의 난수 시드를 바꾸어 다시 100명을 뽑으면 표본 평균과 분포 모양도 달라진다.

B.4.3 표본크기와 표본의 변동성

앞선 실습에서 우리는 100명의 표본을 뽑아 평균을 계산했다. 하지만 만약 우리가 타임머신을 타고 시간을 되돌려 다시 100명을 뽑는다면, 과연 똑같은 평균이 나올까? 당연히 아닐 것이다. 구성원이 달라지므로 평균값도 달라질 수밖에 없다.

이처럼 표집을 반복했을 때 계산된 평균들이 어떻게 퍼져 있는지를 보여주는 것이 표집 분포(Sampling Distribution)다. 이번 실습에서는 R 시뮬레이션을 통해 표본 크기 n 이 커질수록 분포가 어떻게 변하는지 확인한다.

기준점 확인: 모집단의 참값

시뮬레이션의 결과를 평가하기 위해, 먼저 우리가 이미 알고 있는 모집단(40-44세 여성 참가자 전체)의 진짜 평균 기록(μ)을 계산해 둔다. 현실에서는 알 수 없는 '신의 영역'에 있는 값이지만, 시뮬레이션에서는 이 값이 기준점이 된다.

[R 코드 4-9]

```
# 모집단 평균 계산 (참값)
# mean() 함수는 벡터의 산술 평균을 계산합니다.
pop_mean <- mean(boston40$finish_net_minutes)

# 결과 출력: %.2f는 소수점 둘째 자리까지 표시하라는 의미입니다.
cat(sprintf("모집단 평균: %.2f분\n", pop_mean))
```

모집단 평균: 230.74분

모집단의 평균은 약 230.74분이다. 이제 우리가 뽑는 표본들의 평균이 이 값 주변에 어떻게 분포하는지 살펴볼 것이다.

"오늘을 다시 사는" 함수 정의

현실에서는 데이터를 한 번만 수집할 수 있지만, 컴퓨터 세상에서는 얼마든지 가상의 표집을 반복할 수 있다. 다음 함수는 모집단에서 지정된 크기(sample_size)만큼 표본을 뽑아 평균을 구하는 작업을 수천 번(num_simulations) 반복하여 리스트로 반환한다.

[R 코드 4-10]

```
# 표집 분포 시뮬레이션 함수 정의
simulate_sampling_distribution <- function(population_df, sample_size, num_simulations) {
```

```
# replicate()는 지정된 횟수(num_simulations)만큼 블록 안의 코드를 반복하고
# 그 결과값들을 모아 하나의 벡터로 반환합니다.
sample_means <- replicate(num_simulations, {

  # 모집단의 'finish_net_minutes' 컬럼에서 무작위 표본 추출 (복원 추출: replace = TRUE)
  s_data <- sample(population_df$finish_net_minutes, size = sample_size, replace = TRUE)

  # 추출된 표본의 평균 계산
  mean(s_data)
})

return(sample_means)
}
```

시뮬레이션: $n = 100$ vs $n = 400$

이제 두 가지 시나리오를 비교해 보자.

- $n = 100$: 100 명씩 뽑는 조사를 5,000 번 반복한다.
- $n = 400$: 400 명씩 뽑는 조사를 5,000 번 반복한다.

데이터의 개수가 4배 늘어났을 때, 평균들의 분포가 어떻게 달라지는지 확인하는 것이 핵심이다.

[R 코드 4-11]

```
# 1. 시뮬레이션 설정
num_simulations <- 5000

# 2. 시뮬레이션 실행 (앞서 정의한 함수 사용)
means_100 <- simulate_sampling_distribution(boston40, sample_size = 100, num_simulations =
num_simulations)
means_400 <- simulate_sampling_distribution(boston40, sample_size = 400, num_simulations =
num_simulations)
```

시각화 및 결과 해석: 대수의 법칙 확인

이제 두 시나리오의 결과를 히스토그램과 밀도 곡선으로 그려 비교해 보자. 붉은색 점선은 모집단의 진짜 평균이다.

[R 코드 4-12]

```

sim_results <- data.frame(
  mean_val = c(means_100, means_400),
  n = factor(
    rep(c("n=100", "n=400"), each = num_simulations),
    levels = c("n=100", "n=400")
  )
)

min_x <- min(sim_results$mean_val)
max_x <- max(sim_results$mean_val)

ggplot(sim_results, aes(x = mean_val, fill = n)) +
  geom_histogram(
    aes(y = after_stat(density)), bins = 30, color = "black", alpha = 0.7
  ) +
  geom_density(aes(color = n), linewidth = 1, show.legend = FALSE) +
  geom_vline(
    xintercept = pop_mean, color = "red", linetype = "dashed", linewidth = 1
  ) +
  scale_fill_manual(values = c("n=100" = "skyblue", "n=400" = "lightgreen")) +
  scale_color_manual(values = c("n=100" = "blue", "n=400" = "darkgreen")) +
  facet_wrap(~ n, ncol = 1) +
  coord_cartesian(xlim = c(min_x, max_x)) +
  labs(
    title = "표본 크기에 따른 표집 분포 비교",
    subtitle = "데이터가 4배 늘어날 때 표집 분포의 너비가 어떻게 변하는지 확인하세요.",
    x = "표본 평균 (분)", y = "밀도"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    strip.text = element_text(face = "bold", size = 12),
    panel.grid.major = element_line(linetype = "dashed", color = "gray80")
  )

```

[그림 B-4-5] 표본 크기에 따른 표본평균 분포 비교

B.4.4 신뢰구간: 100 개의 그물 던지기 시뮬레이션

앞서, 95% 신뢰구간의 진짜 의미는 "동일한 방법으로 표집과 구간 계산을 100번 반복한다면, 그중 약 95개의 그물은 참값을 안전하게 가두겠지만, 5개 정도는 진실을 놓칠 것이다"라고 했다.

이 철학적 개념을 코드로 증명해 보자. 모집단(boston40)의 평균을 알고 있는 상태에서, 100명씩 뽑아 신뢰구간을 만드는 과정을 100번 반복해 본다. 과연 몇 개의 그물이 진실을 가두는지 확인해 보자.

[R 코드 4-13]

```
# 100번의 신뢰구간 생성 시뮬레이션
pop_mean <- mean(boston40$finish_net_minutes)

ci_results <- replicate(100, {
  s <- sample(boston40$finish_net_minutes, 100)
  se <- sd(s) / sqrt(100)
  low <- mean(s) - 1.96 * se
  high <- mean(s) + 1.96 * se
  (low <= pop_mean & pop_mean <= high) # 포함 여부 반환
})

cat("100개의 신뢰구간 중 참값을 포함한 개수:", sum(ci_results))
```

100개의 신뢰구간 중 참값을 포함한 개수가 대략 95개인 것을 확인하자.

B.4.5 관악초등학교 학생의 평균 에너지 섭취량에 대한 신뢰구간

관악초등학교 학생의 에너지 섭취량 추정 예제를 파이썬으로 구현하여 t-분포를 사용한 신뢰구간을 계산해 보고 결측값(Missing Value)는 어떻게 처리해야 하는지 확인해 보자.

데이터 입력 및 전처리: 결측값과의 조우

우리는 관악초등학교 소득 상위 20% 가구 학생 10명을 무작위로 선정하여 조사했다. 하지만 현실 조사에서는 늘 예상치 못한 변수가 발생한다. 8번 학생이 응답을 거부하거나 기록이 누락된 것이다.

먼저 수집된 데이터를 입력하고, 분석 가능한 상태로 만들어보자. R에서는 결측값을 `NA` (Not Available)라는 전용 상수로 표현하며, 이를 처리하는 방식이 매우 직관적이다.

[R 코드 4-14]

```
# 1. 데이터 입력 (10명, 8번 학생은 결측)
# R에서는 결측값을 NA로 직접 입력합니다.
raw_data <- c(1531, 1809, 2094, 1625, 1115, 1630, 1963, NA, 1744, 2485)

# 2. 전처리: 결측값 제거
# is.na() 함수는 결측값이면 TRUE를 반환합니다.
# 앞에 느낌표(!)를 붙여 "결측값이 아닌(NOT) 것"만 골라냅니다.
data <- raw_data[!is.na(raw_data)]

# 결과 출력
cat(sprintf("원래 표본 크기: %d명\n", length(raw_data)))
cat(sprintf("유효 표본 크기(n): %d명\n", length(data)))
cat("데이터:", data, "\n")
```

```
원래 표본 크기: 10명
유효 표본 크기(n): 9명
데이터: [1531. 1809. 2094. 1625. 1115. 1630. 1963. 1744. 2485.]
```

원래 10명을 계획했으나, 유효한 표본의 크기(n)는 9명이 되었다. 이제 이 9명의 데이터만이 우리가 가진 정보의 전부다.

기초 통계량 계산

이제 9명의 데이터를 바탕으로 표본평균($\bar{x}$)과 표본표준편차(s)를 계산한다. 파이썬에서는 표준편차 계산 시 자유도($ddof=1$)를 별도로 지정해야 하지만, R의 `sd()` 함수는 태생부터 통계 추론용으로 설계되어 별도 설정 없이도 표본표준편차($n - 1$)를 계산한다. 이 점이 R이 통계학자들에게 사랑받는 이유이기도 하다.

[R 코드 4-15]

```
# 3. 통계량 계산
n <- length(data)
x_bar <- mean(data)
```

```
# R의 sd()는 기본적으로 n-1(자유도)을 사용하여 표본표준편차를 구합니다.
s <- sd(data)
```

```
# 4. 표준오차(SE) 계산
# 표본의 변동성(s)을 데이터 개수의 제곱근으로 나누어 줍니다.
se <- s / sqrt(n)
```

```
# 결과 출력
cat(rep("-", 30), "\n", sep = "")
cat(sprintf("표본 평균(x_bar): %.2f kcal\n", x_bar))
cat(sprintf("표본 표준편차(s): %.2f kcal\n", s))
cat(sprintf("표준오차(SE): %.2f kcal\n", se))
cat(rep("-", 30), "\n", sep = "")
```

```
-----
표본 평균(x_bar): 1777.33 kcal
표본 표준편차(s): 384.37 kcal
표준오차(SE): 128.12 kcal
-----
```

t-분포를 사용한 신뢰구간의 계산

자유도($df = n - 1 = 8$)를 고려한 t-값을 계산하고, 관악초등학교 학생의 평균 에너지 섭취량에 대한 신뢰구간을 구한다.

[R 코드 4-16]

```
# 5. 신뢰구간 계산 (t-분포 사용)
alpha <- 0.05
df <- n - 1

# t-분포의 임계값 구하기 (파이썬의 ppf와 대응)
# 상위 2.5% 지점을 찾기 위해 1 - alpha/2를 입력합니다.
t_crit <- qt(1 - alpha/2, df = df)

# 오차 범위(Margin of Error) 계산
margin_t <- t_crit * se
```

```
# 신뢰구간의 하한과 상한
lower_t <- x_bar - margin_t
upper_t <- x_bar + margin_t

# 결과 출력
cat(sprintf("Wn자유도(df): %dWn", df))
cat(sprintf("t-임계값: %.4fWn", t_crit)) # Z값 1.96보다 크다는 점이 핵심
cat(sprintf("[t-분포 사용 시] 95%% 신뢰구간: [%.2f, %.2f]Wn", lower_t, upper_t))
cat(sprintf("구간의 폭: %.2fWn", upper_t - lower_t))

자유도(df): 8
t-임계값: 2.3060
[t-분포 사용 시] 95% 신뢰구간: [1481.88, 2072.79]
구간의 폭: 590.91
```

B.4.6 일표본 모중앙값에 대한 추론 (신뢰구간)

앞서 우리는 평균에 대한 추론을 다루었다. 하지만 소득이나 에너지 섭취량처럼 데이터가 한쪽으로 치우친 경우, 평균보다는 **중앙값(Median)**이 집단의 대표값으로 더 적절할 때가 많다.

문제는 여기서 발생한다. 표본평균은 중심극한정리라는 강력한 이론 덕분에 표준오차를 쉽게 구할 수 있고, 이를 통해 신뢰구간을 계산하는 공식($\bar{x} \pm 2SE$)이 존재한다. 하지만 **표본중앙값의 표준오차**를 구하는 공식은 매우 복잡하거나, 데이터의 분포에 대한 까다로운 가정을 필요로 한다. 공식이 없거나 복잡할 때, 우리는 어떻게 신뢰구간을 구할 수 있을까? 이때야말로 **부트스트랩**이 빛을 발하는 순간이다. 복잡한 수식 없이 컴퓨터의 연산 능력만으로 중앙값의 신뢰구간을 구해 보자.

데이터 준비: 분석 대상 정의하기

이번 실습에서는 국민건강영양조사(KNHANES) 2019년 데이터를 사용한다. 분석의 초점을 명확히 하기 위해, **'20세 이하 남성'** 그룹을 특정하여 그들의 에너지 섭취량(N_EN)을 분석 대상으로 삼았다.

[R 코드 4-17]

```
library(readr)
library(dplyr)

# 1. 데이터 불러오기
hn19 <- read_csv("데이터/hn19.csv")
```

```
# 2. 데이터 필터링: 에너지 섭취량이 기록된 20세 이하 남성
# filter() 함수 내에서 쉼표(,)는 AND 조건을 의미합니다.
hn19_15 <- hn19 %>%
  filter(!is.na(N_EN), # 에너지 섭취량(N_EN)이 결측값이 아닌 경우
         sex == 1,     # 성별이 남성(1)인 경우
         age <= 20)    # 연령이 20세 이하인 경우

# 3. 100명의 표본 추출 (재현성을 위해 seed 설정)
set.seed(123)

# slice_sample()은 최근 dplyr에서 권장하는 샘플링 함수입니다.
sampled_data <- hn19_15 %>%
  slice_sample(n = 100, replace = FALSE)
```

분석을 간단히 하기 위해 위 코드에서는 100명의 표본을 추출했다. 이제 이 100명의 데이터만 가지고 전체 20세 이하 남성의 에너지 섭취량 중앙값이 어디에 있을지 추론해야 한다.

부트스트랩 함수 정의: 컴퓨터로 만드는 가상의 표본들

중앙값의 표본분포를 알 수 없으므로, 우리는 **부트스트랩 재표집**을 통해 이를 직접 만들어낸다. 원리는 간단하다.

- 우리가 가진 100 명의 표본에서 복원 추출(중복 허용)을 통해 다시 100 명을 뽑는다.
- 이 가상의 표본(부트스트랩 표본)에서 중앙값을 계산한다.
- 이 과정을 수천 번 반복하여 중앙값들의 분포(부트스트랩 분포)를 만든다.

[R 코드 4-18]

```
# 부트스트랩 중앙값 신뢰구간 함수 정의
bootstrap_median_ci <- function(data, B = 2000, conf = 0.95) {
  n <- length(data)
  medians <- replicate(B, {
    sample_i <- sample(data, size = n, replace = TRUE)
    median(sample_i)
  })
  alpha <- 1 - conf
  ci <- quantile(medians, probs = c(alpha / 2, 1 - alpha / 2))
}
```

```
estimate <- median(data)
list(
  estimate = round(estimate, 3),
  ci_lower = round(as.numeric(ci[1]), 3),
  ci_upper = round(as.numeric(ci[2]), 3)
)
}
```

신뢰구간 계산 및 해석

이제 함수를 실행해 95% 신뢰구간을 구해보자. 반복 횟수 B 는 1,999번으로 설정한다. (원래 표본의 추정값 1개를 포함하여 총 2,000개를 맞추기 위함이다.)

[R 코드 4-19]

```
# 부트스트랩 재표집을 2,000회 반복한다.
set.seed(123)
result <- bootstrap_median_ci(sampled_data$N_EN, B = 2000, conf = 0.95)

print(result)
cat(sprintf(
  "에너지 섭취량 중앙값 추정치: %.1f kcal\n",
  result$estimate
))
cat(sprintf(
  "95%% 부트스트랩 신뢰구간: [%.1f, %.1f] kcal\n",
  result$ci_lower,
  result$ci_upper
))
```

```
에너지 섭취량 중앙값 추정치: 1845.4 kcal
95% 부트스트랩 신뢰구간: [1616.6, 2042.9] kcal
```

- **점 추정값(estimate):** 표본에서 계산된 중앙값은 1845.4kcal 이다.
- **부트스트랩 퍼센타일 신뢰구간(ci_lower, ci_upper):** 부트스트랩 분포의 하위 2.5%와 상위 97.5% 지점은 (1616.6, 2042.9)kcal 이다.

20세 이하 남성의 에너지 섭취량 중앙값은 약 1845.4kcal로 추정된다. 부트스트랩 퍼센타일 방법으로 계산한 95% 신뢰구간은 1616.6–2042.9kcal이다.

이처럼 부트스트랩을 사용하면 복잡한 수식 없이도 어떤 통계량(중앙값, 최빈값, 분산 등)에 대해서든 신뢰구간을 유연하게 구해낼 수 있다. 이것이 현대 통계학에서 부트스트랩이 강력한 도구로 각광받는 이유다.

B.4.7 모평균에 대한 검정

이번 실습에서는 국민건강영양조사(KNHANES) 2019년 데이터를 사용하여 두 가지 질문에 답할 것이다.

- **일표본 t-검정:** 저소득층 여성의 평균 에너지 섭취량은 권장량인 1,600kcal 보다 적은가?
- **이표본 t-검정:** 저소득층과 고소득층 여성 간에 평균 에너지 섭취량의 차이가 존재하는가?

데이터 준비: 분석 대상 정의하기

먼저 분석에 필요한 데이터를 추출한다. 위의 예제에서는 표본 수 $n = 100$ 인 표본에 대한 예를 들었지만, 이 실습에서는 KNHANES 데이터의 모든 데이터를 전부 사용한다. 성별이 여성(`sex == 2`)이고 만 20세 이상인 성인(`age >= 20`) 중, 소득 수준이 하위 20%인 1분위(저소득층)와 상위 20%인 5분위(고소득층) 데이터를 선별한다.

[R 코드 4-20]

```
library(dplyr)

# 1. 데이터 필터링 및 그룹 변수 생성
# ho_incm5 가 1(저소득) 또는 5(고소득)인 데이터만 추출
hn19_filtered <- hn19 %>%
  filter(ho_incm5 %in% c(1, 5),
         !is.na(N_EN),
         sex == 2,
         age >= 20) %>%
  mutate(
    # 숫자로 된 소득 분위 데이터를 '저소득층', '고소득층'이라는 이름표(label)를 가진 요인(factor)으로 변환
    income_group = factor(ho_incm5,
                          levels = c(1, 5),
                          labels = c("저소득층", "고소득층"))
  )
```

```
# 2. 그룹별 요약 통계 계산
summary_stats <- hn19_filtered %>%
  group_by(income_group) %>%
  summarise(
    평균 = mean(N_EN),
    표준편차 = sd(N_EN),
    n = n()
  )

# 결과 출력
print("그룹별 칼로리 섭취량 요약:")
print(summary_stats)
```

그룹별 칼로리 섭취량 요약:

	income_group	평균	표준편차	n
1	저소득층	1308.	528.	541
1	고소득층	1686.	684.	724

이 코드를 실행하면 표본 수가 높은 데이터에 대한 집단별 통계량을 보여준다. 저소득층의 평균 섭취량은 약 1,308kcal, 고소득층은 약 1,686kcal로 관측된다. 또한 표준편차를 보면 저소득층(약 528)보다 고소득층(약 684)이 훨씬 크게 나타나, 두 집단의 퍼짐 정도가 다를 수 있다.

일표본 t-검정: 기준값과의 비교 `t.test()`

첫 번째 질문은 "저소득층 여성의 섭취량이 1,600kcal보다 유의하게 적은가?"이다. 우리가 입증하고 싶은 것은 "적다"이므로, `alternative="less"` 옵션을 사용하여 단측 검정을 수행한다.

[R 코드 4-21]

```
# 1. 저소득층 데이터 추출 (Vector 추출)
low_income_data <- hn19_filtered %>%
  filter(income_group == "저소득층") %>%
  pull(N_EN)
```

```
# 2. 귀무가설의 모평균 값 설정
mu_0 <- 1600

# 3. 일표본 t-검정 수행
# alternative = "less": 대립가설이 '평균 < 1600'임을 의미합니다.
t_result_one <- t.test(low_income_data, mu = mu_0, alternative = "less")

# 4. 결과 출력
cat(sprintf("저소득층 표본 크기: %d명\n", length(low_income_data)))
cat(sprintf("저소득층 평균 칼로리 섭취량: %.2f kcal\n", mean(low_income_data)))
cat(rep("-", 30), "\n")

# t.test 결과 객체에서 필요한 값만 추출
cat(sprintf("일표본 t-통계량: %.3f\n", t_result_one$statistic))
cat(sprintf("일표본 P-value (단측, less): %.3f\n", t_result_one$p.value))
```

```
저소득층 표본 크기: 541명
저소득층 평균 칼로리 섭취량: 1307.91 kcal
-----
일표본 t-통계량: -12.856
일표본 P-value (단측, less): 0.000
```

결과 해석

분석 결과 t-통계량은 약 -12.856으로 계산된다. 이는 관측된 평균(1,307.91kcal)이 기준값 1,600kcal 보다 표준오차의 약 12.9배만큼 낮다는 뜻이다. 단측 P-값은 0에 매우 가까우므로 귀무가설을 기각하고, 저소득층 성인 여성의 평균 에너지 섭취량이 1,600kcal보다 낮다고 결론 내린다.

이표본 t-검정: 두 집단 간의 비교 `t.test()`

두 번째 질문은 "저소득층과 고소득층 간에 섭취량 차이가 있는가?"이다. 앞서 기초 통계량에서 확인했듯, 두 집단의 표준편차(분산)가 꽤 다르다. 따라서 등분산을 가정하는 일반적인 t-검정 대신, 분산이 달라도 안전하게 사용할 수 있는 **웰치의 t-검정(Welch's t-test)**을 수행해야 한다. 파이썬에서는 `equal_var = FALSE` 옵션으로 이를 간단히 적용할 수 있다.

[R 코드 4-22]

```
# 1. 고소득층 데이터 추출 (Vector 추출)
high_income_data <- hn19_filtered %>%
  filter(income_group == "고소득층") %>%
  pull(N_EN)

cat(sprintf("저소득층 표본 크기: %d명\n", length(low_income_data)))
cat(sprintf("고소득층 표본 크기: %d명\n", length(high_income_data)))
cat(rep("-", 30), "\n")

# 2. 이표본 t-검정 (Welch's t-test)
# R의 t.test()는 var.equal = FALSE가 기본값입니다.
# 즉, 등분산 가정을 하지 않는 더 안전한 검정을 기본으로 수행합니다.
t_result_two <- t.test(low_income_data, high_income_data, var.equal = FALSE)

# 결과 출력
cat(sprintf("Welch's t-statistic: %.3f\n", t_result_two$statistic))
cat(sprintf("P-value: %.3f\n", t_result_two$p.value))
```

```
저소득층 표본 크기: 541명
고소득층 표본 크기: 724명
-----
Welch's t-statistic: -11.077
P-value: 0.000
```

결과 해석

t-통계량은 약 -11.077, P-값은 0에 가까워 유의수준 0.05보다 훨씬 작다. 따라서 "소득 수준에 따라 성인 여성의 평균 에너지 섭취량에는 통계적으로 유의한 차이가 있다"는 결론을 내릴 수 있다.

B.4.8 분산분석

이번 실습에서는 소득 수준(1~5분위)에 따른 대한민국 성인 여성의 에너지 섭취량의 차이를 분산분석(ANOVA)을 이용하여 검정한다.

데이터 준비 및 탐색

먼저 국민건강영양조사 데이터에서 성인 여성을 대상으로 소득 분위별(1~5분위)로 각각 100명씩을 무작위 추출하여 분석 데이터를 생성한다.

[R 코드 4-23]

```
library(dplyr)

# 앞의 t 검정과 동일하게 성인을 만 20세 이상으로 정의한다.
data_anova <- hn19 %>%
  filter(
    ho_incm5 %in% 1:5,
    !is.na(N_EN),
    sex == 2,
    age >= 20
  )

set.seed(123)
sampled_data <- data_anova %>%
  group_by(ho_incm5) %>%
  slice_sample(n = 100) %>%
  ungroup()

summary_stats <- sampled_data %>%
  group_by(ho_incm5) %>%
  summarise(mean = mean(N_EN), std = sd(N_EN), count = n())
print(summary_stats)
```

	mean	std	count
ho_incm5			
1.0	1226.856822	476.428809	100
2.0	1505.740736	509.251808	100
3.0	1599.797384	618.530528	100
4.0	1638.810116	629.228414	100
5.0	1707.945572	720.338576	100

결과 해석

기초 통계량을 살펴보면 소득 1분위의 평균 섭취량은 약 1,226kcal인 반면, 5분위는 약 1,707kcal로 꽤 큰 차이를 보인다. 하지만 표준편차를 보면 그룹 내에서도 개인차가 상당히 큼을 알 수 있다. 과연 이 평균의 차이가 통계적으로 유의미한지 검정해 보자.

분산분석(ANOVA) 수행

R에서는 `aov()`로 모형을 적합하고 `summary()`로 분산분석표를 확인한다. 핵심 수식 `N_EN ~ factor(ho_incm5)`는 에너지 섭취량을 소득 분위라는 범주형 변수로 설명한다는 뜻이다.

[R 코드 4-24]

```
# 4. ANOVA 모델 적합
# N_EN ~ factor(ho_incm5):
# 에너지 섭취량을 소득분위(숫자가 아닌 '그룹'으로 인식)로 설명하겠다는 수식입니다.
anova_model <- aov(N_EN ~ factor(ho_incm5), data = sampled_data)

# 5. 분산분석표(ANOVA Table) 출력
# R의 summary()는 F-통계량, 자유도, P-값을 일목요연하게 보여줍니다.
summary_table <- summary(anova_model)

print(summary_table)
```

```
              Df    Sum Sq Mean Sq F value    Pr(>F)
factor(ho_incm5)  4 10506026 2626506   7.836 3.96e-06 ***
Residuals       495 165915806  335183
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

분산분석표에서 가장 중요한 두 가지 숫자는 F-값 7.86과 P-값($PR(>F)$) 3.96e-06 (3.96×10^{-6})이다. P-값이 0.05보다 훨씬 작으므로, 우리는 귀무가설을 기각하고, **"적어도 하나의 소득 그룹은 다른 그룹들과 평균 섭취량이 다르다"**고 결론 내린다.

분산분석은 적어도 한 집단의 평균이 다르다는 사실만 알려 주며, 어떤 집단 쌍이 다른지는 알려 주지 않는다. 이를 확인하는 대표적 사후 검정이 Tukey HSD이다. Tukey HSD는 모든 쌍 비교에서 가족단위 제1종 오류율(FWER)을 통제하는 다중비교 방법이다.

튜키의 HSD 검정을 소득수준별 평균 에너지 섭취량 데이터에 적용한 결과를 [그림 B-4-6]에 표시하였다. 이 그래프는 모든 가능한 집단 쌍의 평균 차이에 대한 95% 신뢰구간을 보여준다. 그래프 Y축의 "4-1"은 4분위 집단과 1분위 집단의 모평균 차이를 뜻한다.

- **1분위 vs 5분위 (5-1):** 신뢰구간이 0을 포함하지 않고 양수 쪽에 있다. 즉, 5분위가 1분위보다 확실히 칼로리 섭취량이 많다.

- **4 분위 vs 5 분위 (5-4):** 신뢰구간이 0 을 포함하고 있다. 즉, 통계적으로 두 집단 간에는 차이가 있다고 말하기 어렵다.

이처럼 분산분석으로 전체적인 차이의 유무를 먼저 판별(숲을 봄)한 뒤, 사후 검정을 통해 구체적인 차이의 관계(나무를 봄)를 파악하는 것이 올바른 순서다.

[그림 B-4-6] 튜키의 HSD 검정에 의한 모든 집단 쌍별 신뢰구간

튜키의 HSD 검정

ANOVA는 "차이가 있다"는 사실만 알려줄 뿐, 구체적으로 "1분위와 5분위가 다른지", "3분위와 4분위가 다른지"는 알려주지 않는다. 이를 확인하기 위해 **Tukey의 HSD(Honest Significant Difference)** 검정을 수행한다.

[R 코드 4-25]

```
# 6. 사후 검정 (Tukey's HSD Test)
# 앞서 만든 anova_model(aov 객체)을 그대로 사용합니다.
tukey_result <- TukeyHSD(anova_model)

# 결과 출력
# diff(평균 차이), lwr/upr(신뢰구간), p adj(보정된 P-값)를 보여줍니다.
print(tukey_result)

# (참고) 시각적으로 확인하기
# R에서는 사후 검정 결과를 plot() 하나로 바로 시각화할 수 있습니다.
plot(tukey_result)
```

Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = N_EN ~ factor(ho_incm5), data = sampled_data)

```
$`factor(ho_incm5)`
      diff      lwr      upr    p adj
2-1 238.646535 14.48171 462.8114 0.0304007
3-1 312.984174 88.81935 537.1490 0.0013962
```

```

4-1 318.936310 94.77149 543.1011 0.0010525
5-1 435.300778 211.13595 659.4656 0.0000016
3-2 74.337639 -149.82718 298.5025 0.8938317
4-2 80.289775 -143.87505 304.4546 0.8639662
5-2 196.654243 -27.51058 420.8191 0.1164785
4-3 5.952136 -218.21269 230.1170 0.9999937
5-3 122.316603 -101.84822 346.4814 0.5669164
5-4 116.364468 -107.80036 340.5293 0.6142300

```

결과 해석

출력된 표에서 p adj 컬럼이 유의수준 0.05보다 작은 쌍을 찾으면 된다.

- **1분위 vs 5분위:** p adj = 0.0000016. 두 그룹 간에는 확실한 차이가 있다.
- **3분위 vs 4분위:** p adj = 0.9999937. 두 그룹 간의 차이는 통계적으로 유의하지 않다.

이처럼 분산분석으로 전체 차이 유무를 먼저 판단한 후, 사후 검정으로 개별 그룹 간 차이를 확인하는 것이 정석적인 분석 절차다.

B.4.9 카이제곱 독립성 검정

앞서 우리는 범주형 데이터의 관계를 파악하기 위해 카이제곱 검정을 배운 바 있다. 이번 실습에서는 국민건강영양조사 데이터를 이용하여 **"20대 여성의 소득 수준과 주관적 건강 상태 인식 사이에 관련이 있는가?"**라는 질문에 답해보고자 한다.

데이터 준비 및 교차표 생성

먼저 분석 대상을 명확히 한다. 전체 데이터에서 '만 20세 이상 29세 이하의 성인 여성'만을 추출하고, 결측값이 있는 데이터는 제거한다. 분석할 변수는 소득 분위(ho_incm5)와 주관적 건강 상태(D_1_1)다.

[R 코드 4-26]

```

library(dplyr)
library(tidyr)

health_income_data <- hn19 %>%
  filter(
    sex == 2,

```

```

age >= 20, age <= 29,
  D_1_1 != 9
) %>%
dplyr::select(D_1_1, ho_incm5) %>%
drop_na()

health_income_data$D_1_1 <- as.factor(health_income_data$D_1_1)
health_income_data$ho_incm5 <- as.factor(health_income_data$ho_incm5)

cat(sprintf("분석 대상 데이터 크기: %d명\n", nrow(health_income_data)))

contingency_table <- table(
  health_income_data$D_1_1,
  health_income_data$ho_incm5
)
print("건강 상태(D_1_1)와 소득 분위(ho_incm5)의 교차표:")
print(contingency_table)

```

분석 대상 데이터 크기: 314명

건강 상태(D_1_1)와 소득 분위(ho_incm5)의 교차표:

ho_incm5	1.0	2.0	3.0	4.0	5.0
D_1_1					
1.0	4	1	1	2	7
2.0	2	3	19	24	40
3.0	11	15	40	38	53
4.0	5	8	13	9	11
5.0	0	2	1	1	4

분할표를 살펴보면, 특정 셀(Cell)의 빈도가 매우 적은 것을 확인할 수 있다. 예를 들어 건강이 '매우 좋음(1)'이면서 소득 2분위인 표본은 단 1명이다.

카이제곱 검정 수행

이제 `chisq.test()` 함수를 이용해 카이제곱 검정을 수행한다. 이 때의 귀무가설과 대립가설은 다음과 같다.

- 귀무가설 H_0 : 소득 분위와 주관적 건강 상태는 서로 독립이다(관련이 없다).
- 대립가설 H_1 : 소득 분위와 주관적 건강 상태는 서로 독립이 아니다(관련이 있다).

[R 코드 4-27]

```
# 3. 카이제곱 독립성 검정 수행
# chisq.test() 함수에 앞서 만든 분할표를 넣습니다.
chi2_result <- chisq.test(contingency_table)

# 결과 출력
cat(sprintf("카이제곱 통계량: %.3f\n", chi2_result$statistic))
cat(sprintf("P-value: %.3f\n", chi2_result$p.value))
cat(sprintf("자유도 (Degrees of Freedom): %d\n", chi2_result$parameter))

# 4. 기대 빈도표(Expected Frequencies) 확인
# 만약 두 변수가 완전히 독립이라면 나타났어야 할 '이론적 빈도'입니다.
print("기대 빈도표:")
print(chi2_result$expected)
```

카이제곱 통계량: 31.501

P-value: 0.012

자유도 (Degrees of Freedom): 16

[1] "기대 빈도표:"

	1	2	3	4	5
1	1.0509554	1.3853503	3.535032	3.535032	5.493631
2	6.1656051	8.1273885	20.738854	20.738854	32.229299
3	11.0000000	14.5000000	37.000000	37.000000	57.500000
4	3.2229299	4.2484076	10.840764	10.840764	16.847134
5	0.5605096	0.7388535	1.885350	1.885350	2.929936

결과 해석의 주의점

여기서 계산된 P-값을 그대로 믿어도 될까? 카이제곱 검정은 표본의 크기가 충분히 클 때(보통 기대빈도가 5 이상일 때) 검정통계량이 카이제곱 분포를 따른다는 이론에 근거한다. 하지만 앞서 확인했듯이 우리 데이터에는 **빈도가 1, 2 (심지어 0)과 같이 매우 작은 셀들이 존재한다**. 이런 경우 이론적인 카이제곱 분포 근사가 부정확할 수 있어, 계산된 P-값이 왜곡될 위험이 있다.

이때의 해결책 중 하나는 의미가 유사한 두 개 이상의 범주를 합치는 재범주화이다. 건강 상태 1-2는 ' 좋음', 3은 '보통', 4-5는 '나쁨'으로 합치면 원래 분할표보다 기대빈도 조건이 크게 개선된다. 다만 일부 셀의 기대빈도는 여전히 5보다 작으므로, 카이제곱 근사의 조건이 개선되었다는 수준으로 해석하고 결과를 주의해서 본다.

[R 코드 4-28]

```
library(dplyr)

# 1. D_1_1(건강상태)의 범주 재정의
# 1,2 -> ' 좋음', 3 -> '보통', 4,5 -> '나쁨'
health_income_data <- health_income_data %>%
  mutate(health_status_recoded = case_when(
    D_1_1 %in% c(1, 2) ~ " 좋음",
    D_1_1 == 3 ~ "보통",
    D_1_1 %in% c(4, 5) ~ "나쁨",
    TRUE ~ NA_character_ # 정의되지 않은 값 처리
  )) %>%
  # 분석의 편의를 위해 범주형(Factor)으로 변환하고 순서(Level)를 지정합니다.
  mutate(
    health_status_recoded = factor(health_status_recoded, levels = c(" 좋음", "보통", "나쁨")),
    ho_incm5 = as.factor(ho_incm5)
  )

cat(sprintf("분석 대상 데이터 크기: %d명\n", nrow(health_income_data)))

# 2. 재범주화된 분할표 (Contingency Table) 생성
contingency_table_3 <- table(health_income_data$health_status_recoded,
```

```
health_income_data$ho_incm5)
```

```
print("재범주화된 건강 상태와 소득 분위의 교차표:")
```

```
print(contingency_table_3)
```

분석 대상 데이터 크기: 314명

재범주화된 건강 상태와 소득 분위의 교차표:

ho_incm5	1.0	2.0	3.0	4.0	5.0
health_status_recoded					
나쁨	5	10	14	10	15
보통	11	15	40	38	53
좋음	6	4	20	26	47

재범주화된 데이터를 이용하여 독립성 검정을 다시 시행한다.

[R 코드 4-29]

```
chi2_result_3 <- chisq.test(contingency_table_3)
```

```
cat(sprintf("P-value: %.3f\n", chi2_result_3$p.value))
```

```
cat(sprintf(
  "카이제곱 통계량: %.3f, 자유도: %d\n",
  chi2_result_3$statistic,
  chi2_result_3$parameter
))
```

P-value: 0.070

카이제곱 통계량: 14.491, 자유도: 8

카이제곱 검정의 결과 P-값은 0.07로서 유의수준 0.05보다 크다! 귀무가설 (두 변수의 독립성)을 기각할 증거가 충분하지 않기 때문에, 우리는 20대 여성 집단에서 소득 수준과 주관적 건강 인지 사이에 통계적으로 유의미한 관련성을 발견하지 못한 것이다.

B.5 5 부 실습

```
knitr::opts_chunk$set(echo = TRUE)
```

B.5.1 변수선택

이 장에서는 서울시 법정동 단위 아파트 매매가격 데이터를 이용하여 본문의 5부에서 다룬 변수선택 방법을 R로 직접 실습해 본다. R은 변수선택에 특화된 패키지가 잘 갖춰져 있어 최적 부분집합 탐색부터 단계적 선택법까지 간결하게 수행할 수 있다.

탐색적 자료분석

본격적인 분석에 앞서 데이터의 전반적인 구조와 변수 간의 관계를 파악하는 탐색적 자료분석(EDA)을 진행한다. 먼저 분석에 필요한 패키지를 불러온다.

[R 코드 5-1]

```
# 패키지 로딩
library(leaps) # 최적 부분집합 선택법: regsubsets()
library(ggplot2) # 시각화
library(dplyr) # 데이터 전처리
```

본 실습에서 사용하는 핵심 패키지를 간략히 소개한다. leaps 패키지의 regsubsets() 함수는 변수선택의 핵심 도구로, 최적 부분집합 선택법은 물론 전진·후진 선택법까지 method 요인 하나로 모두 수행할 수 있다. ggplot2는 시각화를, dplyr은 데이터 전처리에 활용한다.

다음으로 데이터를 불러오고 분석에 필요한 전처리를 수행한다.

[R 코드 5-2]

```
# 데이터 로드
apt <- read.csv("데이터/seoul_apt_dong_model.csv")

# 식별 변수 제거와 비율 단위 통일을 마친 분석용 데이터
sum(is.na(apt))
head(apt)
```

데이터는 서울시 법정동 단위로 수집된 **268개의 관측치**와 **8개의 설명변수**로 이루어져 있다. 결측값은 존재하지 않는다. 주요 변수를 소개하면 다음과 같다.

변수명	설명
-----	----

y	아파트 매매 평균가격(m²당, 원), 로그 변환값
avg_apt_age	평균 아파트 연령(2023년 기준)
avg_TF_large	대단지 아파트 비율
cnt	아파트 수
rate_big_area_m2	대형 평수(85m² 이상) 비율
pop_50_69	50~69세 인구
rate_buy	매매 거래 비율
rate_welfare_recipient	기초생활수급자 비율
avg_D_Sub	인근 지하철역까지의 평균 거리(m)

이제 변수 간의 선형적 연관성을 파악하기 위해 상관계수 행렬을 계산하고 시각화해 보자.

[R 코드 5-3]

```
# 상관계수 행렬 계산 및 시각화
library(ggcorrplot)
cor_mat <- cor(apt, use = "complete.obs")
ggcorrplot(cor_mat, lab = TRUE, lab_size = 3)
```

[그림 B-5-1] 상관계수 행렬 및 히트맵 시각화

[그림 B-5-1]의 히트맵을 통해 반응변수 y(로그 아파트 가격)와 각 설명변수의 상관관계를 한눈에 파악할 수 있다. rate_big_area_m2(대형 평수 비율)이나 avg_TF_large(대단지 비율)는 가격과 양의 상관관계를, rate_welfare_recipient(수급자 비율)는 음의 상관관계를 나타낼 것으로 예상된다.

변수선택 실습

최적 부분집합 선택법

최적 부분집합 선택법(Best Subset Selection)은 가능한 모든 변수 조합을 탐색하여 각 변수 개수별로 가장 성능이 좋은 모형을 찾는 방법이다. R에서는 leaps 패키지의 regsubsets() 함수를 사용하면

이 과정을 매우 간결하게 수행할 수 있다.

[R 코드 5-4]

```
# 최적 부분집합 선택법 수행
regfit.full <- regsubsets(y ~ ., data = apt, nvmax = 8)
reg.summary <- summary(regfit.full)

# 각 변수 개수별 선택된 변수 확인
reg.summary$outmat
```

##	avg_apt_age	avg_TF_large	cnt	rate_big_area_m2	pop_50_69	rate_buy	rate_welfare_recipient
avg_D_Sub							
## 1 (1)	" "	" "	" " " "	" "	"*"	" "	" "
## 2 (1)	" "	" "	" " " "	" "	"*"	"*"	" "
## 3 (1)	" "	" "	" " " "	" "	"*"	"*"	"*"
## 4 (1)	"*"	" "	" " " "	" "	"*"	"*"	"*"
## 5 (1)	"*"	" "	" " "*"	" "	"*"	"*"	"*"
## 6 (1)	"*"	"*"	" " "*"	" "	"*"	"*"	"*"
## 7 (1)	"*"	"*"	" " "*"	"*"	"*"	"*"	"*"
## 8 (1)	"*"	"*"	"* " "*"	"*"	"*"	"*"	"*"

nvmax = 8은 최대 8개 변수까지의 모형을 탐색하겠다는 의미이다. 출력 결과에서 * 표시는 해당 모형에 그 변수가 포함되어 있음을 나타낸다. 예를 들어, 변수 2개 모형에서는 rate_buy와 rate_welfare_recipient만 선택된다.

BIC 를 통한 변수 개수 선택

변수 개수에 따른 BIC 값을 시각화하여 최적의 변수 개수를 결정한다. BIC는 변수가 늘어날수록 강한 패널티를 부여하므로, 값이 **작을수록** 더 좋은 모형이다.

[R 코드 5-5]

```
# BIC 시각화
best_k <- which.min(reg.summary$bic)

plot(reg.summary$bic, xlab = "변수 개수", ylab = "BIC", type = "l")
points(best_k, reg.summary$bic[best_k], col = "red", cex = 2, pch = 20)

cat("BIC 기준 최적 변수 개수:", best_k, "개\n")
```

```
## BIC 기준 최적 변수 개수: 7 개
```

[그림 B-5-2] BIC 기준 최적 변수 개수 선택

[그림 B-5-2]에서 BIC가 가장 낮은 지점이 빨간 점으로 표시된다. BIC 기준으로는 7개 변수를 포함한 모형이 최적으로 나타난다.

교차검증을 이용한 변수 개수 선택

앞에서는 훈련 데이터 안에서 계산된 BIC로 변수 개수를 선택했다면, 이번에는 모형이 실제로 **보지 못한 데이터**에서 얼마나 잘 예측하는지를 직접 측정하는 교차검증을 수행한다. 여기서는 전체 데이터를 10개 조각으로 나누어 검증하는 **10-겹 교차검증(10-fold CV)**을 사용한다.

regsubsets() 객체는 기본적으로 예측 함수를 제공하지 않으므로, 아래와 같이 예측 함수를 직접 정의해 사용한다.

[R 코드 5-6]

```
# 예측 함수 정의
predict.regsubsets <- function(object, newdata, id) {
  mat <- model.matrix(y ~ ., newdata)
  coefi <- coef(object, id = id)
  mat[, names(coefi), drop = FALSE] %*% coefi
}

# 10-겹 교차검증 수행
set.seed(1)
k <- 10
n <- nrow(ap7)
d <- ncol(ap7) - 1
folds <- sample(rep(seq_len(k), length.out = n))
cv.errors <- matrix(NA_real_, nrow = k, ncol = d)

for (j in seq_len(k)) {
  fit <- regsubsets(y ~ ., data = ap7[folds != j, ], nvmax = d)
  for (i in seq_len(d)) {
    pred <- predict.regsubsets(fit, ap7[folds == j, ], id = i)
```

```

    cv.errors[j, i] <- mean((apt$y[folds == j] - pred)^2)
  }
}

mean.cv.errors <- colMeans(cv.errors)
optimal_k <- which.min(mean.cv.errors)

```

cv.errors는 10행 8열의 행렬로, (j, i) 번째 값은 j 번째 폴드를 테스트로 두고 i 개 변수 모델을 평가한 MSE이다. 모든 반복이 끝나면 열 방향으로 평균을 내어 변수 개수별 평균 테스트 오차를 구하고, 이 값이 가장 작은 변수 개수를 최적으로 선택한다.

[R 코드 5-7]

```

# 교차검증 결과 시각화
plot(mean.cv.errors, type = "b",
      xlab = "변수 개수", ylab = "평균 테스트 오차 (MSE)")
points(optimal_k, mean.cv.errors[optimal_k], col = "red", cex = 2, pch = 20)

cat("교차검증 기준 최적 변수 개수:", optimal_k, "개\n")

## 교차검증 기준 최적 변수 개수: 8 개

```

[그림 B-5-3] 교차검증 결과 시각화

[그림 B-5-3]에서 평균 테스트 오차가 가장 낮은 지점이 빨간 점으로 표시된다. 교차검증 기준으로는 8개 변수 모형이 선택된다.

최종 모형 적합과 해석

BIC와 교차검증 결과를 종합하면 7~8개 변수를 사용하는 모형이 적절하다. 여기서는 교차검증 기준에 따라 **8개 변수 모형**을 최종 선택하고, 전체 데이터로 모형을 적합하여 회귀계수를 확인한다.

[R 코드 5-8]

```

# 최종 모형 계수 확인
reg.best <- regsubsets(y ~ ., data = apt, nvmax = d)
coef_df <- data.frame(계수 = round(coef(reg.best, optimal_k), 4))
print(coef_df)

```

```
##              계수
## (Intercept)  16.8813
## avg_apr_age  -0.0058
## avg_TF_large  0.2993
## cnt          0.0046
## rate_big_area_m2  0.4022
## pop_50_69     0.0000
## rate_buy      -1.9637
## rate_welfare_recipient -2.7120
## avg_D_Sub     -0.0001
```

이 모형의 반응변수 y 는 아파트 가격에 로그를 취한 값이므로, 회귀계수를 해석할 때는 지수함수($\exp$)를 이용해 **배수(비율)** 개념으로 읽는 것이 정확하다. 다른 변수들이 모두 고정되어 있을 때, 평균 아파트 나이가 1년 증가하면 가격은 $\exp(-0.006)$ 배로 미세하게 감소한다. 반면 대단지 비율이나 대형 평수 비율이 높을수록 가격은 상승하고, 기초생활수급자 비율이 높을수록 가격은 현저히 감소하는 경향이 있다.

데이터 분석가의 한마디

최적 부분집합 선택법은 모든 조합을 탐색하므로 이론적으로 가장 완벽한 방법이지만, 변수가 많아질수록 계산 시간이 기하급수적으로 늘어난다는 단점이 있다. 변수가 p 개라면 탐색해야 할 조합의 수는 2^p 개에 달한다. 예를 들어 변수가 20개만 되어도 100만 개 이상의 모형을 비교해야 한다. R의 `regsubsets()`는 내부적으로 효율적인 알고리즘을 사용하여 이 과정을 빠르게 처리하지만, 변수가 매우 많은 경우에는 아래에서 소개할 단계적 선택법이 현실적인 대안이 된다.

단계적 선택법: 전진선택법과 후진선택법

완전탐색 방식의 연산 부담을 줄이기 위해 실무에서는 **단계적 선택법(Stepwise Selection)**을 널리 활용한다. R에서는 `regsubsets()`의 `method` 요인만 바꾸면 동일한 방식으로 전진·후진 선택법을 수행할 수 있다.

전진선택법(Forward Selection)은 아무 변수도 없는 상태에서 시작하여, 모형 성능을 가장 크게 높이는 변수를 한 번에 하나씩 추가한다. 반대로 **후진선택법(Backward Selection)**은 모든 변수가 포함된 상태에서 시작하여, 기여도가 가장 낮은 변수를 하나씩 제거한다.

[R 코드 5-9]

```
# 전진선택법
regfit.fwd <- regsubsets(y ~ ., data = apt, nvmax = 8, method = "forward")
```

```
# 후진선택법
regfit.bwd <- regsubsets(y ~ ., data = apt, nvmax = 8, method = "backward")

# 각 방법의 BIC 기준 최적 변수 개수 비교
best_fwd <- which.min(summary(regfit.fwd)$bic)
best_bwd <- which.min(summary(regfit.bwd)$bic)

cat("전진선택법 BIC 최적 변수 개수:", best_fwd, "개\n")
cat("후진선택법 BIC 최적 변수 개수:", best_bwd, "개\n")

## 전진선택법 BIC 최적 변수 개수: 7 개
## 후진선택법 BIC 최적 변수 개수: 7 개
```

두 방법의 결과를 비교하면 어떤 변수가 일관되게 선택되는지 확인할 수 있다. 두 방법이 같은 변수를 선택한다면 해당 변수들은 모형에 포함할 근거가 더욱 확실하다고 볼 수 있다. 선택된 변수 목록은 아래 코드로 확인한다.

[R 코드 5-10]

```
# 전진·후진 선택법 선택 변수 비교
fwd_vars <- names(coef(regfit.fwd, best_fwd))[-1] # 절편 제외
bwd_vars <- names(coef(regfit.bwd, best_bwd))[-1]

cat("전진선택법 선택 변수:\n", paste(fwd_vars, collapse = ", "), "\n\n")
cat("후진선택법 선택 변수:\n", paste(bwd_vars, collapse = ", "), "\n\n")

## 전진선택법 선택 변수:
## avg_apt_age, avg_TF_large, rate_big_area_m2, pop_50_69, rate_buy, rate_welfare_recipient, avg_D_Sub
##
## 후진선택법 선택 변수:
## avg_apt_age, avg_TF_large, rate_big_area_m2, pop_50_69, rate_buy, rate_welfare_recipient, avg_D_Sub
```

데이터 분석가의 한마디

단순 전진선택법은 현재 선택 집합에 변수를 하나씩 추가하는 탐욕적 절차이며, 한 번 추가한 변수는 이후 단계에서 제거하지 않는다. 따라서 전체 가능한 변수 조합 중 전역 최적 모형을 보장하지 않는다. BIC·교차검증 결과와 도메인 지식을 함께 활용해 최종 모형을 결정하는 것이 바람직하

다.

B.5.2 정규화

이 장에서는 서울시 법정동 단위 아파트 매매가격 데이터를 이용하여 본문에서 다룬 **정규화 (Regularization)** 방법을 R로 직접 실습해 본다. R에서는 glmnet 패키지 하나로 라쏘(LASSO), 릿지(Ridge), Elastic Net을 모두 수행할 수 있어 매우 편리하다.

사전 준비

먼저 필요한 패키지를 불러오고, 데이터를 glmnet() 함수에 맞는 형태로 준비한다.

[R 코드 5-11]

```
# 패키지 로딩
library(glmnet) # LASSO, Ridge, Elastic Net 적합
```

glmnet 패키지의 glmnet() 함수는 alpha 요인 하나로 세 가지 방법을 모두 수행한다. alpha=1이면 라쏘, alpha=0이면 릿지, 그 사이 값이면 Elastic Net이 된다. 또한 glmnet()은 내부적으로 변수를 자동 표준화하므로 별도의 전처리가 필요 없다.

glmnet() 함수는 일반적인 lm()과 달리 수식(formula) 형태가 아닌, 설명변수 행렬 x와 반응변수 벡터 y를 직접 입력해야 한다. model.matrix()를 이용하면 데이터프레임을 행렬 형태로 손쉽게 변환할 수 있다.

[R 코드 5-12]

```
# glmnet 입력 형태로 변환
x <- model.matrix(y ~ ., apt)[-1]
y <- apt$y

# 훈련/테스트 데이터 분할
set.seed(123)
train <- sample(seq_len(nrow(x)), floor(0.7 * nrow(x)))
test <- setdiff(seq_len(nrow(x)), train)
x.train <- x[train, , drop = FALSE]
y.train <- y[train]
x.test <- x[test, , drop = FALSE]
y.test <- y[test]
```

```
# 모든 alpha 후보가 같은 교차검증 폴드를 사용하도록 고정
set.seed(123)
foldid <- sample(rep(1:10, length.out = nrow(x.train)))
```

model.matrix()의 첫 번째 열은 절편이므로 [, -1]로 제거한다. 전체 자료의 70%를 훈련 자료로, 나머지 30%를 테스트 자료로 사용하며, 교차검증 폴드는 훈련 자료 안에서만 고정한다.

라쏘

라쏘는 회귀계수의 **절댓값**에 패널티를 부여하여, 일부 변수의 계수를 정확히 0으로 만드는 변수 선택 효과를 갖는 방법이다.

계수 경로 플롯

R에서는 glmnet() 결과에 plot()을 바로 적용하면 규제 강도 λ 가 변함에 따라 각 회귀계수가 어떻게 변화하는지를 보여주는 **계수 경로 플롯(Coefficient Path Plot)**을 손쉽게 그릴 수 있다. 파이썬에서는 이 플롯을 그리기 위해 별도의 복잡한 구현이 필요해 실습에서 생략했지만, R에서는 단 한 줄로 확인할 수 있다.

[R 코드 5-13]

```
# 계수 경로 플롯 (alpha = 1: LASSO)
grid <- 10^seq(2, -5, length = 100)
lasso.path <- glmnet(x.train, y.train, alpha = 1, lambda = grid)
plot(lasso.path)
```

[그림 B-5-4] 라쏘 계수 경로 플롯

[그림 B-5-4]에서 가로축은 패널티의 비율을, 세로축은 각 변수의 회귀계수 값을 나타낸다. 왼쪽(규제가 강한 쪽)으로 갈수록 계수들이 0으로 수축되며, 특정 시점에서 일부 계수가 정확히 0이 되어 탈락하는 모습을 확인할 수 있다. 이를 통해 어떤 변수가 패널티에 강하게 버티며 끝까지 모형에 남는지, 반대로 어떤 변수가 먼저 탈락하는지 한눈에 파악할 수 있다.

교차검증으로 최적 λ 선택

cv.glmnet()을 사용하면 여러 λ 후보 값에 대해 교차검증을 자동으로 수행하고, 평균 예측 오차(MSE)가 가장 낮은 최적의 λ 를 찾아준다.

[R 코드 5-14]

```
# 훈련 자료 내부 10-겹 교차검증으로 최적 lambda 탐색
cv.lasso <- cv.glmnet(
  x.train, y.train, alpha = 1, foldid = foldid
)
plot(cv.lasso)

best_lambda_lasso <- cv.lasso$lambda.min
cat("LASSO 최적 lambda:", round(best_lambda_lasso, 5), "\n")

## LASSO 최적 lambda: 0.00038
```

[그림 B-5-5] 라쏘 교차검증 곡선

[그림 B-5-5]의 세로축은 교차검증 MSE, 가로축은 λ 를 나타낸다. 그래프 상단의 숫자는 해당 값에서 모형에 포함되는 변수의 수를 의미한다. 왼쪽 점선(lambda.min)이 MSE가 가장 낮은 지점으로, 이 값을 최적 λ 로 사용한다.

최종 라쏘 모형 적합과 해석

교차검증으로 선택된 최적 λ 를 사용하여 전체 데이터에 최종 라쏘 모형을 적합하고 회귀계수를 확인한다.

[R 코드 5-15]

```
# 훈련 자료로 적합하고 테스트 MSE 평가
lasso.train <- glmnet(
  x.train, y.train, alpha = 1, lambda = best_lambda_lasso
)
lasso.pred <- as.numeric(predict(lasso.train, newx = x.test))
lasso_test_mse <- mean((y.test - lasso.pred)^2)

# 최종 계수 제시용으로 전체 자료에 다시 적합
lasso.final <- glmnet(x, y, alpha = 1, lambda = best_lambda_lasso)
lasso.coef <- coef(lasso.final)
round(lasso.coef, 3)
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept)  16.881
## avg_apt_age   -0.006
## avg_TF_large   0.296
## cnt           0.004
## rate_big_area_m2 0.399
## pop_50_69      0.000
## rate_buy       -1.965
## rate_welfare_recipient -2.705
## avg_D_Sub      0.000
```

훈련 자료 내부 교차검증에서 최적 λ 는 0.00038로 선택되었다. 이 값을 적용한 최종 라쏘 모형에서는 모든 설명변수의 계수가 0이 아니었지만, 일부 계수는 매우 작게 수축되었다. 음의 계수 중 절댓값은 rate_welfare_recipient가 가장 크며, 테스트 MSE는 0.04748이다.

0이 아닌 계수만 따로 확인하려면 아래 코드를 사용한다.

[R 코드 5-16]

```
# 0이 아닌 계수와 테스트 MSE 출력
print(lasso.coef[lasso.coef != 0])
cat("LASSO test MSE:", round(lasso_test_mse, 5), "\n")

## [1] 1.688061e+01 -5.769342e-03 2.964978e-01 4.473912e-03 3.990849e-01
## [6] -6.250542e-06 -1.965163e+00 -2.705084e+00 -1.371351e-04
## LASSO test MSE: 0.04748
```

데이터 분석가의 한마디

R의 cv.glmnet()은 λ_{min} 외에도 λ_{1se} 라는 값도 함께 제공한다. 이는 MSE가 최솟값에서 표준오차 1배 이내로 유지되는 범위 중 가장 큰 λ 값으로, 더 강한 패널티를 부여해 보다 간결한 모형을 만든다. 예측 정확도보다 모형의 단순함과 해석 가능성을 중시한다면 λ_{1se} 를 사용하는 것도 좋은 선택이다.

릿지

릿지는 회귀계수의 **제곱합**에 패널티를 부여하는 방법이다. 라쏘처럼 계수를 0으로 만들지는 않지만, 모든 변수의 계수를 전반적으로 작게 줄여 모형을 안정화시킨다. glmnet()에서 alpha=0으로 설정하

면 릿지를 수행한다.

계수 경로 플롯

[R 코드 5-17]

```
# 계수 경로 플롯 (alpha = 0: Ridge)
ridge.path <- glmnet(x.train, y.train, alpha = 0, lambda = grid)
plot(ridge.path)
```

[그림 B-5-6] 릿지 계수 경로 플롯

[그림 B-5-6]을 라쏘의 계수 경로 플롯과 비교해 보면 차이가 뚜렷하다. 라쏘에서는 가 커질수록 일부 계수가 정확히 0이 되어 탈락하는 반면, 릿지에서는 모든 계수가 0을 향해 점진적으로 줄어들지만 끝까지 0에 닿지 않는다. 이 그래프 하나로 "릿지는 변수를 제거하지 않고 모두 조금씩 줄인다"는 릿지의 핵심 특성을 직관적으로 확인할 수 있다.

교차검증으로 최적 λ 선택

[R 코드 5-18]

```
# 같은 foldid를 사용한 Ridge 교차검증
cv.ridge <- cv.glmnet(
  x.train, y.train, alpha = 0, foldid = foldid
)
plot(cv.ridge)

best_lambda_ridge <- cv.ridge$lambda.min
cat("Ridge 최적 lambda:", round(best_lambda_ridge, 5), "\n")

## Ridge 최적 lambda: 0.02819
```

[그림 B-5-7] 릿지 교차검증 곡선

최종 릿지 모형 적합과 해석

[R 코드 5-19]

```
# 훈련 자료로 적합하고 테스트 MSE 평가
ridge.train <- glmnet(
  x.train, y.train, alpha = 0, lambda = best_lambda_ridge
)
ridge.pred <- as.numeric(predict(ridge.train, newx = x.test))
ridge_test_mse <- mean((y.test - ridge.pred)^2)

# 최종 계수 제시용으로 전체 자료에 다시 적합
ridge.final <- glmnet(x, y, alpha = 0, lambda = best_lambda_ridge)
ridge.coef <- coef(ridge.final)
round(ridge.coef, 3)

## 9 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept)  16.855
## avg_appt_age   -0.005
## avg_TF_large   0.282
## cnt            0.003
## rate_big_area_m2 0.374
## pop_50_69      0.000
## rate_buy       -1.854
## rate_welfare_recipient -2.625
## avg_D_Sub      0.000
```

라쏘와 달리 릿지의 계수는 작아지기는 하지만 **0이 되지는 않는다**. 모든 변수가 모형에 남아 있되, 각 변수의 영향력이 완만하게 조정된다. 다중공선성 상황에서 추정치가 안정적으로 유지된다는 것이 릿지의 가장 큰 장점이다.

데이터 분석가의 한마디

LASSO와 Ridge 중 어떤 방법이 더 좋은 질문에는 정답이 없다. 중요한 변수가 소수이고 나머지 변수의 영향이 거의 없다고 판단될 때는 LASSO가, 많은 변수가 비슷한 수준으로 영향을 미치거나 변수 간 상관관계가 높을 때는 Ridge가 유리하다. 두 방법의 장점을 함께 취하고 싶다면 아래에서 소개하는 Elastic Net을 고려해 보자.

Elastic Net

Elastic Net은 라쏘(L1 패널티)와 릿지(L2 패널티)를 **동시에 사용**하는 방법이다. glmnet()의 alpha가

두 패널티의 혼합 비율을 결정하는데, 1에 가까울수록 라쏘에, 0에 가까울수록 릿지에 가까워진다. 최적의 α 를 찾기 위해 여러 후보 값에 대해 각각 `cv.glmnet()`을 반복 수행한 뒤, 평균 테스트 오차가 가장 낮은 α 를 선택한다.

최적 α 와 λ 탐색

[R 코드 5-20]

```
# 모든 alpha 후보를 동일한 foldid로 비교
alphas <- seq(0, 1, by = 0.1)
mse_vec <- numeric(length(alphas))

for (i in seq_along(alphas)) {
  cv.fit <- cv.glmnet(
    x.train, y.train, alpha = alphas[i], foldid = foldid
  )
  mse_vec[i] <- min(cv.fit$cvm)
}

best_alpha <- alphas[which.min(mse_vec)]
cat("Elastic Net 최적 alpha:", best_alpha, "\n")

## Elastic Net 최적 alpha: 0
```

각 α 후보에서 교차검증 MSE의 최솟값을 비교하여 가장 낮은 MSE를 주는 α 를 선택한다.

최종 Elastic Net 모형 적합과 해석

선택된 `best_alpha`로 교차검증을 다시 수행하여 최적 λ 를 구하고, 전체 데이터에 최종 모형을 적합한다.

[R 코드 5-21]

```
# 최적 alpha에서 같은 foldid로 lambda 선택
cv.enet <- cv.glmnet(
  x.train, y.train, alpha = best_alpha, foldid = foldid
)
best_lambda_enet <- cv.enet$lambda.min

enet.train <- glmnet(
```

```

x.train, y.train, alpha = best_alpha, lambda = best_lambda_enet
)
enet.pred <- as.numeric(predict(enet.train, newx = x.test))
enet_test_mse <- mean((y.test - enet.pred)^2)

# 최종 계수 제시용으로 전체 자료에 다시 적합
enet.final <- glmnet(x, y, alpha = best_alpha, lambda = best_lambda_enet)
enet.coef <- coef(enet.final)
round(enet.coef, 3)

## 9 x 1 sparse Matrix of class "dgCMatrix"
##                s0
## (Intercept)    16.855
## avg_appt_age   -0.005
## avg_TF_large    0.282
## cnt            0.003
## rate_big_area_m2 0.374
## pop_50_69       0.000
## rate_buy        -1.854
## rate_welfare_recipient -2.625
## avg_D_Sub       0.000

```

동일한 교차검증 폴드에서 후보 alpha를 비교한 결과 best_alpha는 0으로 선택되었다. 따라서 이 자료와 후보 격자에서는 최종 Elastic Net이 릿지와 같은 경계 모형이 되며, 계수와 테스트 MSE도 릿지 결과와 같다.

세 방법의 최종 계수를 한 표로 비교해 보면 방법 간 차이를 가장 명확하게 확인할 수 있다.

[R 코드 5-22]

```

# 세 모형 계수와 테스트 MSE 비교
coef_compare <- data.frame(
  LASSO = round(as.vector(lasso.coef), 4),
  Ridge = round(as.vector(ridge.coef), 4),
  ElasticNet = round(as.vector(enet.coef), 4),
  row.names = rownames(lasso.coef)
)
print(coef_compare)

```

```
test_mse <- c(
  LASSO = lasso_test_mse,
  Ridge = ridge_test_mse,
  ElasticNet = enet_test_mse
)
round(test_mse, 5)
```

```
##           LASSO  Ridge ElasticNet
## (Intercept) 16.8806 16.8548 16.8548
## avg_apr_age -0.0058 -0.0052 -0.0052
## avg_TF_large 0.2965 0.2817 0.2817
## cnt         0.0045 0.0032 0.0032
## rate_big_area_m2 0.3991 0.3737 0.3737
## pop_50_69     0.0000 0.0000 0.0000
## rate_buy      -1.9652 -1.8540 -1.8540
## rate_welfare_recipient -2.7051 -2.6245 -2.6245
## avg_D_Sub     -0.0001 -0.0001 -0.0001
##   LASSO   Ridge ElasticNet
## 0.04748 0.04803 0.04803
```

비교 표에서 라쏘의 테스트 MSE는 0.04748, 릿지는 0.04803이다. 선택된 α 가 0이므로 Elastic Net의 계수와 테스트 MSE(0.04803)는 릿지와 동일하다. 이 실행에서는 세 모형의 예측 오차 차이가 작다.

데이터 분석가의 한마디

세 가지 정규화 방법 중 어떤 것을 선택할지는 데이터의 특성과 분석 목적에 따라 달라진다. 단순히 예측 정확도만 비교하면 방법 간 차이가 크지 않을 수 있지만, 어떤 변수가 최종 모형에 남아 있는지는 방법마다 크게 다를 수 있다. 파이썬 실습과 R 실습의 결과를 나란히 비교해 보면, 두 언어가 동일한 데이터에 대해 얼마나 유사한 변수를 선택하는지 확인하는 것도 좋은 검증 방법이다.

B.5.3 차원 축소: 주성분분석

주성분분석(PCA)

주성분분석(Principal Component Analysis, PCA)은 여러 개의 변수를 서로 상관되지 않는 새로운 축, 즉 **주성분(Principal Components)**으로 변환하는 차원 축소 기법이다. 원래 변수들이 가진 전체 변동을 최대한 보존하면서도 더 적은 수의 축으로 데이터를 요약할 수 있어, 변수가 많고 서로 상관관

계가 높은 데이터를 단순화하고 시각화하는 데 매우 유용하다. R에서는 기본 함수인 `prcomp()`로 PCA를 간단하게 수행할 수 있다.

탐색적 자료분석

데이터 로드

[R 코드 5-23]

```
# 패키지 로딩
library(tidyverse)
library(ggplot2)
library(lubridate)

# 데이터 로드
subway <- read.csv(
  "데이터/seoul_subway_hourly_2024.csv",
  fileEncoding = "UTF-8",
  check.names = FALSE
)
```

이 데이터의 한 행은 특정 역 × 특정 날짜 × 승하차 구분에 대한 20차원 시간대별 인원 벡터이다. 승차 행만 필터한 뒤에는 역·날짜별 20차원 승차 패턴으로 해석한다.

전처리: 요일 및 평일/주말 변수 생성

요일별 패턴을 분석하기 위해 날짜 정보로부터 요일 변수와 평일/주말 구분 변수를 추가한다.

[R 코드 5-24]

```
# 범주형 변수 factor 변환
subway <- subway %>%
  mutate(across(c("호선", "역번호", "역명", "구분"), as.factor))

# 날짜 및 요일 변수 생성
korean_weekdays <- c(
  "월요일", "화요일", "수요일", "목요일", "금요일", "토요일", "일요일"
)

subway$날짜 <- ymd(subway$날짜)
subway$요일 <- factor(
```

```
korean_weekdays[lubridate::wday(subway$날짜, week_start = 1)],
levels = korean_weekdays
)

subway$평일_주말 <- factor(
  ifelse(subway$요일 %in% c("토요일", "일요일"), "주말", "평일")
)
```

lubridate 패키지의 wday() 함수는 날짜로부터 요일 번호를 추출한다. week_start = 1로 설정하면 월요일이 1번이 되어 한국식 요일 순서에 맞게 정렬할 수 있다. factor()로 순서를 지정해두면 이후 시각화에서 요일이 월~일 순서로 자동 정렬된다.

승차 데이터 추출

주성분분석은 수치형 변수에만 적용할 수 있다. 또한 승차와 하차 패턴이 서로 다를 수 있으므로 각각 따로 분석하는 것이 바람직하다. 여기서는 먼저 **승차 인원** 데이터만 추출하여 분석한다.

[R 코드 5-25]

```
# 승차 데이터 추출 및 수치형 변수(20개 시간대) 선택
subway_boarding <- subset(subway, 구분 == "승차")
subway_boarding_vars <- subway_boarding[, 7:26]

dim(subway_boarding_vars) # (관측치 수, 20)
```

추출된 데이터는 각 행이 특정 역·특정 날짜의 20개 시간대 승차 인원으로 구성된다.

주성분분석 실습

PCA 수행

R의 prcomp() 함수는 center = TRUE, scale. = TRUE 옵션을 지정하면 **표준화**를 내부적으로 자동 수행한다. 시간대마다 승차 인원의 절대적인 규모가 다를 수 있으므로 표준화는 필수적이다. 표준화 없이 PCA를 수행하면 인원이 많은 시간대가 분산을 지배하는 왜곡이 생긴다.

[R 코드 5-26]

```
# PCA 수행 (표준화 포함)
pca_result <- prcomp(subway_boarding_vars, center = TRUE, scale. = TRUE)
```

파이썬에서는 StandardScaler로 표준화를 별도로 수행해야 했지만, R의 prcomp()는 요인 하나로 표

준화와 PCA를 동시에 처리하므로 코드가 훨씬 간결하다.

스크리도표로 주성분 개수 선택

주성분의 개수를 결정하기 위해 **스크리도표(Scree Plot)**를 활용한다. 스크리도표는 각 주성분이 설명하는 분산의 크기를 순서대로 시각화한 것으로, 분산이 급격히 감소하다가 **완만해지는 지점(팔꿈치)**을 기준으로 주성분 개수를 선택한다.

[R 코드 5-27]

```
# 스크리도표 시각화
plot(pca_result, type = "lines", main = "스크리도표(Scree Plot)")
```

[그림 B-5-8] 스크리도표(Scree Plot)

[그림 B-5-8]의 스크리도표를 보면 세 번째 주성분 이후부터 분산 감소 폭이 완만해지는 것을 확인할 수 있다. 이는 앞의 두 개 주성분이 데이터의 대부분의 변동을 설명하고 있음을 시사한다. 따라서 주성분의 개수를 2개로 설정한다.

[R 코드 5-28]

```
# 주성분 2개 추출
subway_pc_df <- as.data.frame(pca_result$x[, 1:2])
colnames(subway_pc_df) <- c("PC1", "PC2")

# 누적 설명 분산 비율 확인
var_explained <- summary(pca_result)$importance
cat("PC1 설명 분산 비율:", round(var_explained[2, 1], 3), "\n")
cat("PC2 설명 분산 비율:", round(var_explained[2, 2], 3), "\n")
cat("누적 설명 분산 비율:", round(var_explained[3, 2], 3), "\n")

## PC1 설명 분산 비율: 0.631
## PC2 설명 분산 비율: 0.216
## 누적 설명 분산 비율: 0.847
```

누적 설명 분산 비율을 확인하면 PC1과 PC2 두 개만으로 전체 변동의 몇 퍼센트를 설명하는지 한눈에 파악할 수 있다.

주성분 해석

로딩>Loading) 확인

주성분의 의미를 해석하기 위해 각 변수의 **로딩>Loading)**을 확인한다. 로딩은 각 원래 변수(시간대)가 특정 주성분에 얼마나 기여하는지를 나타내는 값으로, 절댓값이 클수록 해당 주성분에 대한 영향력이 크다. R에서는 `prcomp()` 결과의 `$rotation` 속성에 로딩 행렬이 저장되어 있다.

[R 코드 5-29]

```
# 로딩 행렬 추출
loading_df <- as.data.frame(pca_result$rotation[, 1:2])
colnames(loading_df) <- c("PC1", "PC2")

# PC1, PC2 기여도 정렬
cat("=== PC1 기여도 ===\n")
print(loading_df %>% arrange(desc(abs(PC1))) %>% round(3))

cat("\n=== PC2 기여도 ===\n")
print(loading_df %>% arrange(desc(abs(PC2))) %>% round(3))

## === PC1 기여도 ===
##           PC1    PC2
## 15시-16시 -0.263 -0.106
## 13시-14시 -0.263 -0.007
## 14시-15시 -0.263 -0.073
## 16시-17시 -0.262 -0.122
## 12시-13시 -0.258  0.055
## ...
##
## === PC2 기여도 ===
##           PC1    PC2
## 07시-08시 -0.122  0.403
## 06시-07시 -0.134  0.403
## 08시-09시 -0.142  0.390
## 06시 이전 -0.139  0.360
## 09시-10시 -0.189  0.340
```

이를 선 그래프로 시각화하면 시간대별 패턴이 더욱 명확하게 드러난다.

[R 코드 5-30]

```
# 로딩 시각화
loading_df <- loading_df %>%
  mutate(시간대 = factor(rownames(loading_df), levels = rownames(loading_df)))

loading_df %>%
  pivot_longer(cols = c("PC1", "PC2"), names_to = "주성분", values_to = "로딩") %>%
  ggplot(aes(x = 시간대, y = 로딩, group = 주성분, color = 주성분)) +
  geom_line() +
  geom_point() +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "주성분별 시간대 로딩", x = "시간대", y = "로딩 값")
```

[그림 B-5-9] 주성분별 시간대 로딩

[그림 B-5-9]의 그래프를 해석하면 다음과 같다. PC1은 낮과 오후 시간대(10시~18시)에서 높은 로딩 값을 가지며, 전반적인 유동 인구 규모를 반영하는 축으로 볼 수 있다. PC2는 출근 시간대(7~9시)와 퇴근 시간대(18~20시)에서 서로 반대 방향의 로딩을 가지며, 전형적인 직장인 출퇴근 패턴을 요약하는 주성분으로 해석할 수 있다. 기본적으로 파이썬과 PC1 분포가 다른 것을 확인할 수 있는 데, 파이썬에서 구한 PC1과 서로 부호가 반대이기 때문에 그래프에서의 차이가 있다. R에서 구한 PC1에 -1을 곱하면 그 결과가 동일하며, 절대적인 크기는 동일하기 때문에 해석 또한 동일하다.

주성분 점수 시각화

이제 차원 축소된 PC1, PC2 값을 이용하여 관측치들의 분포를 살펴본다. 원래 데이터의 요일, 평일/주말, 호선, 역명 정보를 주성분 점수 데이터와 결합한다.

[R 코드 5-31]

```
subway_pc_df$요일 <- factor(
  subway_boarding$요일,
  levels = korean_weekdays
)
subway_pc_df$평일_주말 <- subway_boarding$평일_주말
subway_pc_df$호선 <- factor(subway_boarding$호선)
subway_pc_df$역명 <- factor(subway_boarding$역명)
```

```
weekday_df <- subset(subway_pc_df, 평일_주말 == "평일")
weekend_df <- subset(subway_pc_df, 평일_주말 == "주말")
```

평일과 주말로 명확히 구분하여 한 그래프에 나타내면 두 집단의 차이가 뚜렷하게 드러난다.

[R 코드 5-32]

```
# 평일/주말 산점도

ggplot() +
  geom_point(
    data = weekday_df,
    aes(PC1, PC2, color = 평일_주말, shape = 평일_주말),
    alpha = 0.6
  ) +
  geom_point(
    data = weekend_df,
    aes(PC1, PC2, color = 평일_주말, shape = 평일_주말),
    alpha = 0.6
  ) +
  labs(title = "평일·주말 승차량 분포 비교") +
  theme_minimal()
```

[그림 B-5-10] 평일·주말 승차량 분포 비교

[그림 B-5-10]에서 PC2 값이 양의 방향으로 클수록 평일 패턴이 두드러지게 나타난다. 이는 출퇴근 시간대 승차량이 많은 평일의 특성이 PC2에 반영되었기 때문으로 해석할 수 있다. 반면 주말 관측치들은 PC2 값이 낮고 PC1 값이 넓게 분포하는 경향을 보이는데, 이는 주말에는 출퇴근 패턴 없이 낮 시간대 전반에 걸쳐 다양한 유동 인구 패턴이 나타나기 때문이다.

데이터 분석가의 한마디

PCA는 데이터를 "압축"하는 도구이기도 하지만, 동시에 데이터 속에 숨겨진 패턴을 발견하는 **탐색적 분석 도구**이기도 하다. 이번 분석에서 우리는 20개의 시간대 변수를 단 2개의 주성분으로 줄였음에도 불구하고, 평일과 주말의 승차 패턴 차이를 뚜렷하게 시각화할 수 있었다. 파이썬에서는 표준화와 PCA를 별도로 수행해야 하지만, R의 `prcomp()`는 `scale. = TRUE` 옵션 하나로 두 과정을 동시에 처리해 코드가 더욱 간결하다는

장점이 있다. 다만 PCA는 원래 변수들의 선형결합으로만 주성분을 구성하기 때문에, 비선형적인 구조가 강한 데이터에서는 한계가 있다. 이런 경우에는 t-SNE, UMAP 등 비선형 차원 축소 기법을 함께 고려해 볼 수 있다.

B.5.4 차원 축소: 요인분석

요인분석(Factor Analysis, FA)은 여러 개의 변수들을 소수의 **공통 요인(Common Factor)**로 요약하는 차원 축소 기법이다. 주성분분석(PCA)이 변수들의 전체 분산을 최대한 보존하는 새로운 축을 찾는 것이라면, 요인분석은 변수들 사이의 **상관관계 구조**를 설명하는 잠재적인 공통 원인을 발견하는데 초점을 맞춘다. 예를 들어 설문조사에서 여러 항목에 대한 응답이 비슷한 패턴을 보인다면, 그 배경에 하나의 공통 요인이 있을 수 있다. R에서는 psych 패키지의 fa() 함수를 사용한다.

데이터 소개

이번 분석에서는 K공기업 임직원 510명을 대상으로 수집한 설문조사 데이터를 활용한다. 12개 문항은 외재적·내재적 보상 선호와 관련된 응답을 담고 있다.

요인분석에 사용할 변수는 **선호 보상방식**에 관한 12개 설문 문항이다. 각 문항은 5점 리커트 척도로 측정되었으며, 이론적으로 아래 세 가지 보상 유형으로 구분된다.

보상 유형	변수명	설명
외재적 보상	pay	금전적 보수(기본급, 성과급 등) 중요도
	welfare	금전적 복리후생(주택자금, 의료비 지원 등) 중요도
	training	교육훈련 기회(학위 취득, 외부연수 등) 중요도
	position	원하는 보직·인사이동 중요도
	work	유연근무·휴가 등 근무여건 중요도
내재적 보상	achievement	직무 성취감 중요도
	responsibility	직무 책임감 중요도

	interest	직무 흥미성 중요도
	pride	조직에 대한 자부심 중요도
	recognition	조직 내 인정 중요도
복합적 보상	bonding	조직 내 유대감 중요도
	promotion	승진 중요도

요인분석의 목적은 이 12개 문항이 이론적 구조(내재적·외재적·복합적 보상)에 실제로 부합하는지 타당도를 검토하는 것이다.

탐색적 자료분석

데이터 로드

[R 코드 5-33]

```
# 패키지 로딩
library(tidyverse)
library(psych)
library(corrplot)
library(ggrepel)

# CSV 데이터 로드
k_survey <- read.csv("데이터/k_corporation_reward_survey.csv")
```

psych 패키지는 심리측정학적 분석에 특화된 패키지로, 요인분석에 필요한 fa() 함수와 다양한 진단 도구를 제공한다. skip = 2는 엑셀 파일의 첫 두 행을 건너뛰고 데이터를 읽는 옵션이다.

분석 변수 추출 및 표준화

요인분석에 사용할 선호 보상방식 관련 12개 변수(10~21번 열)만 추출하고 표준화한다.

[R 코드 5-34]

```
# CSV의 12개 열이 모두 보상 설문 문항이다.
survey_vars <- k_survey
```

```
# 변수 표준화
survey_vars <- as.data.frame(scale(survey_vars))
```

상관행렬 시각화

변수들 간에 충분히 높은 상관관계가 있어야 의미 있는 요인 구조가 존재한다. 분석 전에 상관행렬을 시각화하여 변수들의 구조를 파악한다.

[R 코드 5-35]

```
# 상관행렬 계산 및 시각화
corr_matrix <- cor(survey_vars, use = "pairwise.complete.obs")
corrplot(corr_matrix, method = "color")
```

[그림 B-5-11] 선호 보상방식 변수 간 상관계수 행렬

[그림 B-5-11]의 히트맵을 살펴보면 achievement(성취감), responsibility(책임감), recognition(조직 내 인정) 등 내재적 보상 변수들끼리 높은 상관관계를 보이고, pay(보수), welfare(복리후생), work(근무여건) 등 외재적 보상 변수들끼리도 서로 높은 상관관계를 나타낸다. 이는 두 개의 뚜렷한 요인 구조가 존재할 가능성을 시사한다.

요인분석 실습

요인분석 수행

요인의 개수를 **2개**로 설정하고, 요인 회전 방식으로 **Varimax**를 사용하여 요인분석을 수행한다. Varimax 회전은 각 변수가 가능한 한 하나의 요인에만 높은 적재값을 갖도록 만들어 요인의 의미를 더 명확하게 해석할 수 있게 한다.

[R 코드 5-36]

```
# 요인분석 수행 (요인 수 = 2, Varimax 회전)
n_factors <- 2

survey_fa <- fa(survey_vars, nfactors = n_factors,
               rotate = "varimax", scores = "regression")

# 각 요인의 설명력 확인
```

```
survey_fa$Vaccounted
```

출력 결과를 보면 F1은 전체 분산의 약 **24.5%**, F2는 약 **16.5%**를 설명하며, 두 요인을 합산한 누적 설명 분산 비율은 약 **41.0%**이다. 즉, 12개의 설문 문항에서 두 가지 핵심 요인만으로 전체 변동의 약 41%를 요약할 수 있다.

요인 적재값>Loading) 확인

요인 적재값은 각 변수가 특정 요인에 얼마나 강하게 연결되어 있는지를 나타낸다. 절댓값이 클수록 해당 요인에 대한 영향력이 크다.

[R 코드 5-37]

```
# 요인 적재값 행렬 추출
survey_fa_loadings <- as.data.frame(survey_fa$loadings[, 1:n_factors])
colnames(survey_fa_loadings) <- paste0("F", 1:n_factors)

# F1 기준 내림차순 정렬
cat("=== F1 적재값 ===\n")
print(survey_fa_loadings %>% arrange(desc(F1)) %>% round(3))

# F2 기준 내림차순 정렬
cat("\n=== F2 적재값 ===\n")
print(survey_fa_loadings %>% arrange(desc(F2)) %>% round(3))
```

F1의 적재값을 살펴보면 achievement(직무 성취감, 0.791), recognition(조직 내 인정, 0.752), responsibility(직무 책임감, 0.745), interest(직무 흥미성, 0.603), pride(조직 자부심, 0.495) 순으로 높다. 이는 직무 내면적 동기를 반영하므로 F1을 '내재적 보상 선호' 요인으로 해석할 수 있다.

F2는 welfare(복리후생, 0.689), pay(보수, 0.600), work(근무여건, 0.572), position(보직 및 인사이동, 0.556) 등 외재적 보상 항목의 적재값이 높다. 따라서 F2를 '외재적 보상 선호' 요인으로 해석할 수 있다.

요인 회전 전후 비교 시각화

요인 회전이 요인의 해석을 얼마나 명확하게 만드는지 시각적으로 확인해 보자. 회전 전후 각 변수의 요인 적재값을 산점도로 비교한다.

[R 코드 5-38]

```
# 회전 전 요인분석 추가 적합
survey_fa_none <- fa(
  survey_vars, nfactors = 2, rotate = "none", scores = "regression"
)

L_before <- as.data.frame(unclass(survey_fa_none$loadings))[, 1:2]
L_after <- as.data.frame(unclass(survey_fa$loadings))[, 1:2]
colnames(L_before) <- colnames(L_after) <- c("F1", "F2")

plot_df <- rbind(
  data.frame(
    F1 = L_before$F1,
    F2 = L_before$F2,
    var = rownames(L_before),
    group = "회전 전"
  ),
  data.frame(
    F1 = L_after$F1,
    F2 = L_after$F2,
    var = rownames(L_after),
    group = "회전 후"
  )
)

ggplot(plot_df, aes(x = F1, y = F2, color = group, label = var)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
  geom_point(size = 2) +
  geom_text_repel(size = 3) +
  coord_cartesian(xlim = c(-1, 1), ylim = c(-1, 1)) +
  scale_color_manual(values = c("회전 전" = "#6D9EC1", "회전 후" = "#163657")) +
  labs(x = "요인 1", y = "요인 2", color = NULL) +
  theme_bw()
```

[그림 B-5-12] 요인 회전 전후 비교

회전 전에는 변수들이 두 축 사이에 뭉쳐 있어 어떤 변수가 어떤 요인에 속하는지 구분하기 어렵다. 반면 Varimax 회전 후에는 외재적 보상 변수(pay, work, position, welfare)가 F2 방향으로, 내재적 보상 변수(achievement, responsibility, recognition)가 F1 방향으로 명확하게 분리된다. 흥미로운 점은 복합적 보상인 promotion(승진)이 F1(내재적 보상) 방향에 위치한다는 것으로, 응답자들이 승진을 외재적 보상보다는 내재적 동기와 더 관련지어 인식하고 있음을 시사한다.

요인 점수(Factor Score) 시각화

요인 점수는 각 관측치(응답자)가 두 요인에서 어떤 위치에 있는지를 나타내는 값이다. 이를 산점도로 시각화하면 응답자들의 보상 선호 패턴을 직관적으로 파악할 수 있다.

[R 코드 5-39]

```
# 요인 점수 추출
survey_factor_df <- as.data.frame(survey_fa$scores)
colnames(survey_factor_df) <- c("F1", "F2")

# promotion 한 문항을 기준으로 승진 선호 여부 구분
survey_factor_df$reward_mix <- factor(
  ifelse(k_survey$promotion >= 3, 1, 0)
)

ggplot(survey_factor_df, aes(x = F1, y = F2, color = reward_mix)) +
  geom_point(alpha = 0.6) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "green") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "green") +
  scale_color_manual(
    values = c("0" = "#6D9EC1", "1" = "#163657"),
    labels = c("0" = "승진 비선호", "1" = "승진 선호")
  ) +
  labs(
    title = "요인 점수 산점도 (승진 선호 여부)",
    x = "F1 (내재적 보상 선호)",
    y = "F2 (외재적 보상 선호)",
    color = NULL
  ) +
  theme_bw()
```

[그림 B-5-13] 요인 점수 산점도

산점도를 살펴보면 F2(외재적 보상 선호) 방향으로 두 집단이 고르게 분포하지만, F1(내재적 보상 선호) 방향에서는 차이가 뚜렷하게 나타난다. 승진을 선호하는 응답자(1)들은 F1의 양의 방향에 더 많이 위치하고, 승진을 덜 중요하게 여기는 응답자(0)들은 음의 방향에 더 많이 분포한다. 이는 앞서 확인한 적재값 분석 결과와 일치하는 것으로, **승진에 대한 선호가 외재적 보상보다는 내재적 동기 구조와 더 밀접하게 연관되어 있음**을 보여준다.

데이터 분석가의 한마디

이번 요인분석의 결과는 기존 이론적 구분(외재적/내재적/복합적 보상)과 대체로 일치하지만, 흥미로운 예외도 발견된다. '승진'(promotion)은 이론상 복합적 보상으로 분류되지만, 실제 응답 데이터에서는 내재적 보상 요인(F1)에 더 가깝게 위치했다. 이는 K공기업 임직원들이 승진을 단순한 외재적 보상(급여 인상 등)이 아니라 조직 내 인정이나 성취감과 같은 내면적 동기와 연관 지어 인식하고 있음을 시사한다. 이처럼 요인분석은 데이터에서 이론이 예상하지 못한 패턴을 발견하는 데에도 매우 유용한 도구다.

B.5.5 군집분석

군집분석(Clustering)은 정답(Label)이 주어지지 않은 자료에서 관측치들 간의 유사성을 바탕으로 비슷한 개체들을 그룹으로 묶는 대표적인 **비지도 학습(Unsupervised Learning)** 방법이다. 분석 목적에 따라 비슷한 특성을 가진 집단을 발견하거나 데이터 전체의 패턴을 탐색하는 데 폭넓게 활용된다. R에서는 kmeans()와 hclust() 함수로 두 가지 대표적인 군집분석을 간결하게 수행할 수 있다.

사전 준비**[R 코드 5-40]**

```
# 패키지 로딩
library(tidyverse)
library(ggplot2)
```

K-평균 군집분석

K-평균 알고리즘은 전체 관측치를 데이터 분석가가 미리 지정한 k 개의 군집으로 나누는 방법이다. 각 군집은 중심점(Centroid)을 가지며, 모든 관측치는 가장 가까운 중심점에 속한 군집에 할당된다.

이후 할당된 데이터를 바탕으로 중심점을 다시 계산하고, 관측치를 재할당하는 과정을 반복하면서 **군집 내 거리 제곱합**을 최소화하는 방향으로 수렴한다.

데이터 준비 및 표준화

PCA 실습에서 사용했던 서울시 지하철 승차 인원 데이터(subway_boarding_vars)를 그대로 사용한다. K-평균은 거리 기반 방법이므로, 출퇴근 시간대처럼 절대적인 승차량이 큰 시간대가 군집을 지배하는 왜곡을 막기 위해 반드시 **표준화**가 필요하다.

[R 코드 5-41]

```
# 표준화
subway_boarding_scaled <- scale(subway_boarding_vars)
subway_boarding_scaled <- as.data.frame(subway_boarding_scaled)
```

군집모형 적합

[R 코드 5-42]

```
# K-평균 군집분석 (K=8)
set.seed(0)
k <- 8

kmeans_result <- kmeans(subway_boarding_scaled, centers = k)

# 각 관측치의 군집 라벨 확인 (앞 5개)
kmeans_result$cluster[1:5]
```

[그림 B-5-14] K-평균 군집 결과(PCA 2차원 시각화)

kmeans() 함수는 centers 인자로 군집 수 K 를 지정하면 바로 실행된다. set.seed()로 초기 중심점의 무작위 배치를 고정하면 분석을 반복해도 동일한 결과를 재현할 수 있다. 실행 결과 각 관측치에는 1부터 8까지의 군집 라벨이 부여된다.

군집 결과 시각화

승차 데이터는 20차원이므로 2차원으로 직접 시각화할 수 없다. PCA 실습에서 이미 계산해 둔 subway_pc_df에 군집 라벨을 추가하고 산점도로 확인한다.

[R 코드 5-43]

```
# PCA 결과에 군집 라벨 추가
subway_pc_df$clusterlabel <- factor(kmeans_result$cluster)

ggplot(subway_pc_df, aes(x = PC1, y = PC2, color = clusterlabel)) +
  geom_point(alpha = 0.6, size = 1.5) +
  labs(
    title = "역·날짜별 K-평균 군집 결과 (PCA 2차원 시각화)",
    color = "군집"
  ) +
  theme_minimal()
```

[그림 B-5-14]는 PC1과 PC2를 기준으로 유사한 시간대별 승차 패턴을 보이는 역·날짜 관측치가 같은 군집으로 묶이는 양상을 보여 준다. 군집 대상은 지하철역 자체가 아니라 역·날짜별 승차 패턴이다.

데이터 분석가의 한마디

K-평균의 가장 큰 고민은 K 를 **몇으로 설정할 것인가**이다. 실습에서는 8로 지정했지만, 실제 분석에서는 K 를 1부터 늘려가며 군집 내 거리 제곱합이 급격히 감소하다가 완만해지는 지점(팔꿈치 지점)을 찾는 **엘보우 방법(Elbow Method)**을 자주 활용한다. 또한 K-평균은 초기 중심점에 따라 결과가 달라질 수 있고, 구형(Spherical) 군집이 아닌 복잡한 형태의 데이터에서는 성능이 떨어질 수 있다는 점도 함께 고려해야 한다.

계층군집화

계층군집화는 각 관측치를 개별 군집으로 시작하여 가장 가까운 군집끼리 단계적으로 병합해 나가는 **상향식(Bottom-up)** 방법이다. K-평균과 달리 K 를 미리 지정할 필요가 없으며, 병합 과정을 **덴드로그램(Dendrogram)**으로 시각화하여 적절한 군집 수를 사후에 결정할 수 있다는 것이 가장 큰 장점이다.

데이터 준비 및 표준화

계층군집화 실습에는 자동차의 기술 제원을 담은 mtcars 데이터를 사용한다. R의 대표적인 데이터셋을 데이터/mtcars.csv로 함께 제공하며, 32개 자동차 모델에 대해 연비, 마력, 무게 등 11개의 수치형 변수가 기록되어 있다.

변수명	설명
mpg	연비 (Miles/gallon)
cyl	실린더 수
disp	배기량 (cu.in.)
hp	마력
drat	후륜 기어비
wt	차량 무게 (1000 lbs)
qsec	1/4마일 주파 시간 (sec)
vs	엔진 형태 (0=V형, 1=직렬)
am	변속기 (0=자동, 1=수동)
gear	전진 기어 수
carb	카뷰레터 수

[R 코드 5-44]

```
# mtcars 데이터 로드 및 표준화
mtcars <- read.csv("데이터/mtcars.csv")

mtcars_scaled <- scale(mtcars[, setdiff(names(mtcars), "model")])
mtcars_scaled <- as.data.frame(mtcars_scaled)
```

R에서도 다른 실습과 같은 경로 규칙을 적용하여 `read.csv("데이터/mtcars.csv")`로 불러온다.

계층군집화 모형 적합

R에서는 `dist()`로 거리 행렬을 계산한 뒤 `hclust()`로 계층군집화를 수행한다. `cutree()`를 이용하면 원하는 군집 수로 관측치를 분할할 수 있다. 연결법은 파이썬과 동일하게 **Ward 방법**(`ward.D2`)을 사용한다.

[R 코드 5-45]

```
# 거리 행렬 계산 및 Ward 계층군집화 후 3개 군집으로 분할
```

```
dist_matrix <- dist(mtcars_scaled)
hc_mtcars <- hclust(dist_matrix, method = "ward.D2")
ward_clusters <- cutree(hc_mtcars, k = 3)
```

덴드로그램 시각화 및 해석

R에서는 `plot(hclust 객체)`만으로 덴드로그램을 바로 그릴 수 있다. `rect.hclust()`를 추가하면 지정한 군집 수에 해당하는 영역을 색깔 박스로 표시할 수 있어 군집의 경계를 직관적으로 확인할 수 있다.

[R 코드 5-46]

```
# 덴드로그램 시각화 및 군집 박스 표시
plot(hc_mtcars, main = "덴드로그램 (Ward 연결법)",
     xlab = "자동차 모델", ylab = "거리", cex = 0.7)
rect.hclust(hc_mtcars, k = 3, border = 2:4)
```

[그림 B-5-15] 계층군집화 덴드로그램(Ward 연결법)

파이썬에서는 `scipy`로 병합 행렬을 별도 계산해야 했지만, R에서는 `hclust()` 결과 객체를 그대로 `plot()`에 전달하면 되므로 훨씬 간결하다.

[그림 B-5-15]의 덴드로그램은 32개 자동차 모델이 서로 유사한 순서대로 단계적으로 병합되어 가는 전체 과정을 보여준다. 가장 아래쪽에서는 "Mazda RX4"와 "Mazda RX4 Wag"처럼 제원이 거의 동일한 모델들이 먼저 합쳐진다. 거시적으로는 전체 데이터가 소형·고연비 차량 군집(Honda Civic, Fiat 128, Toyota Corolla 등)과 대형·고성능 차량 군집으로 크게 양분된다. 대형차 군집 내부에서는 다시 Lincoln Continental, Cadillac Fleetwood 등의 대형 세단과 Maserati Bora, Ford Pantera L 등의 고성능 스포츠카로 세분화되는 2단계 계층 구조를 확인할 수 있다.

데이터 분석가의 한마디

K-평균과 계층군집화는 서로 다른 장단점을 가진다. K-평균은 대용량 데이터에서 빠르게 작동하지만 K 를 미리 정해야 하고 초기값에 민감하다. 반면 계층군집화는 덴드로그램을 통해 데이터의 계층 구조를 한눈에 파악할 수 있고 K 를 사후에 결정할 수 있다는 장점이 있지만, 관측치 수가 많아질수록 계산 시간이 급격히 증가한다. 실제 분석에서는 계층군집화 덴드로그램으로 적절한 군집 수를 파악한 뒤, 그 K 값으로 K-평균을 수행하는 방식을 자주 활용한다.